

26. 在一个连接中, *cwnd*的值是3 000而*rwnd*的值是5 000。主机已发送2 000个字节,但它们还未确认,试问还有多少个字节可以发送?
27. TCP用初始序列号(ISN) 14 534打开一个连接,而另一方用ISN为21 732打开该连接。试表示连接建立期间的这三个TCP段。
28. 一个客户使用TCP将数据发送给一个服务器,数据共有16个字节。试计算在TCP级的传输效率(有用字节与总字节之比)。计算在IP级的传输效率,假定IP头部无选项。计算在数据链路层传输的效率,假定IP头部无选项且在数据链路层使用以太网。
29. TCP发送数据的速率是每秒1M字节。如果序列号从7 000开始,试问经过多长时间序列号又回到0?
30. 一个TCP连接使用10 000字节的窗口大小,而上一次的确认号是22 011。它接收到一个确认号是22 001的段并且通告窗口大小为12 000。试用图说明窗口情况前后的变化情况。
31. 一个窗口保存了字节2 001到5 000,下一个要发送的字节是3 001。试用图说明下列两个事件之后窗口的情况:
 - a. 接收到带有确认号为2 500的ACK段并且窗口通告大小为4 000;
 - b. 携带1 000个字节的段被发送。
32. 在SCTP中, SACK的*cumTSN*值是23, SACK上次的*cumTSN*值是29,试问这是什么问题?
33. 在SCTP中,接收方的状态如下:
 - a. 接收队列中有大块1到8、11到14以及16到20;
 - b. 在队列中有1 800个字节为空;
 - c. *lastACK*的值是4;
 - d. 没有接收到重复的大块;
 - e. *cumTSN*的值是5。
 试说明接收队列的内容及其变量。
34. 在SCTP中,发送方的状态如下:
 - a. 发送队列中有大块18到23;
 - b. *cumTSN*的值是20;
 - c. 窗口大小值是2 000个字节;
 - d. *inTransit*的值是200。
 如果每一个数据大块包含100个字节的数据,现在能有多少个DATA大块可发送?下一个DATA大块是什么?

探究题

35. 寻找有关ICANN更多的信息,在这名称之前它称做什么?
36. TCP用一个状态转换图处理段的发送与接收。画出该图并说明它如何处理流量与控制。
37. STCP用一个状态转换图处理段的发送与接收。画出该图并说明它如何处理流量与控制。
38. 在TCP中,半打开情形是怎样?
39. 在TCP中,半双工关闭情形是怎样?
40. 在UNIX或LINUX中, *tcpdump*命令可用来打印出一个网络接口的一些分组的头部信息。用*tcpdump*命令还可以查看已发送和接收的段。
41. 在STCP中,如果一个SACK大块被延迟或遗失,将会发生什么事?
42. 查明TCP中所用的计时器的名称和功能。
43. 查明STCP中所用的计时器的名称和功能。
44. 在SCTP中查找有关ECN更多的信息。查明这两个大块的格式。
45. 有些应用程序,如FTP,当使用TCP时需要多个连接。查明SCTP多流服务如何帮助这些应用程序用多个流仅建立一个关联。

第24章 拥塞控制和服务质量

拥塞控制和服务质量是紧密联系在一起的两个问题：改进了其中的一个问题，则另一个问题也会有所改善；忽视了其中的一个问题，则通常意味着另一个也被忽视。在一个网络中，大多数防止或消除拥塞的技术也能改进网络的服务质量。

之所以推迟到现在才讨论这些问题，是因为这些问题不止涉及到一层，而是涉及到三层：数据链路层、网络层和传输层。而且现在才讨论它们也是为了能一次性地对这些问题做集中的探讨，而不需要多次讨论同一个主题。本章也给出了一些不同层中的拥塞控制和服务质量的示例。

24.1 数据通信量

对拥塞控制和服务质量关注的重要方面是数据通信量。在拥塞控制中，要设法避免通信量拥塞。在服务质量中，要设法为通信量创造合理的环境。因此，在讨论拥塞控制和服务质量之前，首先讨论数据通信量本身。

24.1.1 通信量描述符

通信量描述符是描述数据流的定性值。图24.1用这些值描述了通信量。

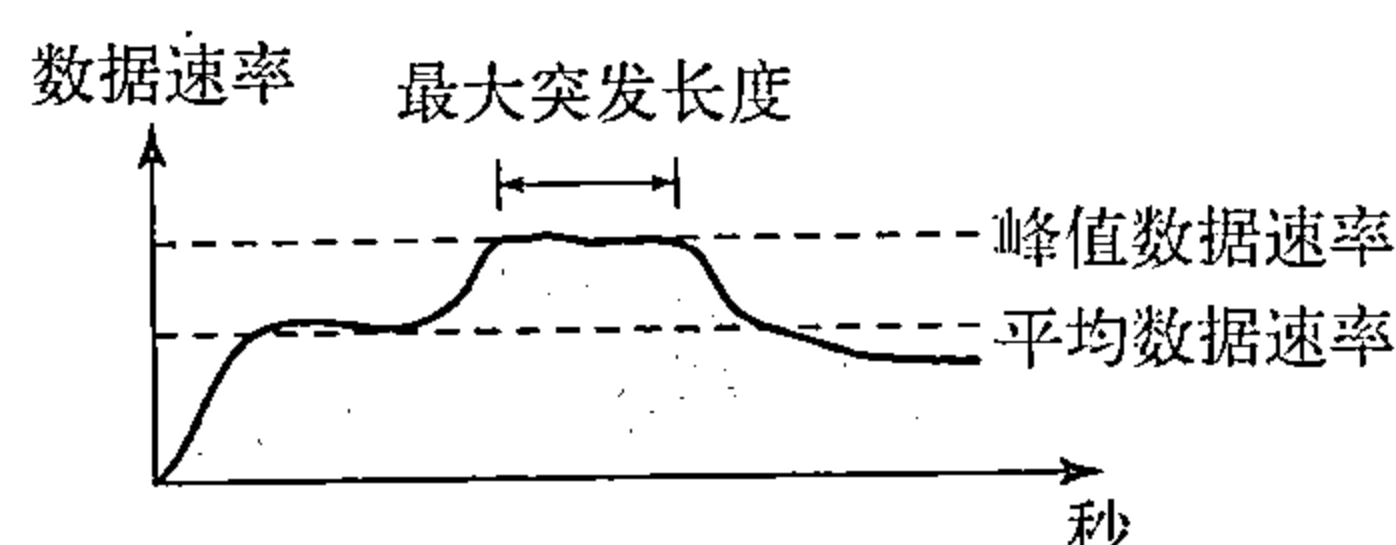


图24.1 通信量描述符

平均数据速率

平均数据速率（average data rate）是在一段时间内发送的数据位数除以该时段所含的秒数。我们使用如下的等式：

$$\text{平均数据速率} = \frac{\text{数据量}}{\text{时间}}$$

平均数据速率是通信量的一个非常有用的特性，因为它表明通信量所需要的平均带宽。

峰值数据速率

峰值数据速率（peak data rate）定义了通信量的最大数据速率，如图24.1所示，它是y轴方向上的最大值。峰值数据速率是一个很重要的概念，因为它表明让通信量通过网络而且无须改变数据流的情况下，网络所需的峰值带宽。

最大突发长度

虽然峰值数据速率是很关键的，但是如果峰值持续时间很短，那么它通常可忽略不计。例如，如果数据正以1Mbps的速率稳定地传送，而突然以2Mbps的峰值数据率传送了1ms，网络也许可以处理这种情况。但是如果峰值速率持续60ms，那么网络可能会出问题。最大突发长度（maximum burst size）一般是指以峰值速率传输通信量所持续时间的最大值。

有效带宽

有效带宽（effective bandwidth）是指网络需要分配给通信流的带宽。有效带宽是三个值的函数：平均数据速率、峰值数据速率和最大突发长度。有效带宽的计算过程非常复杂。

24.1.2 通信量特征值

根据需要，数据流可以用以下通信量特征值之一来表述：恒定比特率、可变比特率或突发性。如图24.2所示。

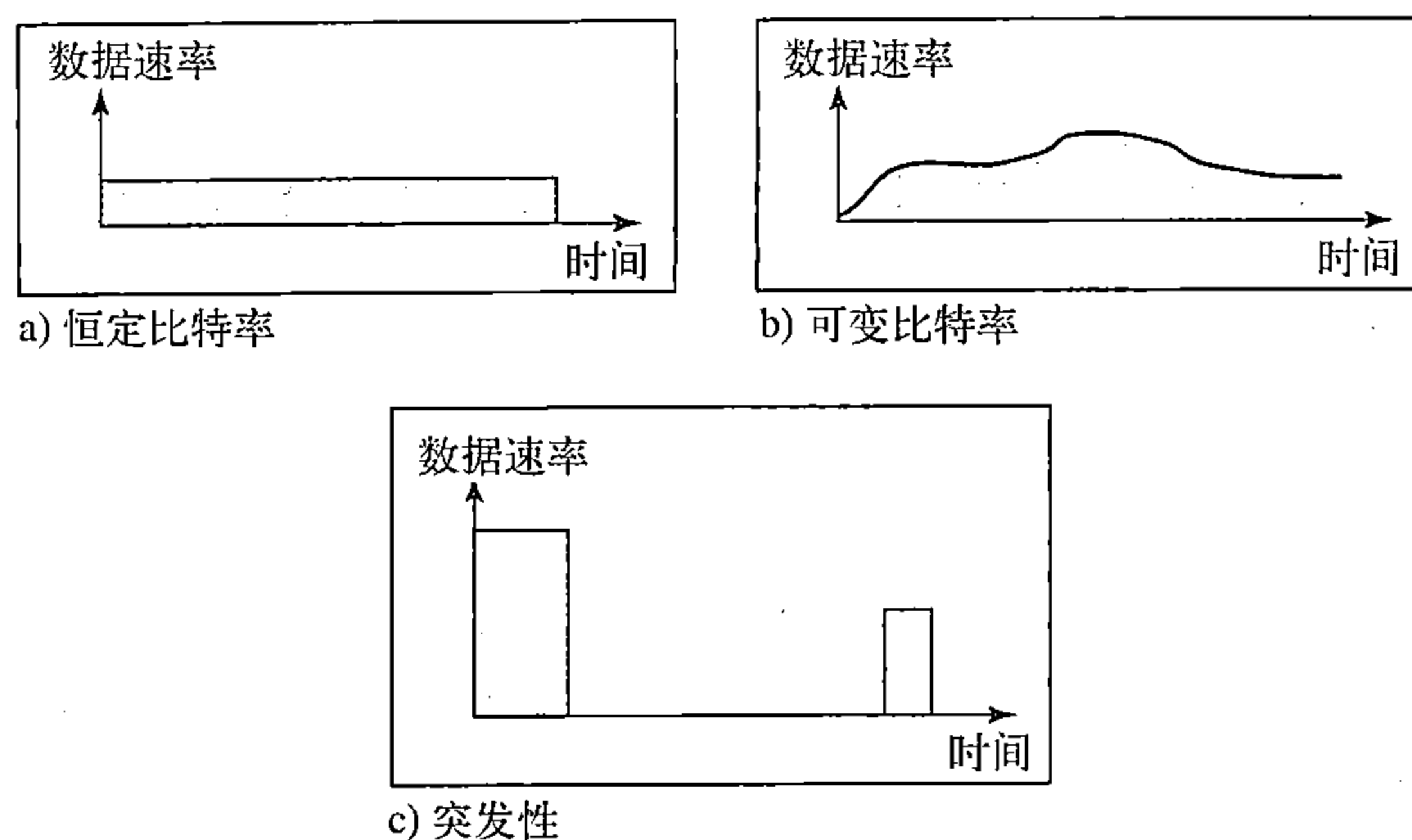


图24.2 三个通信量特征值

恒定比特率

恒定比特率 (constant-bit-rate, CBR) 或固定速率的通信量模型具有恒定不变的数据速率。在这种类型的流中，平均数据速率和峰值数据速率是相同的。最大突发长度是无效的。对于一个网络而言，这种类型的通信量是可预知的，因而很容易处理。网络可以提前知道分配多少带宽给这种类型的流。

可变比特率

在**可变比特率** (variable-bit-rate, VBR) 类型中，数据流的速率随时间发生平滑的、而不是突然的或急剧的变化。在这种类型的流中，平均数据速率和峰值数据速率是不同的。最大突发长度通常是一个很小的值。与恒定比特率的通信量相比，这种类型的通信量更难处理，但是它一般不需要重新整形，这将在后面可以看到。

突发性数据

突发性数据 (bursty data) 是指在很短的时间内数据速率突然发生了变化。例如，它可能在几微秒内从零猛增到1Mbps，或从1Mbps猛降为零。它也可能在这个值上保持一会儿。在这种类型的数据流中，平均比特率和峰值比特率非常不同。最大突发长度非常重要。对于网络，这是最难处理的一种通信量类型，因为其特征值是不可预知的。为了处理这种类型的通信量，网络通常需要使用重新整形技术对其进行重新整形。这种技术稍后将会看到。突发性通信量是网络中发生拥塞的主要原因之一。

24.2 拥塞

拥塞 (congestion) 是分组交换网络中的一个重要问题。如果网络中的载荷 (load)，即发送到网络中的分组数量，超过了网络的容量，即网络中能处理的分组数量，那么在网络中就可能发生拥塞。**拥塞控制** (congestion control) 指的是控制拥塞和使载荷低于网络容量的机制和技术。

读者可能会问，为什么在网络中会发生拥塞。拥塞可能发生在任何涉及等待机制的系统之中。例如，发生在高速公路上的拥塞是因为车流的任何异常，如在高峰期的事故，都可能

造成交通阻塞。

在网络或者互联网络中也会发生拥塞，这是因为路由器和交换机中存在队列——在处理前后保存分组的缓冲区。例如，路由器的每个接口中都有输入队列和输出队列。当分组到达输入接口时，在将其转发前要经历三个步骤，如图24.3所示。

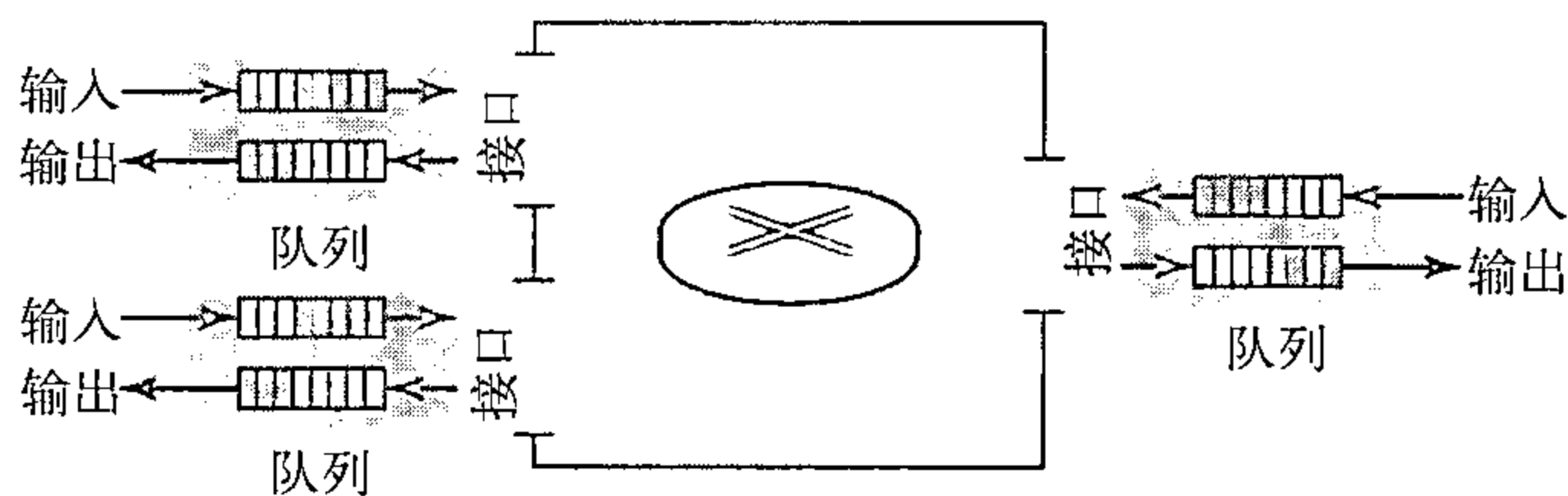


图24.3 路由器中的队列

1. 在等待检验时，将分组置于输入队列的尾部。

2. 一旦分组到达了队列的前端，路由器的处理模块就将其从输入队列中移除，并运用路由表和目的地址来查找其路由。

3. 将分组置于适当的输出队列中，等待发送。

这里需要注意两个问题。第一，如果分组的到达速率高于分组的处理速率，那么输入队列会变得越来越大。第二，如果分组的转发速率低于分组的处理速率，输出队列也会变得越来越长。

网络的性能

拥塞控制包括了两个测试网络性能的要素：延迟和吞吐量。图24.4说明了作为载荷函数的这两个性能度量。

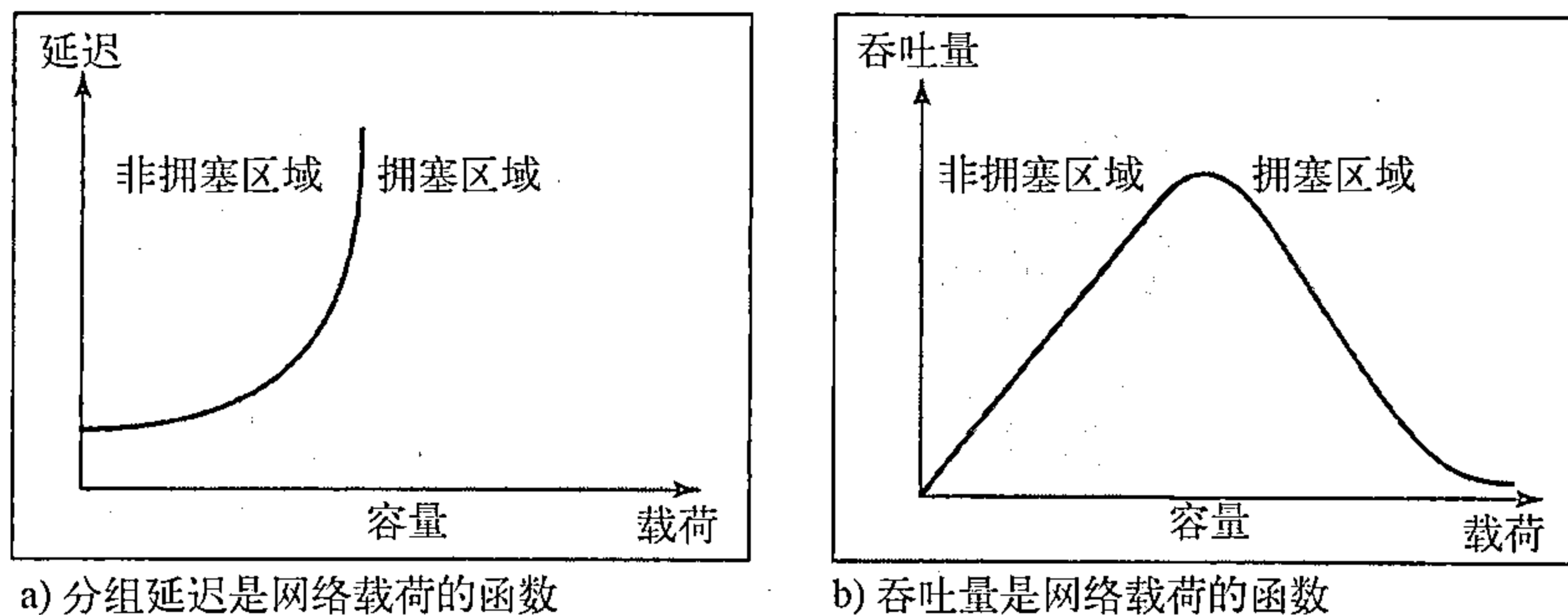


图24.4 分组延迟和吞吐量是网络载荷的函数

延迟和载荷

注意，当载荷比网络容量小得多时，延迟（delay）最小。最小延迟是由传播延迟和处理延迟所组成的，并且它们都可以忽略不计。然而，当载荷达到网络容量时，延迟就会急剧增加，因为现在需要将等待在队列（对路径中所有的路由器）中的时间加到总的延迟中。注意，当载荷大于网络容量时，延迟会变为无穷大。如果这还不能说明问题，那么就请考虑一下在几乎没有分组到达目的端或经过无穷大的延迟到达目的端时队列的长度；此时队列变得越来越长。延迟会加重载荷，并因此会导致拥塞现象发生。当分组被延迟时，源端就会在没有收到确认的情况下重发分组，这个过程则会加重延迟和拥塞。

吞吐量和载荷

在第3章中，我们将吞吐量定义为在1秒内通过一个节点的位数。可以将位换成分组，将

节点换成网络，来扩展这个定义，从而将网络的吞吐量（throughput）定义为单位时间内通过网络的分组数量。注意，当载荷小于网络容量时，吞吐量随载荷的增加成比例地增长。当载荷达到网络容量后，期望吞吐量保持恒定。但是实际情况恰好相反，吞吐量会急剧下降。原因是路由器丢弃了分组。当载荷超过网络容量时，队列饱和了，此时路由器不得不丢弃一些分组。丢弃分组并不能减少网络中分组的数量，因为当分组没有到达目的端时，源端就运用超时机制重发它们。

24.3 拥塞控制

拥塞控制是指在拥塞发生之前预防拥塞、或在拥塞发生之后消除拥塞的技术和机制。通常，我们把拥塞控制分为两大类：开环拥塞控制（预防）和闭环拥塞控制（消除）。如图24.5所示。

24.3.1 开环拥塞控制

在开环拥塞控制（open-loop congestion control）中，在拥塞发生之前，应用某种策略来预防拥塞现象的发生。在这些机制中，源端或目的端都可以处理拥塞控制。下面简要列出能预防拥塞的几种策略。

重传策略

有时重发是不可避免的。如果发送方认为一个发送分组丢失或损坏，则该分组就需要重发。重发一般在网络上会增加拥塞现象，然而一个好的重传策略能预防拥塞。必须优化设计重传策略和重传定时器，使之具有高效率，并用来预防拥塞。例如，TCP所用的重传策略（稍后说明）的设计是防止和减轻拥塞。

窗口策略

发送方窗口的类型也会影响拥塞。对拥塞控制而言，选择性重复窗口要优于回退N帧窗口。在回退N帧窗口中，当一个分组的计时器到时，可以重发多个分组。虽然有一些可安全与可靠地到达接收方，但这种重复会使拥塞更加严重。而选择性重复窗口试图发送那些被丢失或损坏的特定分组。

确认策略

接收方使用的确认策略也可能影响拥塞。如果接收方并不对它所接收的每一个分组进行确认，则它会使发送方放慢发送速度，从而有助于预防拥塞。有些方法是采用这种办法。如果接收方有一个要发送的分组或一个特定的计时器到时，接收方才发送一个确认。接收端可以决定每次对N个分组仅发送一个确认。我们要知道确认也是网络中负载的一个部分，发送确认越少意味着网络的负载越轻。

丢弃策略

路由器使用好的丢弃策略可以预防拥塞，同时不破坏传输的完整性。例如，在声音传输中，如果在可能发生拥塞时丢弃那些不敏感的分组，则仍可保证声音的质量，并且还能预防或减轻拥塞现象的发生。

许可策略

在虚电路网络中，许可策略是一种服务质量机制，也能预防拥塞。在允许数据流进入网

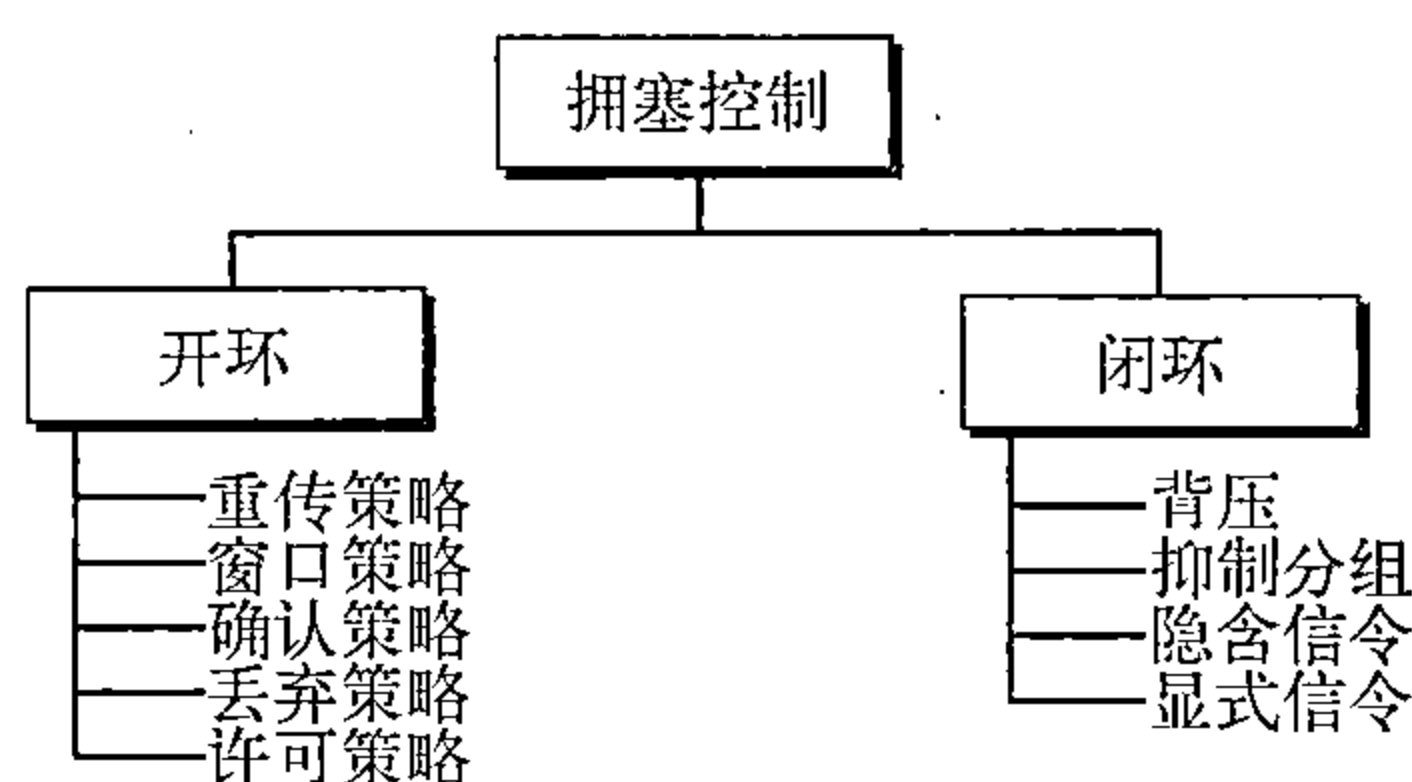


图24.5 拥塞控制分类

络之前，与数据流通路相连的交换机首先检查其资源需求。如果网络有拥塞或可能出现拥塞，路由器将拒绝建立虚电路。

24.3.2 闭环拥塞控制

在拥塞发生之后，采用闭环拥塞控制（closed-loop congestion control）可以缓解拥塞状况。不同的协议采用了多种机制，下面对这几种机制进行介绍。

背压

背压技术是一种拥塞控制机制。在这种技术中，一个拥塞点停止接收来自直接上行节点或一些近邻节点的数据。这会引引起上行节点或一些近邻节点发生拥塞，它们依此拒绝它们的上行节点或一些近邻节点的数据，依此类推。背压是点到点拥塞控制，它从一点开始，然后传播，数据流反方向到达源端。背压技术仅用于虚电路网络。在虚电路网络中，每个节点知道数据的流是来自它的上行节点。图24.6表示了背压的思想。

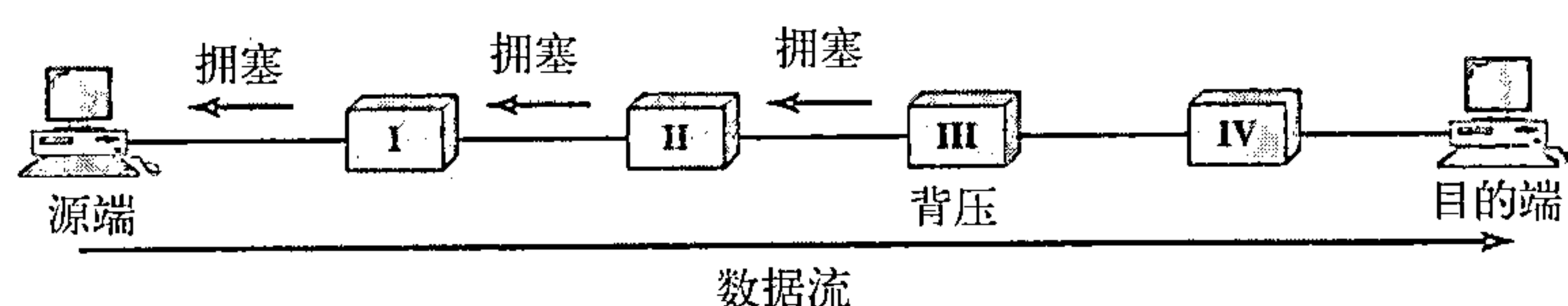


图24.6 减轻拥塞背压方法

在图24.6中，节点Ⅲ有了超出它处理能力的输入数据，它停止将某些分组放入输入缓存区中，并通知节点Ⅱ降低传输速率。依次，节点Ⅱ由于降低输出数据流的速率可能会拥塞。如果节点Ⅱ拥塞，则它通知节点Ⅰ降低传输速率，它也可能会形成拥塞。如果是如此，则节点Ⅰ通知源端降低数据传输速率。这样可以及时减轻拥塞。注意：为了消除阻塞，对节点Ⅲ的压力反向移动到源端。

本书讨论的一些虚电路网络不使用背压策略。但是，在第一个虚电路网络X.25，使用过这策略。这种技术在数据报网络不可能实现，因为在这种类型的网络中，一个节点（路由器）没有关于上行节点的知识。

抑制分组

抑制分组（choke packet）是一个分组，该分组由节点发送给源端，通知它发生拥塞的情况。注意背压和抑制分组方法的不同。在背压方法中，警告从一个节点到它的上行节点，虽然警告可能最后到达源端。而在抑制分组方法中，警告从已经发生拥塞的路由器直接传到源端，该分组经过的那些中间的节点没被警告。我们在ICMP中已看到过这种控制类型。当因特网中一个路由器被大量的IP数据报淹没时，它可能丢弃一些数据报，但它使用一个ICMP源站抑制ICMP报文通告源主机。警告报文直接发送给源站点，中间的路由器不进行任何处理。图24.7表示抑制分组的思想。

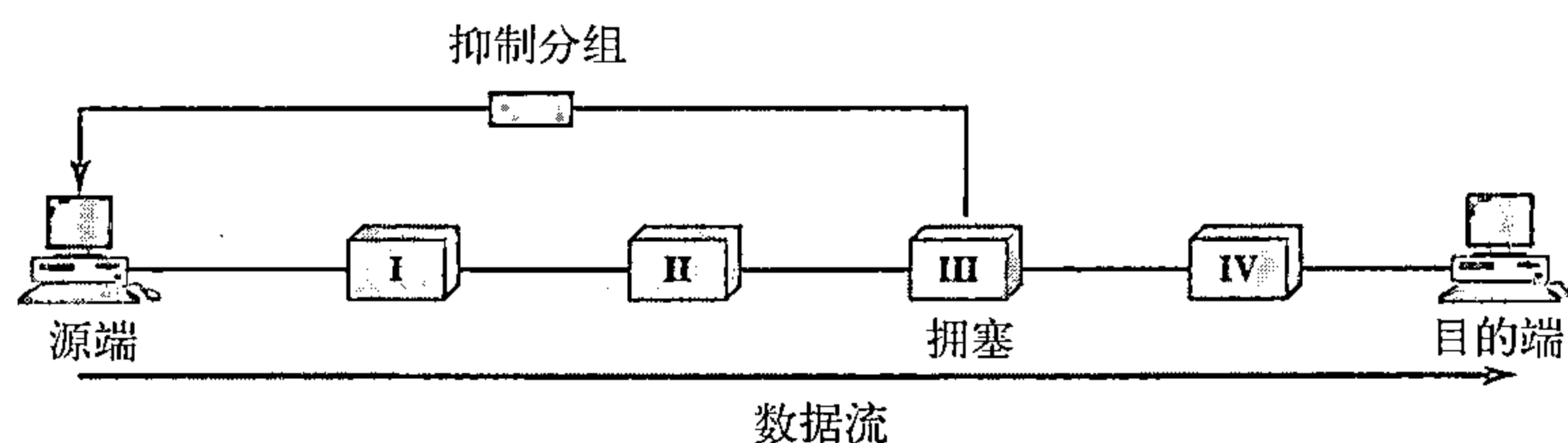


图24.7 抑制分组

隐含信令

在隐含信令中，拥塞节点或节点与源端之间没有通信。源端能从其他有关征兆中察觉出在网络某处有拥塞。例如，当源端发送多个分组，暂时还没有收到确认时，一种设想是网络发生了拥塞。在接收确认过程中发生的延迟现象就可以认为网络发生了拥塞；源端应该降低发送速率。在本章后面讨论TCP拥塞控制时，将看到这种类型的信令。

显式信令

发生拥塞的节点能发送一种显式信令通知源端或目的端发生了拥塞。但是，显式信令方法与抑制分组方法是不同的。在抑制分组方法中，有一个单独的分组用于此目的；而在显式信令方法中，信号包含在携带数据的分组中。就像在帧中继拥塞控制中所看到的那样，显式信令也具有前向或后向特征。

后向信令 将一个位设置在分组中，并使之向与拥塞发生方向相反的方向移动。该位提示源端网络发生了拥塞，并提示它需要放慢发送速度，以避免分组的丢失。

前向信令 将一个位设置在分组中，并使之向发生拥塞的方向移动。该位提示目的端网络发生了拥塞。在此情况下，接收端能使用诸如放慢确认发送速度的策略来减轻拥塞。

24.4 两个示例

为了更好地理解拥塞控制的概念，我们举两个例子：一个是TCP中的拥塞控制，另一个是帧中继中的拥塞控制。

24.4.1 TCP中的拥塞控制

我们在第23章中已讨论过TCP，现在我们说明TCP如何利用阻塞控制避免或减轻网络中的拥塞。

拥塞窗口

在第23章中，我们曾提到过流量控制，并试图讨论应如何解决接收方由于大量数据的到来而陷入瘫痪的情况。我们讲过，发送方窗口大小取决于接收方可用的缓冲空间 (*rwnd*)。换言之，我们假设只有接收方能够规定发送方的发送方窗口大小。在这里我们完全忽视了另一个实体，即网络。如果网络不能够像发送方产生数据那样快传递数据，它就应当告诉发送方减慢发送速率。也就是说，除了接收方之外，网络应当是确定发送方窗口大小的第二个实体。

现在，发送方窗口大小不仅取决于接收方，而且还取决于网络拥塞的情况。

发送方有两种信息：接收方通告的窗口大小和拥塞窗口大小。实际的窗口大小是这两者中的最小者。

实际的窗口大小 = $\min(rwnd, cwnd)$ 。

我们简要说明如何确定拥塞窗口大小 (*cwnd*)。

拥塞策略

TCP处理拥塞的一般策略基于三个阶段：慢速启动、拥塞避免和拥塞检测。在慢速启动阶段，发送方用很慢的传输速率启动，但迅速地增加到阈值。在达到阈值时，为了避免拥塞而降低数据速率。最后，如果检测到拥塞，则发送方基于如何检测拥塞而返回到慢速启动或拥塞避免阶段。

慢速启动：指数增长 TCP拥塞控制所使用的一种算法称为慢速启动 (*slow start*)。这种算法是基于这样的想法，它在开始时设置拥塞窗口大小 (*cwnd*) 为一个最大段长度 (*MSS*)。

这个MSS是连接建立期间由最大段长度选项所决定的。每次接收到一个确认时，窗口大小增加一个MSS值。正如其名的含义，窗口是慢速启动，但是按指数规则增长的。为了说明这个思想，让我们观察图24.8。注意：为了使讨论更清晰，我们进行了三个简化。使用段的个数而不是字节的个数（好像每段仅有一个字节）；假定 $rwnd$ 比 $cwnd$ 大得多，这样发送方窗口大小永远等于 $cwnd$ 。还假定每段都是单独进行确认。

发送方以 $cwnd = 1MSS$ 开始。这意味着发送方仅能发送一个段。接收到段1的确认后，拥塞窗口增加1， $cwnd$ 是2。这意味着现在可发送另外两个段。当接收到每一个确认时，窗口大小都增加1MSS。当所有7个段都被确认时， $cwnd = 8$ 。

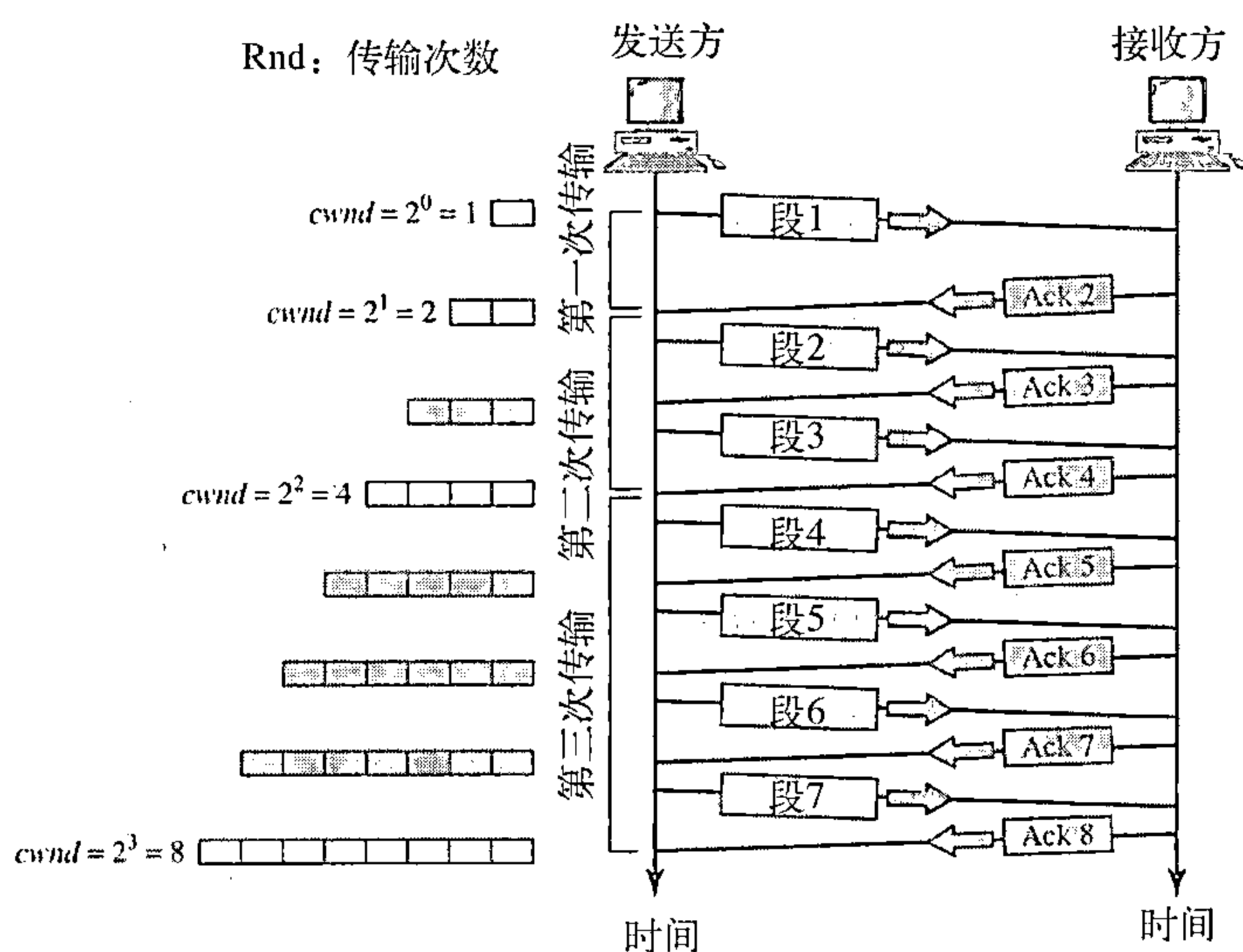


图24.8 慢速启动：指数增加

如果我们按照传输次数观察 $cwnd$ 的大小（整个窗口中的段被确认），则发现其速率是按指数规律增长如下所示：

开始 $\Rightarrow cwnd = 1$
 第一次传输后 $\Rightarrow cwnd = 2^1 = 2$
 第二次传输后 $\Rightarrow cwnd = 2^2 = 4$
 第三次传输后 $\Rightarrow cwnd = 2^3 = 8$

我们需要指出的是，如果有被延迟的ACK，则窗口大小的增长是小于2的幂。

慢速启动不能一直继续下去，到达阈值必须停止该阶段。发送方保存一个称为 $ssthresh$ （慢速启动阈值）变量。当拥塞窗口中的字节达到这个阈值时，慢速启动阶段结束而下一个阶段开始。在大多数实现中， $ssthresh$ 值是65 535字节。

在慢速启动算法中，拥塞窗口大小按指数规律增长直到达到阈值。

拥塞避免：加性增加 如果我们以慢速启动算法开始，则拥塞窗口大小按指数规律增长。为了在拥塞发生之前避免拥塞，必须降低指数增长的速度。TCP定义了另一个算法，称为拥塞避免（congestion avoidance），这个算法是加性增加（additive increase），而不是指数增加。当拥塞窗口的大小达到慢速启动的阈值时，慢速启动阶段停止，加性增加阶段开始。在这个算法中，每次整个窗口所有段都被确认（一次传输）时，拥塞窗口才增加1。为了说明这个概念，我们将这个算法应用到与慢速启动相同的情况中，不过我们将会见到的拥塞避免算法通

常开始时窗口大小是大于1的。图24.9表示这个思想。

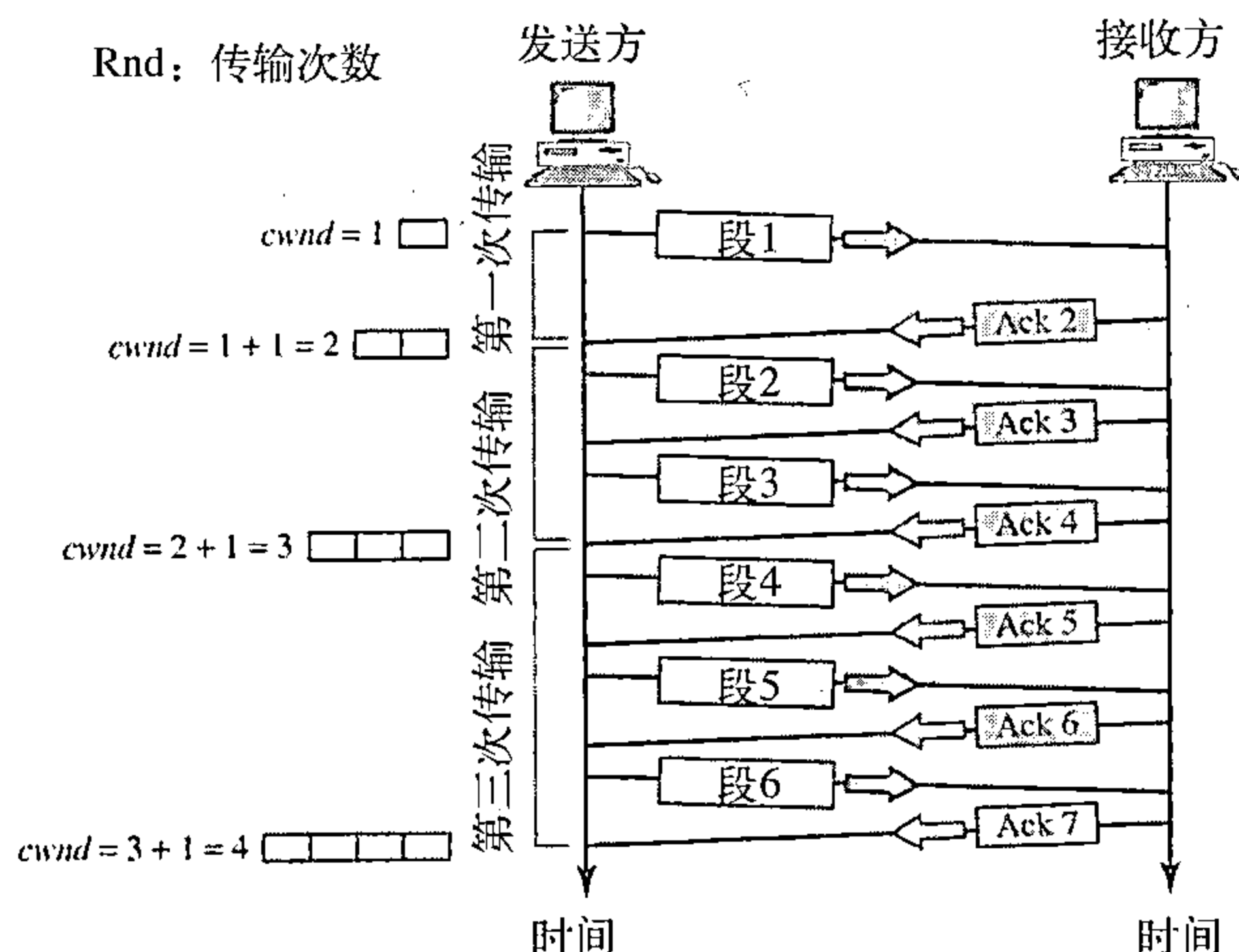


图24.9 拥塞避免：加性增加

在这种情况下，发送方接收到对一个完整窗口大小段的那些确认后，窗口大小才增加一个段。

如果我们按照传输次数观察cwnd的大小，则发现其速率是按加性规律增长，如下所示：

开始 $\Rightarrow cwnd = 1$
 第一次传输后 $\Rightarrow cwnd = 1 + 1 = 2$
 第二次传输后 $\Rightarrow cwnd = 2 + 1 = 3$
 第三次传输后 $\Rightarrow cwnd = 3 + 1 = 4$

在拥塞避免算法中，拥塞窗口大小是加性增加的直到检测到拥塞。

拥塞检测：乘性减少 如果发生拥塞，拥塞窗口的大小必须减小。发送方能推测出发生拥塞现象的唯一方法是通过重传段的要求。可是重传是在两种情况下发生：重传计时器到时或接收到了三个ACK。在这两种情况下，阈值就下降一半，即乘性减少（multiplicative decrease）。大多数TCP实现包含两个反应：

1. 如果计时器到时，那么存在着非常严重的拥塞的可能性；一个段可能已在网络中丢失并且没有关于该发送段的消息。

在这种情况下，TCP做出强烈的反应：

- 设置阈值为当前拥塞窗口大小的一半；
- 设置cwnd为一个段的大小；
- 启动慢速启动阶段。

2. 如果接收到三个ACK，那么存在着轻度拥塞的可能性。一个段可能已丢失，但自从接收到三个ACK后，有一些段可能已安全到达。这称为快速传送和快速恢复。

在这种情况下，TCP做出轻度的反应：

- 设置阈值为当前拥塞窗口大小的一半；
- 设置cwnd为阈值（有些实现是阈值加上三个段）；
- 启动拥塞避免阶段。

用下列方法之一对拥塞检测做出反应：

- 如果检测出计时器到时，那么一个新的慢速启动阶段开始。
- 如果检测出三个ACK，那么一个新的拥塞避免阶段开始。

总结 在图24.10中，概括了TCP的拥塞策略与三个阶段之间的关系。

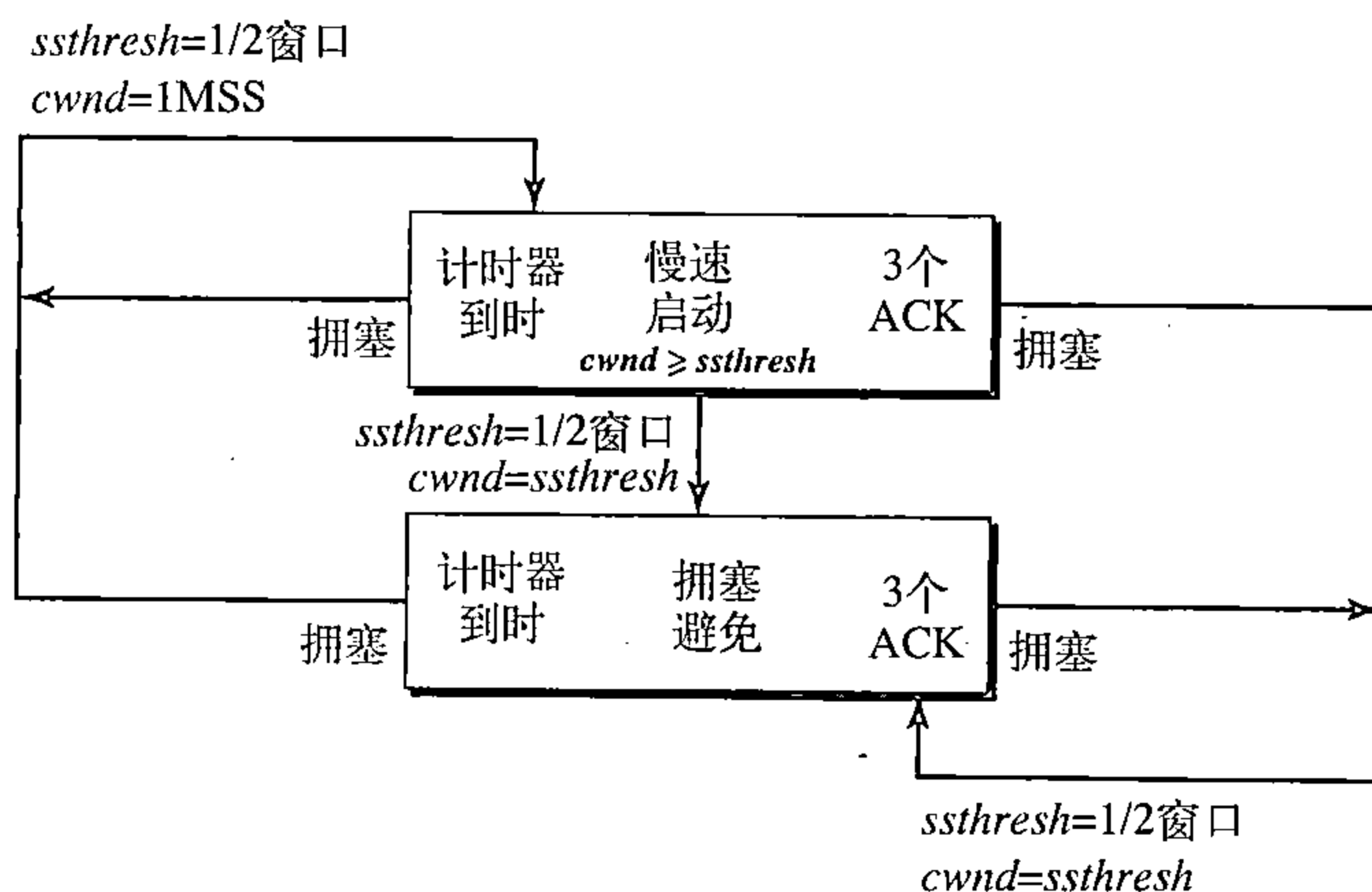


图24.10 TCP拥塞策略总结

我们在图24.11中给出了一个例子。假定最大窗口大小是32个段，阈值设置为16个段（最大窗口的一半）。在慢速启动阶段，窗口大小从1开始按指数规律增长直到它达到阈值。当它达到阈值后，拥塞避免（加性增加）过程允许窗口大小线性增长直到计时器到时或到达最大窗口大小。在图24.11中，当窗口大小为20时，计时器到时。此时，进入乘性减少过程，将阈值设置为当前窗口大小的一半。当计时器到时，当前窗口大小是20，因此现在阈值是10。

TCP再次进入慢速启动，并置窗口大小为1。当到达新的阈值时，TCP进入加性增加阶段。当窗口大小为12时，三个ACK事件发生。再次进入乘性减少过程，阈值设置为6，这时TCP进入加性增加阶段，该阶段一直维持到另一个计时器到时或者另外三个ACK事件发生为止。

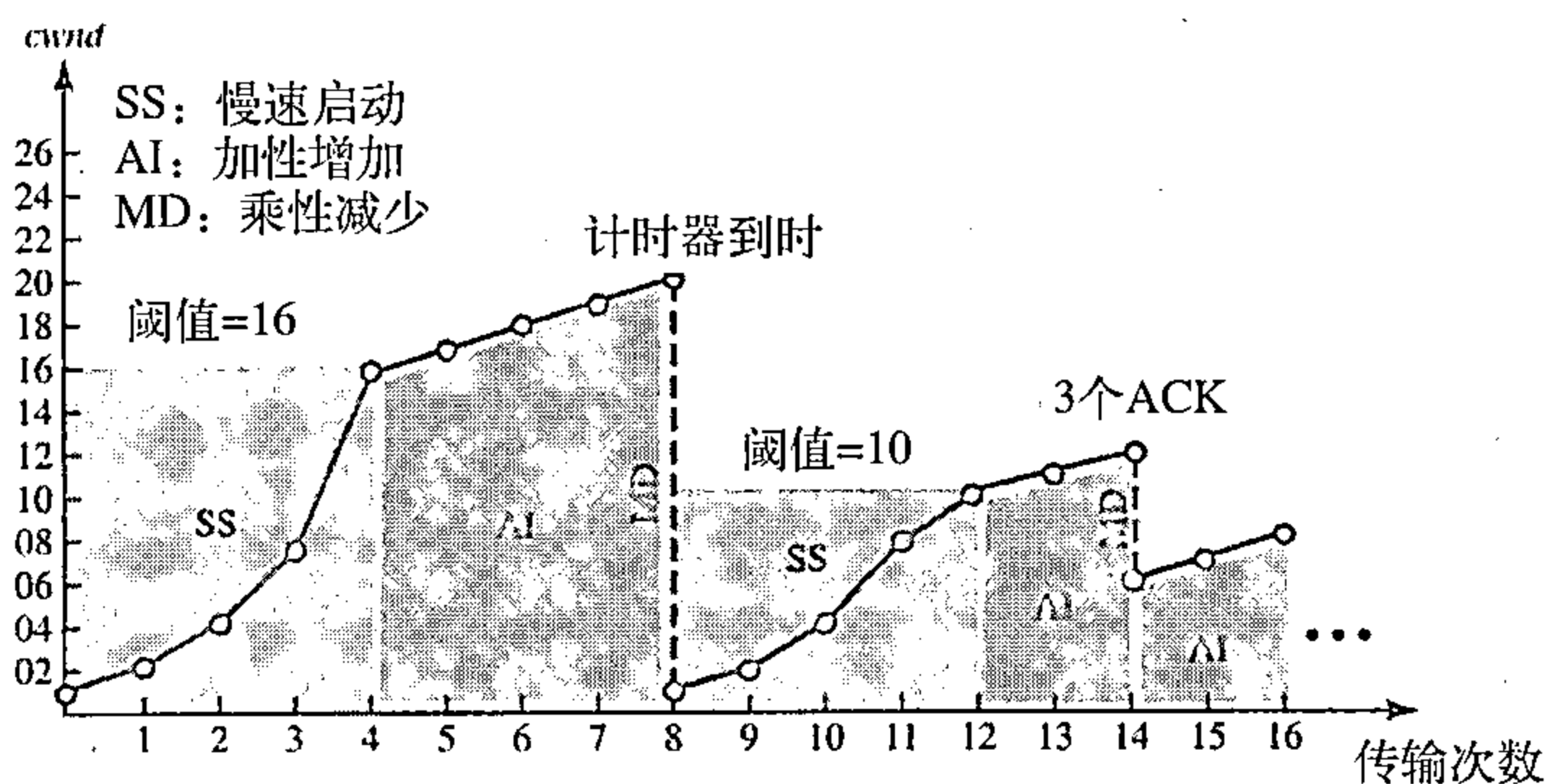


图24.11 拥塞例子

24.4.2 帧中继中的拥塞控制

帧中继网络中的拥塞会导致吞吐量降低和延迟的增加。高吞吐量和低延迟是帧中继协议的主要目标。帧中继协议没有流量控制机制。另外，帧中继允许用户传送突发性数据。这意味着帧中继网络有可能因为通信量过大而发生拥塞现象，因此需要有拥塞控制机制。

避免拥塞

为了避免拥塞，帧中继协议在帧中利用2个位来明确地提示源端和目的端拥塞的发生。

BECN 后向显式拥塞通知（backward explicit congestion notification, BECN）位提示发

送方网络中的拥塞情况。因为帧是从发送方传送出来的，有人可能问这是如何实现的。实际上有两种方法：交换机利用来自接收方（全双工模式）的响应帧；或者为了特定的目的，交换机可使用一个预定义连接（DLCI=1023）来发送专门的帧。发送方仅通过减小数据速率对该提示信息做出响应。图24.12说明了运用BECN的情况。

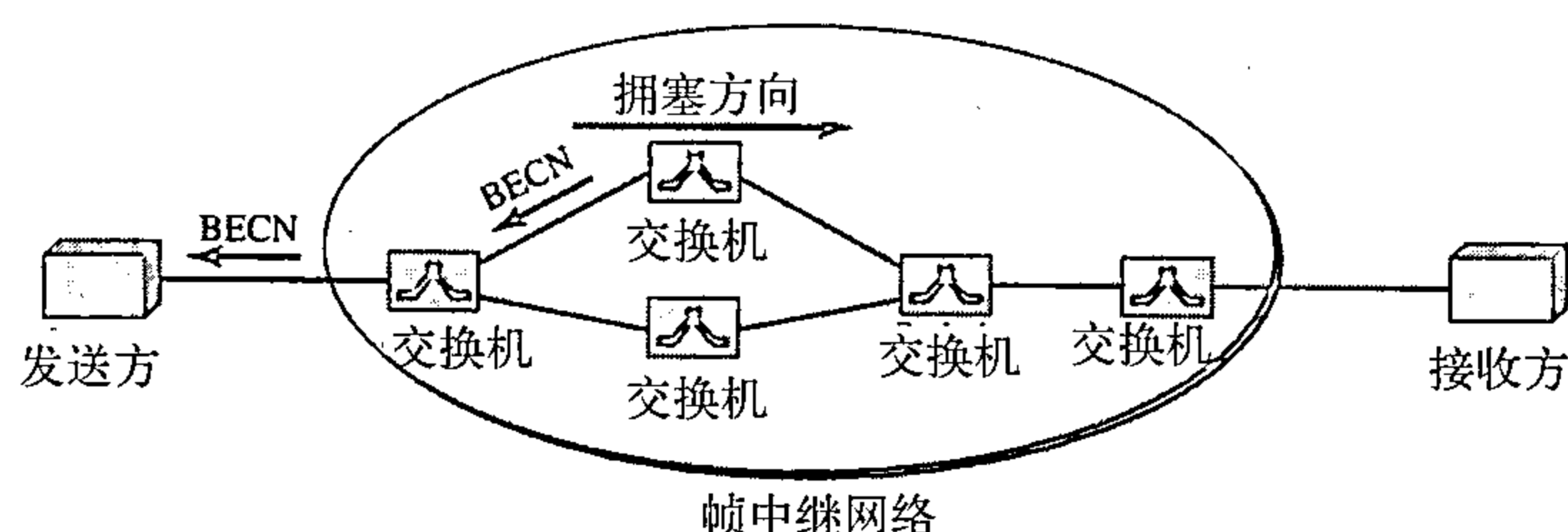


图24.12 BECN

FECN 前向显式拥塞通知 (forward explicit congestion notification, FECN) 位是用来提示接收方网络中拥塞的情况。有可能会出现接收方不能采取任何措施来减轻拥塞的情况。但是，帧中继协议假定发送方和接收方能相互通信并在较高层使用某种类型的流量控制机制。例如，如果在较高层有确认机制，接收方就能延迟发送确认，这样就可以迫使发送方放慢发送速度。图24.13说明了运用FECN的情况。

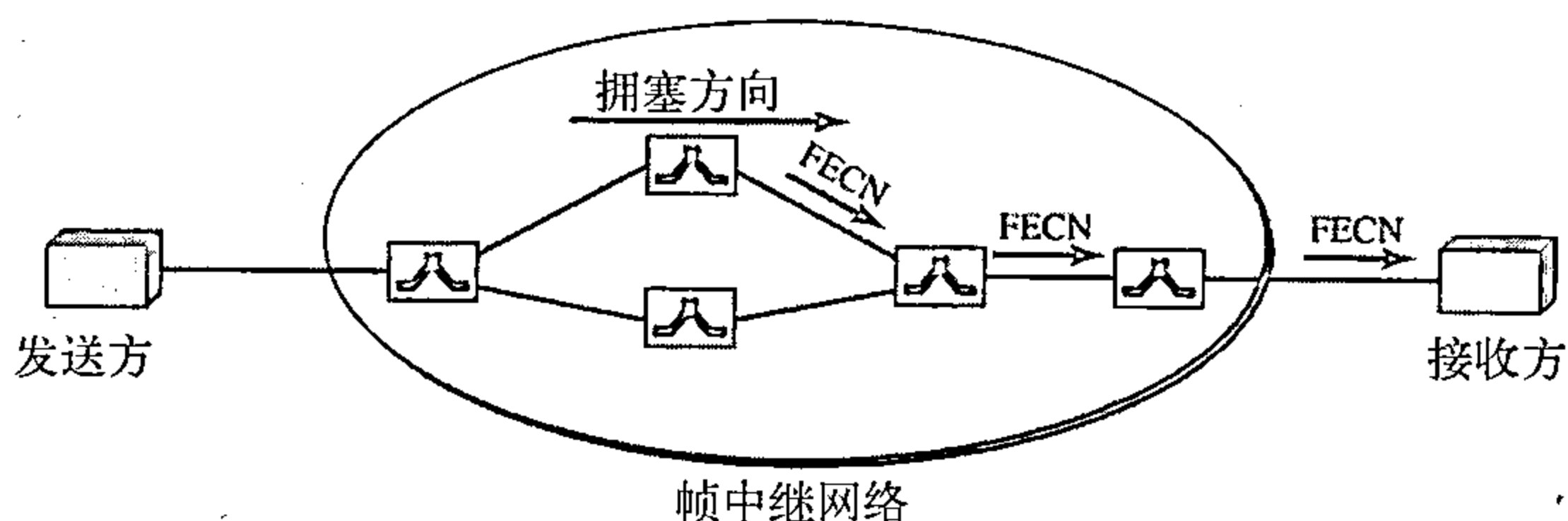


图24.13 FECN

当两个端点运用帧中继网络进行通信时，可能会出现四种与拥塞有关的情况。图24.14说明了这四种情况以及FECN和BECN的值。

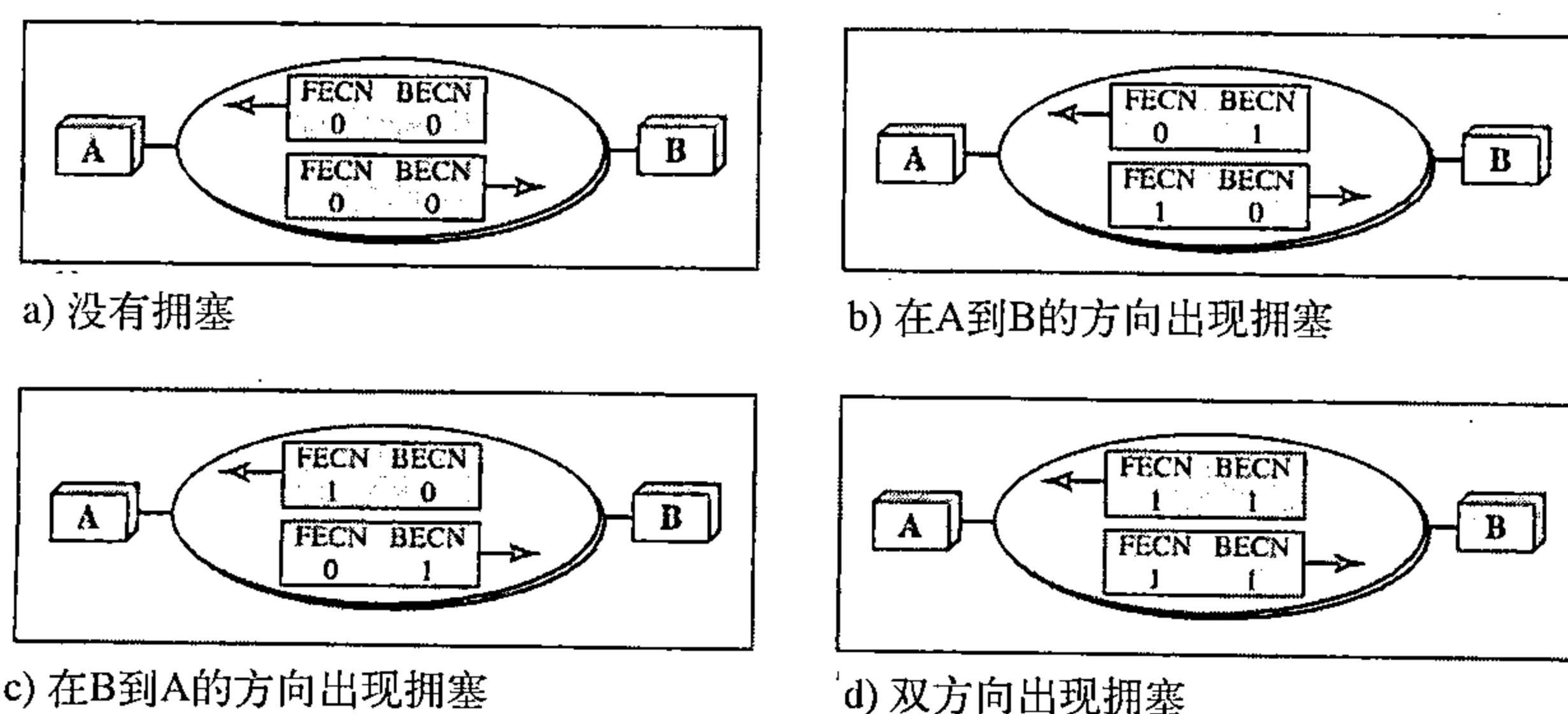


图24.14 拥塞的四种情况

24.5 服务质量

服务质量 (quality of service, QoS) 是一个网络互联问题，对该问题的讨论已经远远超出对它的定义。我们可以非形式地将服务质量定义为数据流所追求的某种目标。

24.5.1 数据流特性

按传统的观点，数据流具有四种特性：可靠性、延迟、抖动和带宽，如图24.15所示。

可靠性

可靠性 (reliability) 是数据流所需要的一个特性。缺乏可靠性意味着分组或确认的丢失，这些都必须通过重传来弥补。然而，应用层程序对可靠性的敏感程度是不同的。例如，电子邮件、文件传输和因特网访问与电话或音频会议相比，保证其可靠传输更为重要。

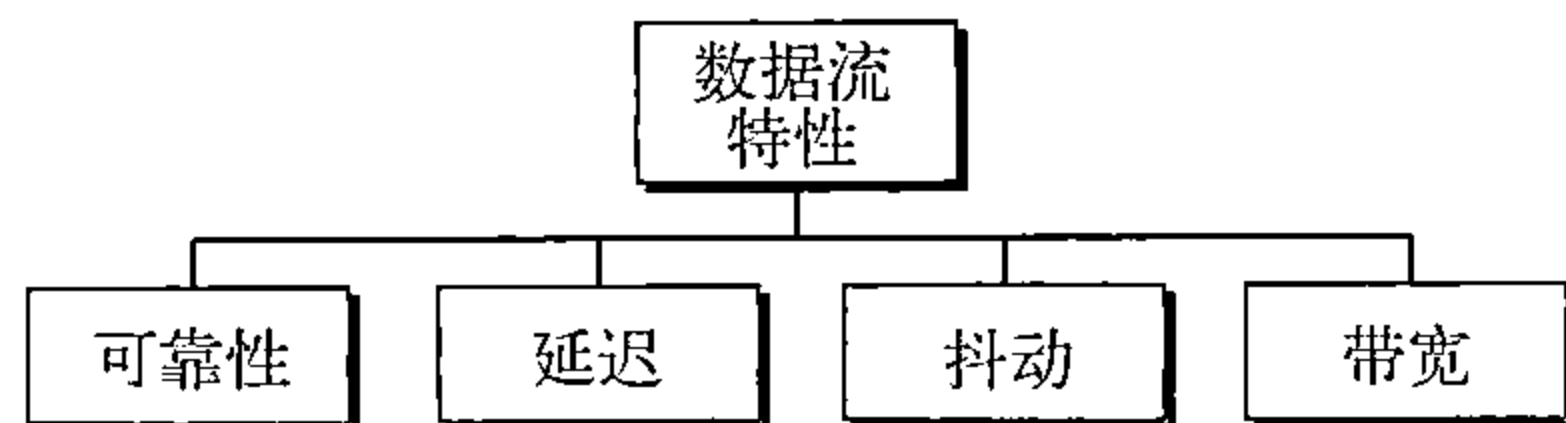


图24.15 数据流特性

延迟

源端到目的端的延迟 (delay) 是另一个数据流特性。此外，不同的应用对延迟的容许程度是不同的。在此情况下，电话、音频会议、视频会议和远程登录都需要最小的延迟，但是在文件传输或电子邮件中，延迟并不那么重要。

抖动

抖动 (jitter) 是属于同一数据流的分组延迟的变化。例如，如果4个分组离开的时间分别是0, 1, 2, 3，而到达的时间分别是20, 21, 22, 23，则它们都有相同的延迟，为20。另一方面，如果上述4个分组的到达时间分别是21, 23, 21, 28。则它们延迟是不同的：21, 22, 19, 24。对于音频和视频的应用，前者是完全可接受的，而后者是不能接受的。只要延迟对所有的分组都是相同的，分组以短或者长的延迟到达，无关紧要。对于这个应用，后者的情形是不可接受的。抖动被定义为分组延迟的变化。较高的抖动意味着在延迟之间的差异较大，而低的抖动意味着差异很小。

在第29章中，我们将会看到多媒体通信如何处理抖动。如果抖动高，那么为了使用接收到的数据，就需要执行一些动作。

带宽

不同的应用需要不同的带宽。在视频会议中需要发送数百万比特率来刷新彩色屏幕，但是在一封电子邮件中的全部位的个数甚至都不到一百万。

24.5.2 数据流类型

基于数据流的特性，将数据流分成不同的组，每个组都有相似的特性。这种分类方法既不够规范也不够普遍。一些诸如ATM之类的协议定义了类型的概念，稍后可以看到这一点。

24.6 改进QoS的技术

在第24.5节中，根据数据流的特性对QoS进行了定义。本节将讨论一些用于改进服务质量的方法。这里简要地讨论四种常用方法：调度、通信量整形、许可控制和资源预留。

24.6.1 调度

来自不同数据流的分组到达交换机或路由器，并由它进行处理。一种好的调度技术会以公平合理的方式来对待不同的数据流。已经设计了多种调度技术用来改进服务质量，在这里讨论其中的三种技术：FIFO队列、优先权队列和加权公平队列。

先进先出队列

在先进先出 (first-in, first-out, FIFO) 队列中，分组在缓冲区 (队列) 中等待，直到节

点（路由器或交换机）准备处理它们为止。如果平均到达速率高于平均处理速率，那么队列将被填满，新的分组将被丢弃。FIFO队列和那些必须在公共汽车站等待汽车的人类似。图24.16说明了FIFO队列的概念。

优先权队列

在优先权队列（priority queuing）中，首先给不同的分组分配一个不同的优先权类。每个优先权类都有自己的队列。在最高优先权队列中的分组首先得到处理，而最后才对最低优先权队列中的分组进行处理。注意：系统不停地为一个队列提供服务，直到它变空为止。图24.17说明了具有两个优先权级别的优先权队列（为了简单起见）。

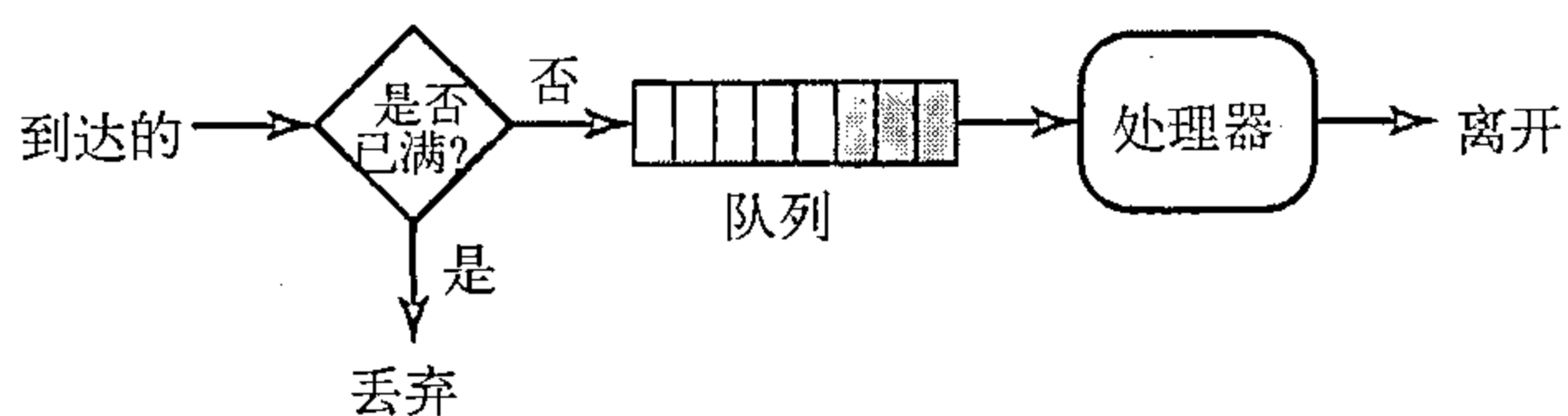


图24.16 FIFO队列

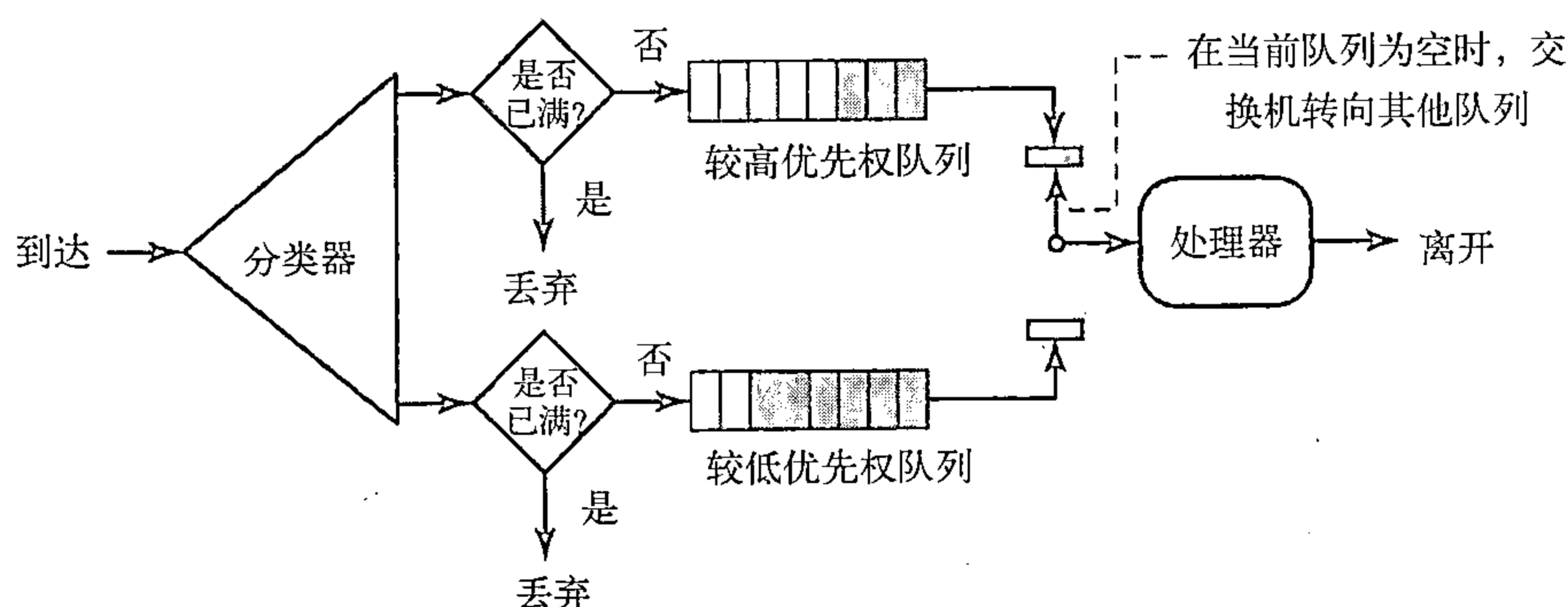


图24.17 优先权队列

与FIFO队列相比，优先权队列能提供更好的QoS，因为具有较高优先权的通信量，如多媒体，能用较少的延迟到达目的端。但是，优先权队列也有潜在的缺点。如果在高优先权队列中有持续的通信量，那么处于较低优先权队列中的分组将永远得不到处理。这种情形称为“饥饿”。

加权公平队列

较好的调度算法是加权公平队列（weighted fair queuing）。在这种技术中，分组仍然被分成不同的类，并且属于不同的队列。然而，队列是基于队列的优先权来分配权重的，较高的优先权就意味着具有较高的权重。系统以轮换的方式来处理每个队列中的分组，所处理的分组的数量等于相应队列的权重。例如，如果这些权重是3、2和1，那么就对第一个队列中的3个分组、第二个队列中的2个分组和第三个队列中的1个分组进行处理。如果系统并没有给不同类指定不同的权重，那么所有的权重就是相等的。这就是加权公平队列的原理。图24.18以三个类为例说明了这种技术。

24.6.2 通信量整形

通信量整形（traffic shaping）是一种控制发送到网络中的通信量和速率的机制。通信量整形有两种技术：漏桶和令牌桶。

漏桶

如果桶在底部有一个小洞，只要桶中有水，水便从桶中以不变的速率漏下。如果桶中有

水，水漏的速率并不依赖于将水倒入桶中的速率。输入速率可以发生变化，但是输出速率保持恒定。同样，在网络中一种被称为漏桶（leaky bucket）的技术能消除突发性通信量。将突发性大块数据存储到桶中，然后以平均速率发送出去。图24.19说明了漏桶和其效果。

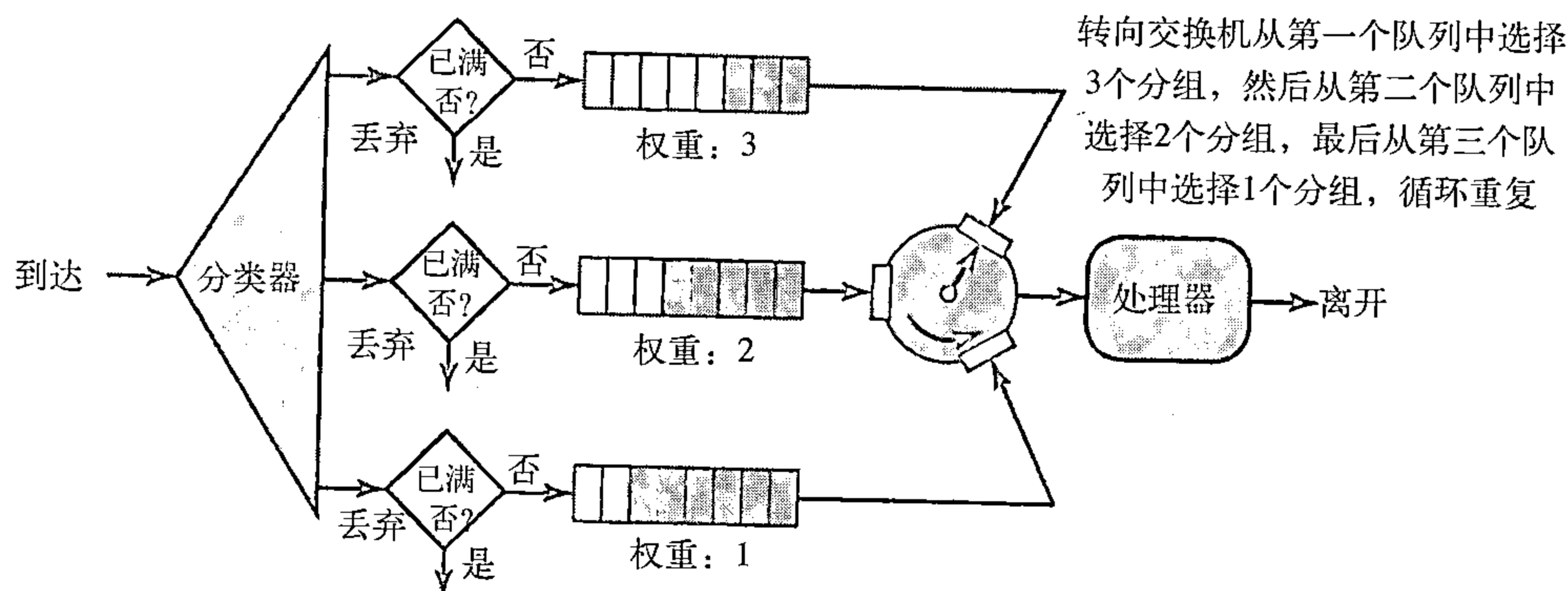


图24.18 加权公平队列

在图24.19中，假设网络为每台主机都设定了3Mbps的带宽。使用漏桶对输入通信量进行整形，是为了使通信量符合这个设定的带宽。在图24.19中，主机以12Mbps的速率发送了2秒的突发性数据，数据一共是24兆位。主机在之后的5秒内没有任何反应，然后以2Mbps的速率发送数据，持续时间为3秒，此时的全部数据为6兆位。主机在10秒内总共发送了30兆位的数据。在这同样的10秒内，漏桶以3Mbps的速率发送数据来使通信量保持平稳。如果没有漏桶，因为该主机占用了比分配给它的更大的带宽，它开始所发送的突发性数据可能已经导致网络拥塞。而且，漏桶还可以避免拥塞。做个类比，以高峰期间（突发性通信量）的高速公路为例，如果往返者们能错开他们的工作时间，那么高速公路上的拥塞状况就可以得到避免。

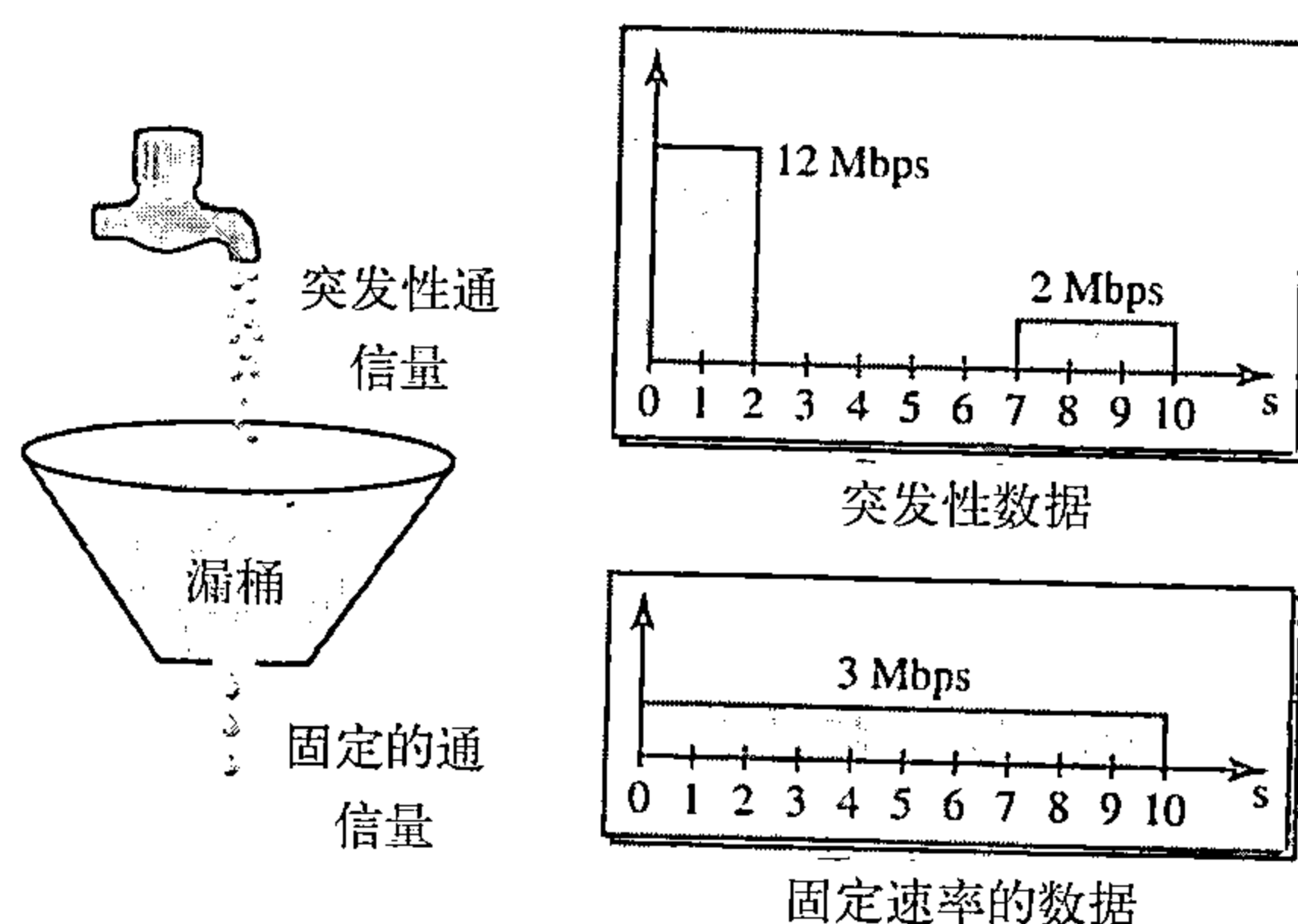


图24.19 漏桶

简单的漏桶实现过程如图24.20所示。FIFO队列保存了分组。如果这个通信量由固定大小的分组（例如，ATM网中的信元）组成，那么在每个时钟单位时间内，进程从队列中移除固定数量的分组。

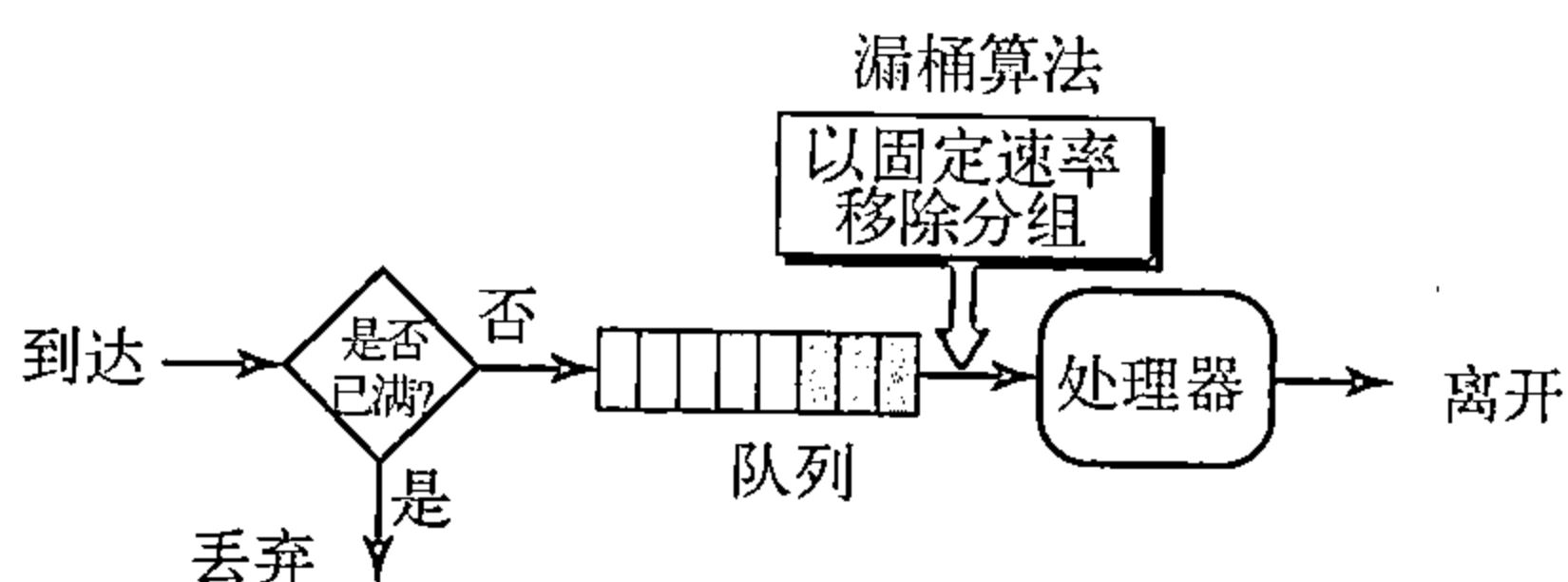


图24.20 漏桶的实现过程

如果通信量是由变长分组组成的，那么固定输出速率必须是基于字节个数或位个数。

以下是变长分组的算法：

1. 在计时开始时，将计数器初始化为 n ；
2. 如果 n 比分组的长度大，就发送分组，并将计数器的值减去分组的长度。重复该步骤，直到 n 值小于分组的长度；
3. 重新设置计数器，并返回到步骤1。

漏桶算法通过均分数据速率将突发性通信量整形为固定速率的通信量。如果桶已满，就可能丢失分组。

令牌桶

漏桶算法有很大的局限性，它不能给空闲的主机提供信用。例如，如果一台主机在一段时间内没有发送数据，其漏桶就变为空桶了。现在即使主机中有突发性数据，漏桶也只能允许平均速率的数据。此时并没有考虑主机空闲时间。另一方面，令牌桶（token bucket）算法允许空闲主机以令牌的形式为未来累积信用。在每个时钟单位时间内，系统发送 n 个令牌到桶中。在每个数据信元（或字节）发送后，系统就从中去掉一个令牌。例如，假设 n 是100，主机空闲了100个时钟单位时间，那么令牌桶便收集了10 000个令牌。主机可以在一个时钟单位时间消费所有这些令牌来发送10 000个信元，或者花去1 000个时钟单位时间，每个时钟单位时间内发送10个信元。换句话说，只要令牌桶不是空的，主机就能发送突发性数据。图24.21说明了该算法的原理。

使用计数器可以很容易地实现令牌桶算法。令牌初始化为0。每次增加1个令牌，计数器就加1；每次发送一个数据单元，计数器就减1。当计数器为0时，主机不能发送数据。

令牌桶允许规定的最大速率发送突发性通信量。

令牌桶和漏桶的结合使用

可将两种技术结合起来给空闲主机提供信用，同时调整通信量。在令牌桶之后应用漏桶，并且漏桶的速率应比将令牌放入桶中的速率高。

24.6.3 资源预留

数据流需要诸如缓冲区、带宽、CPU时间等资源。如果提前对这些资源进行预留，将会提高服务质量。我们在这一节将讨论一个称为综合业务的QoS模型，该模型极大地依赖资源预留来提高服务质量。

24.6.4 许可控制

许可控制指的是由路由器或交换机使用的一种机制，它基于一个称为数据流规范的预定义参数来接收或拒绝数据流。在路由器接收数据流并对其进行处理之前，它检查数据流规范，查看其性能（根据带宽、缓冲区大小及CPU速度等）和它与其他数据流先前的约定能否处理新的数据流。

24.7 综合业务

根据第24.5节和第24.6节的讨论，在因特网中已经设计出了两种用于提供服务质量的模型：综合业务和差分业务。两个模型都强调在网络层（IP）使用服务质量，尽管这些模型也可以用于诸如数据链路层的其他层。本节讨论综合业务，第24.8节讨论差分业务方面的内容。正如在第20章中所了解到的，IP最初是为尽力传递而设计的。这意味着每个用户接受同样级别的服务。这种类型的传递不能保证一项业务的最低要求，例如实时音频和视频应用所需的带宽。即使这样的应用偶然获得了额外的带宽，也可能对其他应用造成不利影响，并导致拥塞

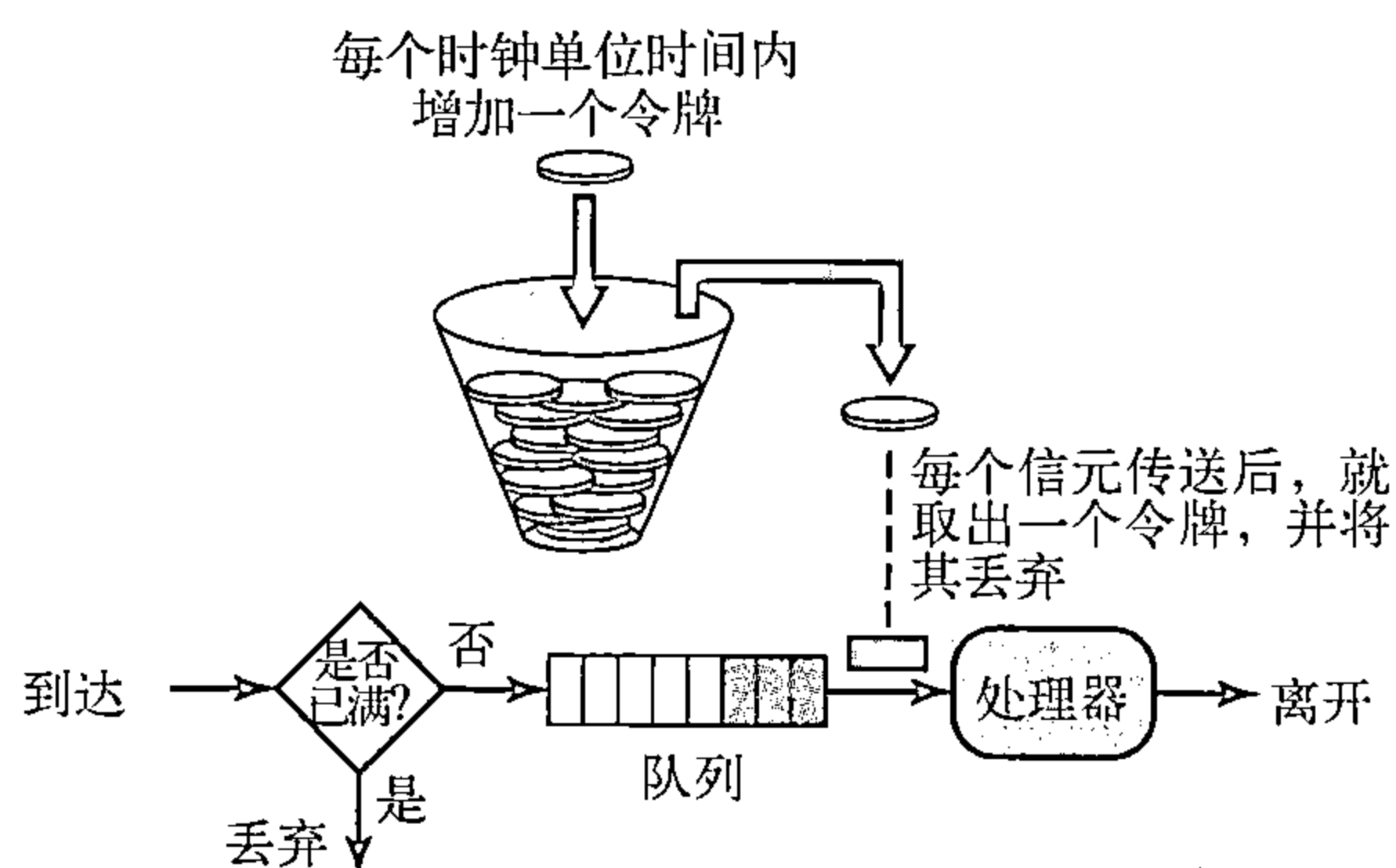


图24.21 令牌桶算法的原理

现象发生。

综合业务 (integrated service) 有时称为IntServ, 它是基于数据流的QoS模型。这意味着用户需要创建一条从源端到目的端的流, 即一种虚电路, 并将资源需求通知给所有的路由器。

综合业务是设计用于IP的基于数据流的QoS模型。

24.7.1 信令

读者可能记得, IP是一种无连接的、数据报、分组交换协议。如何在一种无连接的协议上实现一个基于数据流的模型呢? 解决办法是在IP上使用信令协议, 它为资源预留提供信令机制。这个协议称为**资源预留协议** (Resource Reservation Protocol, RSVP), 稍后会讨论到它。

24.7.2 数据流规范

当源端做资源预留时, 需要定义数据流规范。数据流规范包括两个部分: Rspec (资源规范) 和Tspec (通信量规范)。Rspec定义数据流需要预留的资源 (如缓冲区、带宽等), Tspec定义数据流的通信量特征。

24.7.3 许可

在路由器从一个应用接收到数据流规范后, 它决定许可或拒绝这个业务。所做出的决定基于路由器的先前约定和资源的当前有效性。

24.7.4 业务类型

综合业务已经定义了两种类型的业务: 保证型业务和受控载荷型业务。

保证型业务

这种类型的业务用于实时传输, 这种传输需要保证端到端延迟最小。端到端延迟是指在各路由器中的延迟之和、在介质中的传播延迟和设置机制。只有第一项, 即在各路由器中的延迟之和可由路由器来保证。这类业务保证分组在一定的传递时间内到达, 并且只要数据流通信量在Tspec边界范围之内, 即可保证不丢弃分组。可以说保证型业务就是定量业务, 在这种业务中, 端到端延迟量和数据速率必须由应用来定义。

受控载荷型业务

这种类型的业务用于可以接受一定延迟, 但对超载网络和丢失分组的危险很敏感的应用。这些类型应用的较好实例有文件传输、电子邮件和因特网访问。受控载荷型业务是一种定性业务, 因为应用要求较低的分组丢失率或零分组丢失率。

24.7.5 RSVP

在综合业务模型中, 应用程序需要资源预留。正如在IntServ模型中所讨论的那样, 为数据流进行资源预留。这意味着如果要在IP层使用IntServ, 则需要在IP层之外创建一条数据流通路, 它是一种虚电路网络, 最初是用于数据报分组交换网络。在数据传输可以启动之前, 虚电路网络需要信令系统来建立虚电路。资源预留协议 (RSVP) 是一种信令协议, 它帮助IP创建一条数据流通路, 从而实现资源预留。在讨论RSVP之前, 需要指出的是, 它是一种与综合业务模型分开的独立协议。它可以用于其他模型中。

多播树

RSVP不同于以前看到的一些其他信令系统, 因为它是一种用于多播的信令系统。但是,

RSVP也能用于单播，因为单播仅仅是在多播组中只有一个成员的多播的一个特例。这样设计的原因是为了使RSVP能为包括经常使用多播的多媒体在内的各种通信量提供资源预留。

基于接收方的预留

在RSVP中，由接收方（而不是发送方）建立预留。这种策略和其他多播协议相匹配。例如，根据多播路由选择协议，由接收方（而非发送方）来决定加入或离开多播组。

RSVP报文

RSVP有多种类型的报文。然而根据需要，下面只讨论其中的两种：路径（Path）和预留（Resv）。

路径（Path）报文 回想一下在RSVP中数据流中的接收方预留资源。但是在预留资源之前，接收方并不知道分组经过的路径，而预留资源是需要知道路径的。为了解决这个问题，RSVP使用路径报文。路径报文从发送方传送出来，到达多播路径中的所有接收方。在整个路径中，路径报文为接收方存储必要的信息。在多播环境中将路径报文发送出去，并且在路径分叉时会产生新的报文。图24.22说明了路径报文。

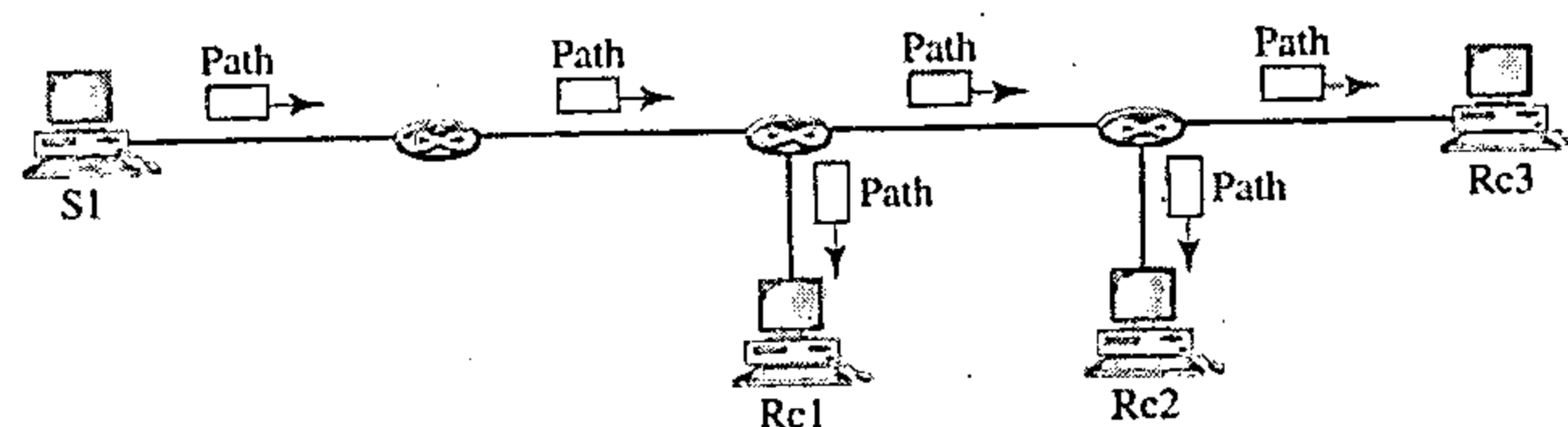


图24.22 路径报文

预留（Resv）报文 在接收方已经收到路径报文之后，它发送预留报文。预留报文朝着发送方传送（上行），并且在支持RSVP的路由器上预留资源。如果路由器不支持路径中的RSVP，它将使用以前所讨论的尽力传递方法对分组进行路由。图24.23说明了预留报文。

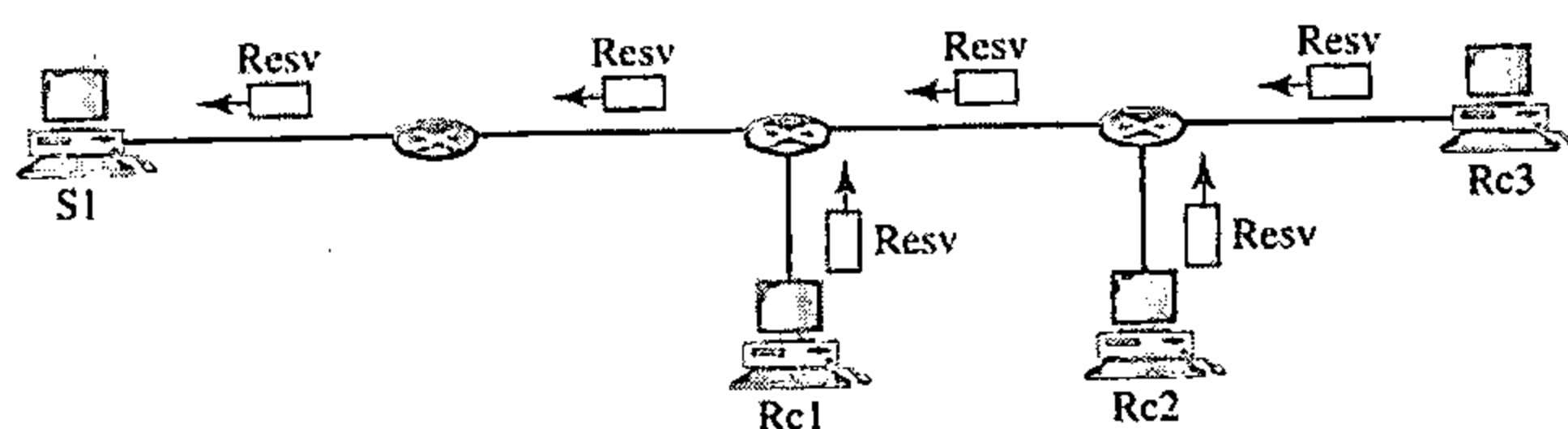


图24.23 预留报文

预留合并

在RSVP中，并没有为数据流中的每个接收方预留资源，而是将资源预留进行了合并。在图24.24中，Rc3请求2Mbps带宽，而Rc2请求1Mbps的带宽需要建立带宽预留的路由器R3将这两个请求合并。只对2Mbps带宽的那个请求（即两个中较大的那个请求）做预留，这是因为2Mbps的输入预留就能处理这两个请求。这种情况对R2也是适用的。读者可能问，为什么Rc2和Rc3（两者都属于一个单一数据流）要请求不同的带宽值呢？答案是，在多媒体环境中不同的接收方可以处理不同的质量等级。例如，Rc2或许只能以1Mbps（较低质量）的速率接收视频信息，而Rc3或许可以以2Mbps（较高质量）的速率接收视频信息。

预留方式

当存在多条数据流通路时，路由器需要预留资源来接收所有的数据流。RSVP定义了三种预留方式，如图24.25所示。

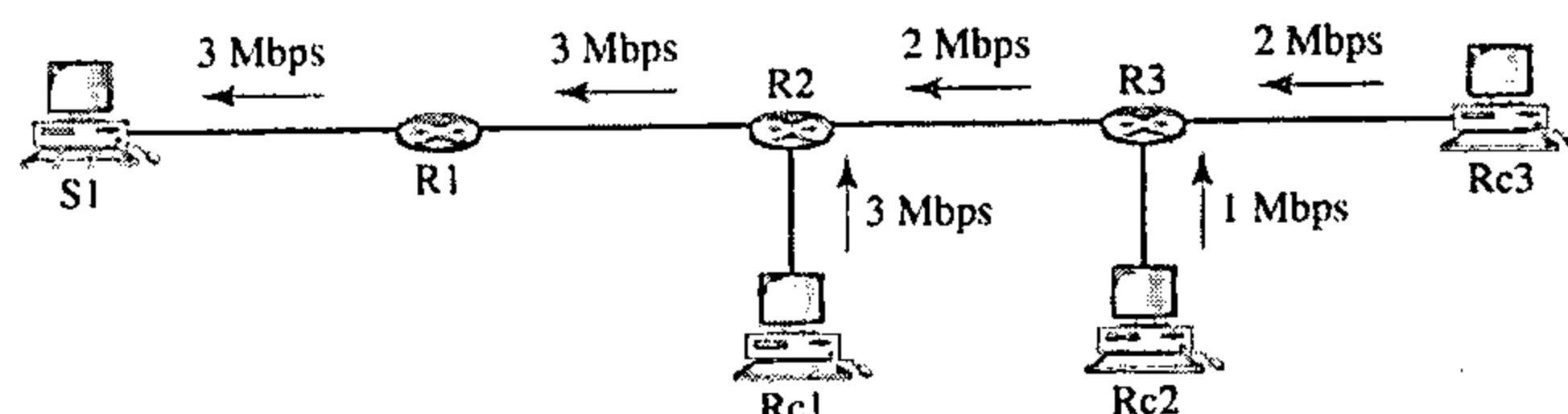


图24.24 预留合并

通配过滤器方式 在这种方式中，路由器为所有发送方创建一个预留。这个预留是基于最大的请求。这种方式用于来自不同发送方的数据流不同时出现的情况。

固定过滤器方式 在这种方式中，路由器为每个数据流创建一个不同的预留。这意味着如果有 n 个数据流，就要建立 n 个不同的预留。这种方式用于来自不同发送方的数据流同时出现的概率很高的情况。

共享显式方式 在这种方式中，路由器只创建一个预留，该预留可以被一组数据流共享。

软状态

为数据流而存储在每个节点中的预留信息（状态）需要定期进行刷新。与用于其他诸如ATM或帧中继这类虚电路协议的硬状态相比，这种状态称为软状态。在硬状态中，有关数据流的信息被一直保留着，直到这些信息被擦除为止。目前默认的刷新闻隔是30秒。

24.7.6 综合业务所存在的问题

在综合业务中至少存在两个问题，这可能会妨碍了它在因特网中的实现。它们是可伸缩性和服务类型限制。

可伸缩性

综合业务模型要求每台路由器都要为每个数据流保存信息。但随着因特网规模的日益增长，这是一个严重的问题。

业务类型限制

综合业务只提供了两种类型的业务，即保证型业务和受控载荷型业务。那些反对该模型的人指出，应用可能需要比这两种业务类型更多的业务类型。

24.8 差分业务

差分业务（Differentiated Services, DS或Diffserv）是由因特网工程任务组（Internet Engineering Task Force, IETF）提出的用于弥补综合业务缺陷的一种业务。与综合业务相比，它有两个基本变化：

1. 主要处理过程从网络核心转移到网络边缘。这就解决了可伸缩性的问题。路由器不需要存储有关数据流的信息。每次发送分组时，应用程序或主机定义它们所需要的业务类型。

2. 将针对每条数据流的业务改变为针对每种类型的业务。路由器根据在分组中定义的业务类型，而不是根据数据流来发送分组。这就解决了业务类型限制的问题，可根据应用程序的需要定义不同的业务类型。

差分业务是用于IP的基于类的QoS模型。

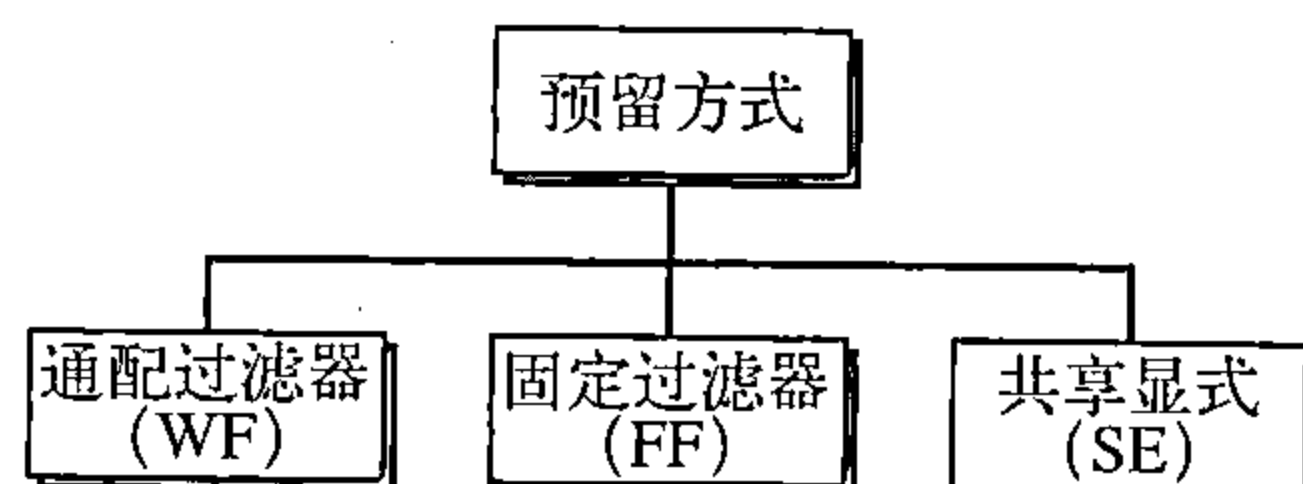


图24.25 预留方式

DS字段

在Diffserv中，每个分组都包含一个称为DS的字段。该字段的值是在网络边界由主机或用作边界路由器的第一台路由器来设置的。IETF提议用DS字段替换现存的IPv4中的TOS（服务类型）字段或IPV6中的类字段，如图24.26所示。



图24.26 DS字段

DS字段包含两个子字段：DSCP和CU。差分业务编码点（Differentiated Services Code Point, DSCP）是一个6位的子字段，它定义了逐跳行为（per-hop behavior, PHB）。长度为2位的当前未用（currently unused, CU）子字段当前还未使用。

Diffserv可用的节点（路由器）使用DSCP的6位作为定义分组处理机制表的索引，并且该定义是针对当前正在处理的分组的。

逐跳行为

Diffserv模型为接收到分组的每个节点定义了逐跳行为（PHB）。到目前为止已定义了三种PHB：默认（DE）PHB、加速转发（EF）PHB和保证转发（AF）PHB。

DE PHB 默认PHB 和尽力传递是相同的，它与TOS相兼容。

EF PHB 加速转发PHB提供如下的服务：

- 低丢失率
- 低等待时间
- 可保证的带宽

这与源端和目的端之间使用的虚连接是相同的。

AF PHB 只要类通信量不超过节点的通信量规范，AF PHB就能以较高可靠性传送分组。网络用户要意识到一些分组可能会丢失。

通信量调节器

为了实现Diffserv，DS节点使用了几种调节器，如测量器、标记器、整形器和丢弃器。如图24.27所示。

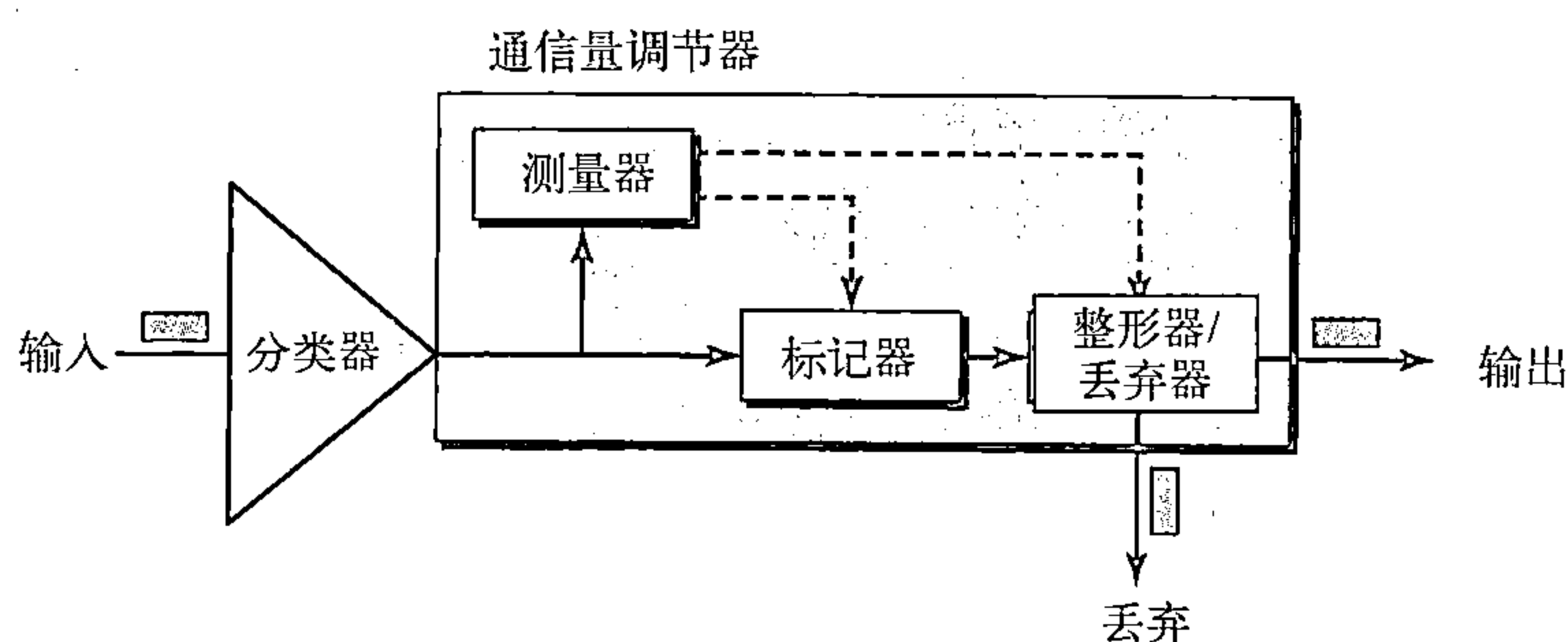


图24.27 通信量调节器

测量器 测量器检查输入数据流和协商的通信量规范是否匹配，并将这个结果发送给其他组件。测量器也能利用诸如令牌桶之类的工具来检查配置文件。

标记器 标记器能重新标记正使用尽力传递的分组（DSCP：000000）或依据接收自测量器的信息给分组作降标记。如果数据流与其对应的规范不匹配，就会对其作降标记（降低数据流的等级）。标记器不对分组作升标记（提升等级）。

整形器 当通信量与协商的规范不匹配时，整形器使用接收自测量器的信息对该通信量进行重新整形。

丢弃器 丢弃器的运作过程就像没有缓冲区的整形器。如果数据流严重违反协商的规范，那么丢弃器就要丢弃这些分组。

24.9 交换网络中的QoS

讨论了IP协议中提出的QoS模型之后，现在来讨论一下用于两个交换网络中的QoS：帧中继和ATM。这两个网络都是虚电路网络，它们需要一个诸如RSVP的信令协议。

24.9.1 帧中继中的QoS

在帧中继中已经设计了控制通信量的四个不同的属性指标：访问速率、提交突发业务量 B_c 、提交信息速率（CIR）和超突发长度 B_e 。它们是在用户和网络进行协商期间设置的。对于PVC连接而言，它们需协商一次；而对于SVC连接而言，在连接建立期间要为每个连接进行协商。图24.28说明了这四个属性指标间的关系。

访问速率

每个连接都要定义访问速率（access rate）（以每秒位为单位）。访问速率实际上是由连接用户与网络的通道的带宽来决定的。用户永远不能超过这个速率。例如，如果用T-1线路将用户连接到帧中继网络，则访问速率是1.544Mbps，并且用户永远无法超过这个速率。

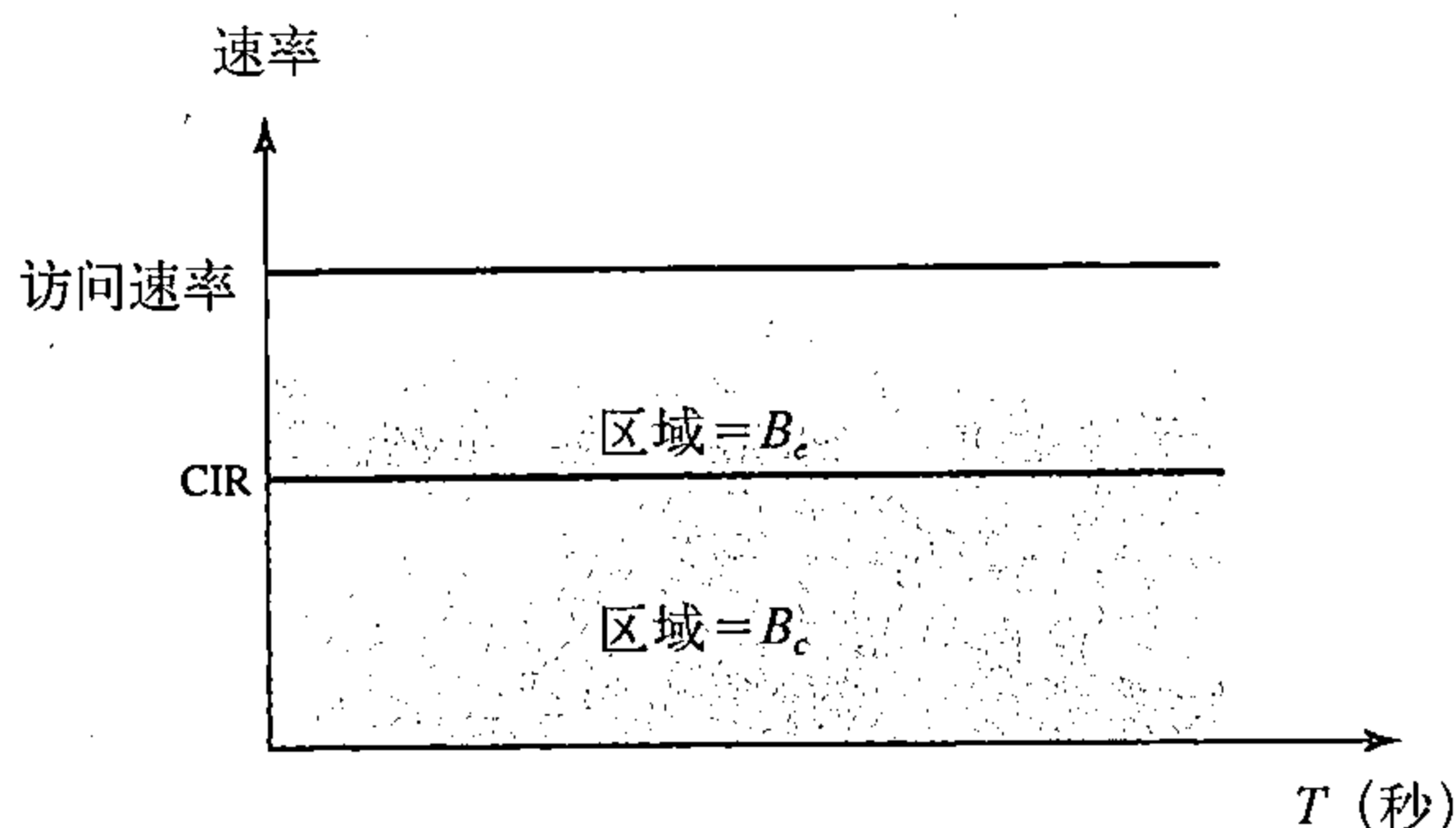


图24.28 通信量控制属性指标间的关系

提交突发业务量

对于每个连接而言，帧中继定义了提交突发业务量（committed burst size） B_c 。该值是在一段预定义的时间内，在没有丢弃任何帧或设置DE位的条件下，网络所传输的最大比特数。例如，在4秒的时间段内 B_c 的值为400kbit是可以的，因为用户能在4秒的间隔内传送400千位的数据，而不用担心会出现任何帧丢失的情况。注意这个量并不是针对每个1秒而定义的速率，而是一个累积的度量。用户可以在第一个1秒发送300kbit，在第二个和第三个1秒期间不发送任何数据，而最后在第四个1秒期间发送100kbit。

提交信息速率

提交信息速率（committed information rate, CIR）除了定义了以每秒比特为单位的平均速率之外，它和提交突发业务量在概念上类似。如果用户持续地以该速率发送数据，那么网络就能一直发送帧。但是，因为它是一个平均度量，因此用户有时可以发送高于CIR的数据而在其他时间里发送低于CIR的数据。只要预定义期间的平均速率符合要求，就会发送这些帧。在预定义期间，发送的累积比特数不能够超过 B_c 。注意CIR不是一个独立的度量，可使用如下的公式对它进行计算：

$$\text{CIR} = \frac{B_c}{T} \text{ (bps)}$$

例如，如果在5秒内， B_c 是5kbit，那么CIR就是5 000/5，即1kbps。

超突发长度

对于每个连接而言，帧中继都定义了超突发长度（excess burst size） B_e 。该值是在一段预定义的时间内，用户发送的超过 B_c 的最大位数。如果网络中没有拥塞，网络就保证传输这些

比特。注意，这种约定实现的可能性比在 B_c 情况下要低，因为网络本身实现这种约定是有条件的。

用户速率

图24.29说明了用户如何才能发送突发性数据。如果用户发送的数据没超过 B_c ，那么网络就能保证传送帧而不会出现丢弃帧的情况。如果用户发送的数据超过了 B_c 而小于 $B_c + B_e$ （也就是全部比特数小于 $B_c + B_e$ ），那么在网络中没有拥塞的条件下，它就能保证传送所有的帧。如果出现拥塞情况，将会丢失一些帧。从用户接收帧的第一台交换机有一个计数器，它可为超过 B_c 的帧设置DE位。如果网络出现拥塞情况，那么其余的交换机将会丢弃这些帧。注意，需要较快发送数据的用户可能超过 B_c 。只要该指标不超过 $B_c + B_e$ ，那么这些帧就有可能到达目的端而不丢失。然而，要记住的是，当在某一时刻用户发送的数据超过了 $B_c + B_e$ ，那么第一台交换机就会丢弃此后所发送的所有帧。

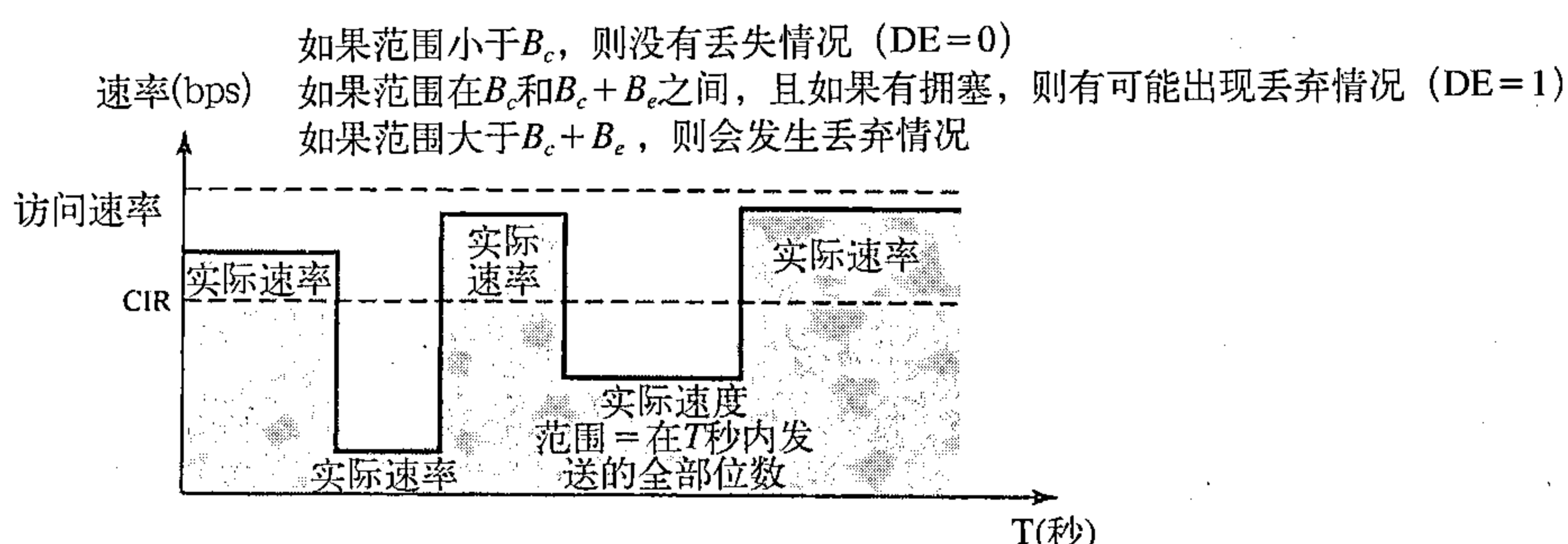


图24.29 用户速率和 B_c 与 $B_c + B_e$ 的关系

24.9.2 ATM中的QoS

ATM中的QoS是基于类、与用户相关的属性和与网络相关的属性。

类

ATM论坛定义四种服务类型：恒定比特率（CBR）、可变比特率（VBR）、可用比特率（ABR）和未指定比特率（UBR）（见图24.30所示）。

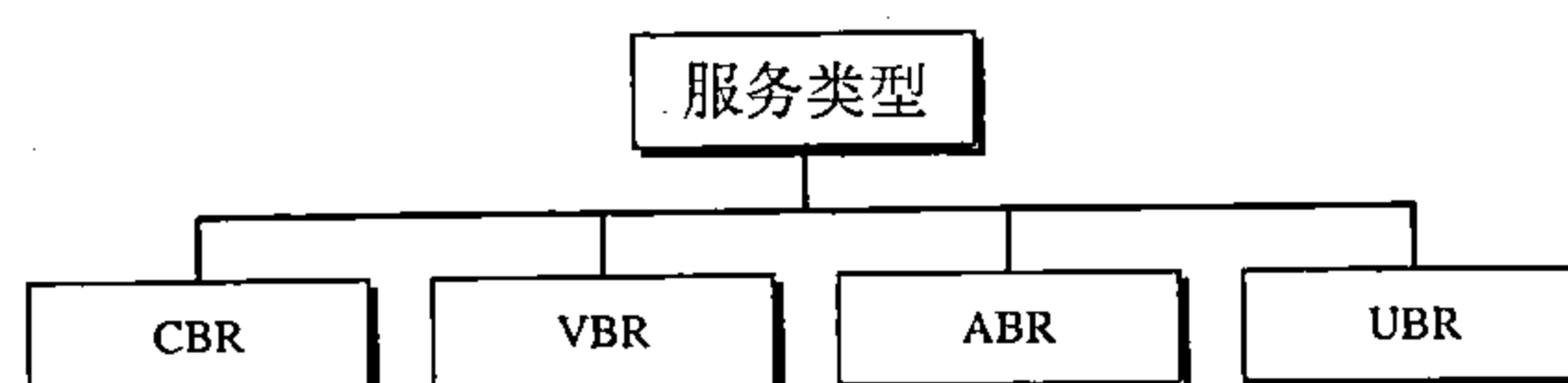


图24.30 服务类型

CBR 恒定比特率（constant-bit-rate, CBR）类用于需要实时音频或视频服务的客户。这个服务与诸如T线之类的专线提供的服务类似。

VBR 可变比特率（variable-bit-rate, VBR）类分为两个子类：实时（VBR-RT）和非实时（VBR-NRT）。VBR-RT用于需要实时服务（如声音和视频传输）和使用压缩技术来生成可变比特率的用户；VBR-NRT用于不需要实时服务，但是使用压缩技术生成可变比特率的那些用户。

ABR 可用比特率（available-bit-rate, ABR）类以最小的速率发送信元。如果有更多的网络资源可用的话，那么就能突破这个最小比特率。ABR特别适用于突发性的应用。

UBR 未指定比特率（unspecified-bit-rate, UBR）类是一种尽力传递但没有任何保证的服务。

图24.31说明了不同类的服务与网络总容量的关系。

用户相关属性

ATM定义了两组属性。用户相关属性是定义用户要以多快的速率发送数据的属性，是在用户和网络之间约定时协商而定的。下面是一些用户相关属性：

SCR 持续信元速率 (sustained cell rate, SCR) 是一段较长时间间隔的平均信元速率。实际信元速率可能低于或高于这个值，但是平均值应该等于或小于SCR。

PCR 峰值信元速率 (peak cell rate, PCR) 定义了发送方的最大信元速率。只要能维持SCR，用户的信元速率有时就能达到该峰值。

MCR 最小信元速率 (minimum cell rate, MCR) 定义了发送方能接受的最小信元速率。例如，如果MCR是50 000，那么网络必须保证发送方至少每秒发送50 000个信元。

CVDT 信元可变延迟极值 (cell variation delay tolerance, CVDT) 是表示信元传输时间变化量的一个量。例如，如果CVDT是5ns，则表示在传递信元的过程中，最大延迟和最小延迟之差不能超过5ns。

网络相关属性

网络相关属性定义了网络的特性。下面是一些网络相关属性：

CLR 信元丢失率 (cell loss ratio, CLR) 定义的是在传输期间信元丢失（或传递得太迟以至于它们被视为丢失）的比率。例如，如果发送方发送了100个信元，丢失了1个信元，则CLR是

$$\text{CLR} = \frac{1}{100} = 10^{-2}$$

CTD 信元传送延迟 (cell transfer delay, CTD) 是一个信元从源端传输到目的端所需要的平均时间。最大CTD和最小CTD也属于网络相关属性。

CDV 信元延迟变化量 (cell delay variation, CDV) 是CTD的最大值和最小值之差。

CER 信元差错率 (cell error ratio, CER) 定义错误传递信元的比率。

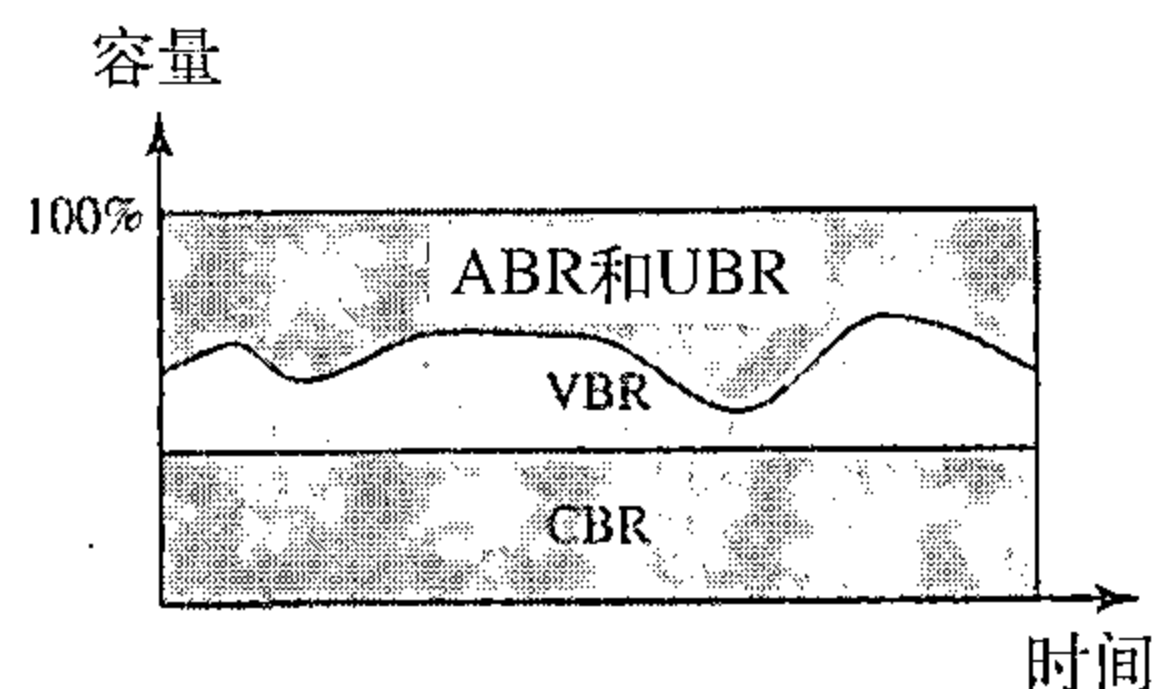


图24.31 不同类型服务和网络总容量间的关系

推荐读物

为了深入讨论与本章有关的主题，推荐下列读物和网站，在本书末列出方括号[……]中的参考资料。

读物

拥塞控制和QoS在[Tan03]中的5.3节和5.5节以及[Sta98]的第3章中有讨论。关于QoS入门读物可参看[FH98]。在[Bla00]中有关于QoS的完整讨论。

本章小结

- 平均数据速率、峰值数据速率、最大突发长度和有效带宽是用来描述数据流的定性值。
- 数据流可以具有恒定比特率、可变比特率或者突发性通信量。
- 拥塞控制是指控制拥塞和保持载荷低于容量的机制和方法。
- 延迟和吞吐量可衡量网络的性能。
- 开环拥塞控制是预防拥塞；闭环拥塞控制是消除拥塞。
- TCP通过使用两个策略来避免拥塞：慢速启动与加性增加相结合和乘性减少。

- 帧中继通过使用两个策略来避免拥塞：后向显式拥塞通知（BECN）和前向显式拥塞通知（FECN）。
- 数据流可以由可靠性、延迟、抖动和带宽这些特性来描述。
- 调度、通信量整形、资源预留和许可控制都是提高服务质量（QoS）的方法。
- FIFO队列、优先权队列和加权公平队列都是调度方法。
- 漏桶和令牌桶是通信量整形方法。
- 综合业务是用于IP的基于数据流的QoS模型。
- 资源预留协议（RSVP）是一种信令协议，它帮助IP生成数据流，并建立资源预留。
- 差分业务是用于IP的基于类的QoS模型。
- 在帧中继中，访问速率、提交突发业务量、提交信息速率和超突发长度都是控制通信量的属性。
- ATM中的服务质量是基于类、用户相关属性和网络相关属性的。

习题

复习题

1. 拥塞控制和服务质量的关系如何？
2. 什么是通信量描述符？
3. 平均数据速率和峰值数据速率之间关系是什么？
4. 突发性数据的定义是什么？
5. 开环和闭环拥塞控制之间的区别是什么？
6. 试列出防止拥塞的一些策略的名称。
7. 试列出减轻拥塞的一些机制。
8. 在TCP中，发送窗口大小由什么来决定？
9. 帧中继如何控制拥塞？
10. 哪些特性可以用来描述数据流？
11. 提高服务质量的四个常用方法是什么？
12. 什么是通信量整形？试列出通信量整形的两种方法的名称。
13. 综合业务和差分业务的主要区别是什么？
14. 资源预留协议和综合业务有什么关系？
15. 在帧中继中用于通信量控制的属性是什么？
16. 在ATM中，就服务质量来说，网络相关性与用户相关性的差别是什么？

练习题

17. 帧中继的地址字段是1011000000010111。试问在前向有拥塞吗？在后向有拥塞吗？
18. 一个帧从A到B，在两个方向都有拥塞。试问FECN位是否置位？BECN位是否置位？
19. 使用漏桶控制液体的流量，如果输出速率是5gal/min，输入突发性速率是100gal/min，持续12s，在48s内没有任何输入。试问桶中还剩下多少加仑(gal)的液体？
20. 交换机的输出接口使用漏桶算法以8000字节/s（节拍）的速率发送数据。如果按下列的帧顺序接收，试说明每秒发送了哪些帧。
 帧1, 2, 3, 4: 每帧4000字节
 帧5, 6, 7: 每帧3200字节
 帧8, 9: 每帧400字节

帧10, 11, 12: 每帧2000字节

21. 一个用户通过T-1线连接到帧中继网络, 允许的CIR是1Mbps, B_c 是5Mb/5s而 B_e 是1Mb/s。试回答下列问题:
- 访问速率是多少?
 - 用户能以1.6Mbps发送数据吗?
 - 用户能始终以1Mbps发送数据吗? 在这种情况下, 它能保证不丢失帧吗?
 - 用户能始终以1.2Mbps发送数据吗? 在这种情况下, 它能保证不丢失数据吗? 如不是, 它是否能保证只有在网络发生拥塞时, 才会丢失帧?
 - 以恒定比特率1.4Mbps重复d的问题。
 - 用户能始终使用而无需担心帧丢失的最大数据速率是多少?
 - 如果用户想冒险, 那么在网络没有拥塞, 没有丢弃帧的情况下, 能使用的最大数据速率是多少?
22. 在练习21中, 用户以1.4Mbps发送了2s的数据, 接下来的3s内没有发送任何数据, 如果网络没有拥塞, 试问有丢弃帧的危险吗? 如果网络中有拥塞, 有丢弃帧的危险吗?
23. 如果每个信元都经过10 μ s到达目的端, CTD的值是多少?
24. 在10 000个信元中, 网络已丢失了5个信元, 其中2个是错误信元, 那么CLR的值是多少? CER的值是多少?

第六部分 应用层

目标

应用层使用户能够访问网络，用户可以是人或软件。它为用户提供服务的接口和支持，如电子邮件、文件访问和传输、访问系统资源、游览万维网，及网络管理。

应用层负责向用户提供服务。

本部分简要地讨论为因特网中的客户机/服务器模式设计的一些应用程序。客户机向服务器发送一个服务请求，而服务器对客户机做响应。

本书第六部分重点介绍应用层及其提供的服务。

第25章 域名系统

在因特网模型的应用层中，许多应用都遵循客户机/服务器模式。客户端/服务器应用程序可以分成两类：一种直接被用户使用，例如电子邮件；另一种是支持其他应用程序的应用程序。域名系统（DNS）就是一种支持程序，它被其他的应用程序使用，例如电子邮件。

图25.1举例说明了DNS客户端/服务器程序如何支持电子邮件程序找到电子邮件接收者的IP地址。电子邮件程序的用户可以知道接收者的电子邮件地址，但是IP协议需要知道IP地址。DNS客户端程序就向DNS服务器发送查询请求，获取与电子邮件地址相对应的IP地址。

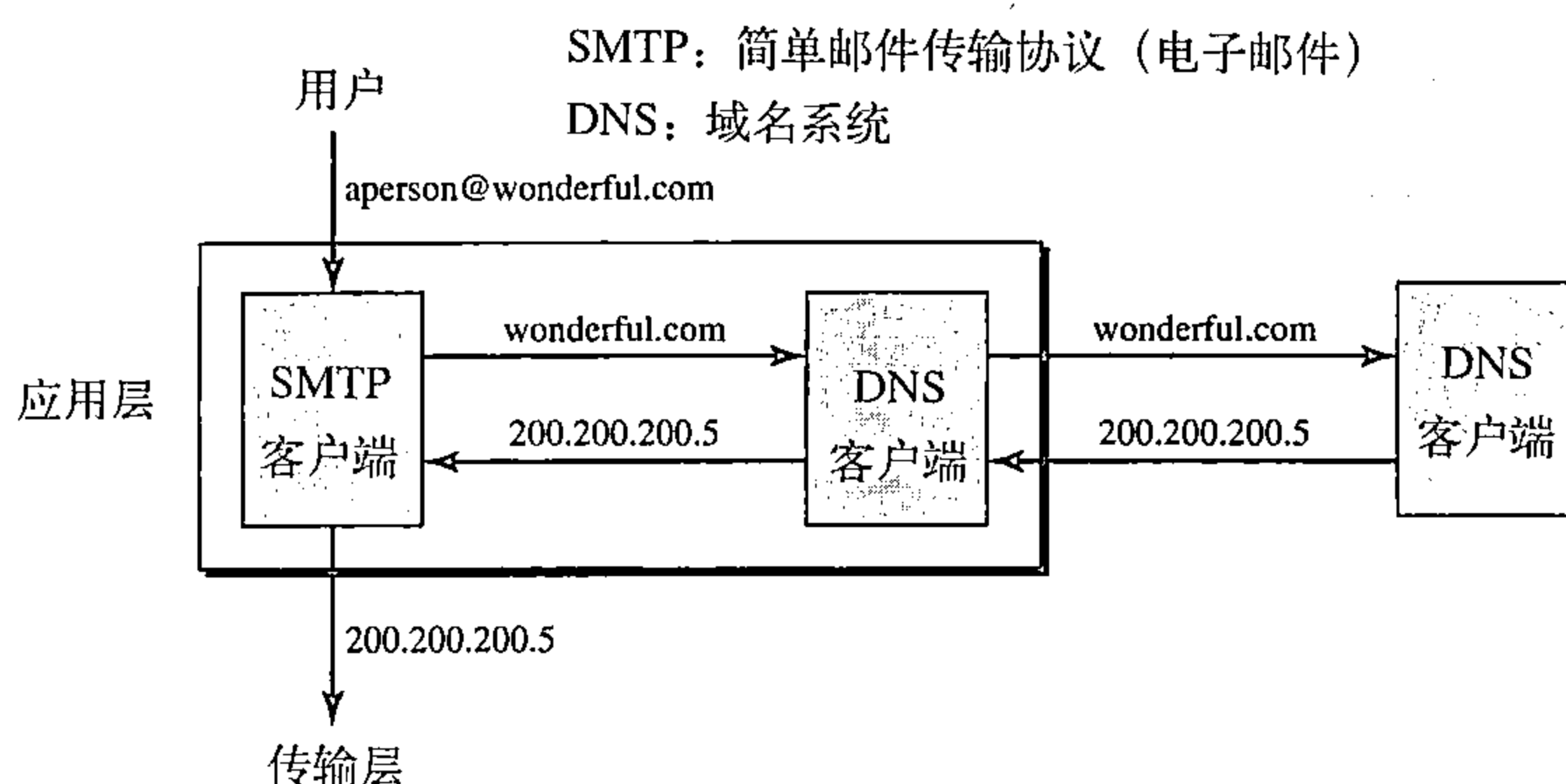


图25.1 使用DNS服务的例子

为了识别一个实体，TCP/IP协议使用IP地址唯一地确定一台主机到因特网的连接。然而，人们更喜欢使用名字而不是数字地址。所以，需要一种能够完成名字到地址或者地址到名字的映射的系统。

在因特网发展初期，使用一个主机文件（host file）实现映射。主机文件只有两列：名字和地址。每台主机可以将主机文件存储在主机磁盘中，并以一个标准主机文件为依据定期进行更新。当程序或者用户需要将某一名字映射到一个地址时，主机就查询主机文件并找到相应的映射。

但目前，不再可能用一个单独的主机文件将每一个地址与名字关联起来，反过来也是一样。主机文件会因为太大而无法存储在每一台主机中。另外，也不能每次在发生变化时，更新世界上所有的主机文件。

一种解决方案是将全部主机文件存储在一台单独的计算机中，允许每一台需要映射的计算机访问这个集中化的信息。但是，这样会在因特网上产生非常大的通信量。

另一种解决方案，也是目前所使用的，是将这种巨大的信息数据分割为许多更小的部分，并将每一部分存储在不同的计算机中。采用这种方式，需要映射的计算机可寻找到最近一台持有所需信息的计算机。域名系统（Domain Name System，DNS）就使用这种方法。本章先讨论DNS的概念和思想，然后描述DNS协议本身。

25.1 名字空间

为实现无二义性，分配给机器的名字必须从名字空间（name space）中仔细地选择。该名

字空间完全控制对名字和IP地址的绑定。换句话说，因为地址是唯一的，所以名字也必须唯一的。名字空间将每一个地址映射到一个唯一的名字，它可以按两种方式进行组织：平面的和层次的。

25.1.1 平面名字空间

在平面名字空间 (flat name space) 中，一个名字分配给一个地址。空间中的名字是一个无结构的字符序列。名字之间可能有也可能没有公共部分；即使有公共部分，也没有实际含义。平面名字空间的主要缺点是它必须集中控制才能避免二义性和重复，因而不能用于如因特网这样的大规模系统中。

25.1.2 层次名字空间

在层次名字空间 (hierarchical name space) 中，每一个名字由几个部分组成。第一部分可以定义组织的性质，第二部分可以定义一个组织的名字，第三部分可以定义组织的部门，等等。在这种情况下，分配和控制名字空间的机构就可以分散化。中央管理机构可以分配名字的一部分，这部分定义组织的性质和组织的名字。名字其他部分的分配可交给这个组织自身。这个组织可以给名字加上后缀（或前缀）来定义主机或者其他资源。这个组织机构的管理机构不必担心为一个主机选择的后缀会被其他组织机构所采用，因为即使地址的某一部分相同的，整个地址也是不同的。例如，假定两所学院和一个公司都将它们的一台计算机命为 challenger。第一所学院由中央管理机构分配的名字是 fhda.edu，第二所学院获得的名字是 berkeley.edu，而公司获得的名字是 smart.com。当这些组织的每一个在已有的名字加上名字 challenger 后，得到了三个不同的名字：challenger.fhda.edu、challenger.berkeley.edu 和 challenger.smart.com。这些名字是唯一的，而不需要由中央管理机构来分配。中央管理机构只控制名字的一部分，而不是整个名字。

25.2 域名空间

为了获得层次结构的名字空间，设计了域名空间 (domain name space)。在这种设计方式中，所有的名字由根在顶部的倒置树结构定义。该树最多有128级：0级（根节点）~127级（见图25.2所示）。

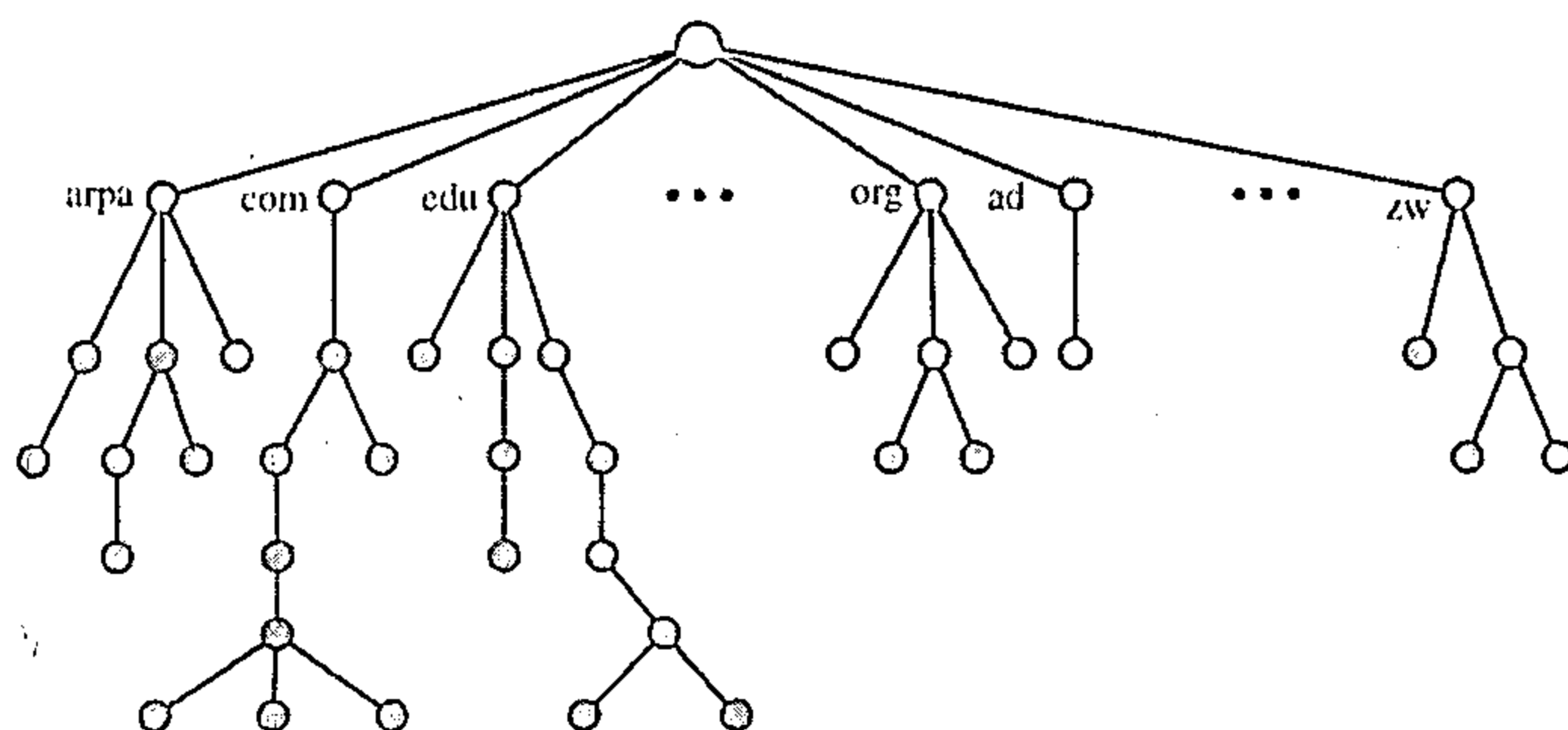


图25.2 域名空间

25.2.1 标号

树上的每一个节点有一个标号 (label)。标号是一个最多为63个字符的字符串。根节点的标号是空字符串（空串）。DNS要求每一个节点的子节点（从同一节点分支出来的节点）有不

同的标号，这样就确保了域名的唯一性。

25.2.2 域名

树上的每一个节点都有一个域名。一个完全的域名 (domain name) 是用点 (.) 分隔的标号序列。域名总是从节点向上读到根节点。最后一个标号是根节点的标号 (空)。这表示一个完全的域名总是以一个空标号结束，也意味着最后一个字符是一个点 (.)，因为空字符串表示什么也没有。图25.3给出了某些域名。

全称域名

如果一个标号以一个空字符串结束，则它就称全称域名 (fully qualified domain name, FQDN)。全称域名是包含一台主机所有名字的域名。它包含所有的标号，从最具体的到最一般的标号，并能唯一地定义一台主机的名字。例如，域名

`challenger.atc.fhda.edu.`

是一个全称域名，代表安装在De Anza大学的高级技术中心 (ATC) 的一台名字为`challenger`的计算机。DNS服务器只能和FQDN的一个地址相匹配。注意：名字必须以空标号结束，但是由于空标号表示没有东西，所以这种标号以一个点 (.) 结束。

部分域名

如果一个域名不是以空字符串结束，则称为部分域名 (partially qualified domain name, PQDN)。部分域名起始于一个节点，但没有到达根节点。当这个需要解析的名字属于和客户机相同的站点时使用部分域名。这种情况下，解析程序能够提供省略的部分，称为后缀 (suffix)，以创建FQDN。例如，如果在`fhda.edu`站点的一个用户想要获取`challenger`计算机的IP地址，用户就可定义这个部分域名

`challenger`

DNS客户在将地址传送到DNS服务器后就加上后缀`atc.fhda.edu.`。

DNS客户端通常会保存一个后缀列表。下面这些是De Anza大学的后缀列表。空后缀表示什么也没有，当用户定义一个全称域名的时候，就会加上这个后缀。

`atc.fhda.edu`
`fhda.edu`
`null`

图25.4表示了一些FQDN和PQDN。

25.2.3 域

域 (domain) 是域名空间的一棵子树。这个域的名字是子树顶部节点的域名。图25.5列出了一些域。注意：一个域本身也可以再分划为多个域，(也就是平时所说的子域 (subdomain))。

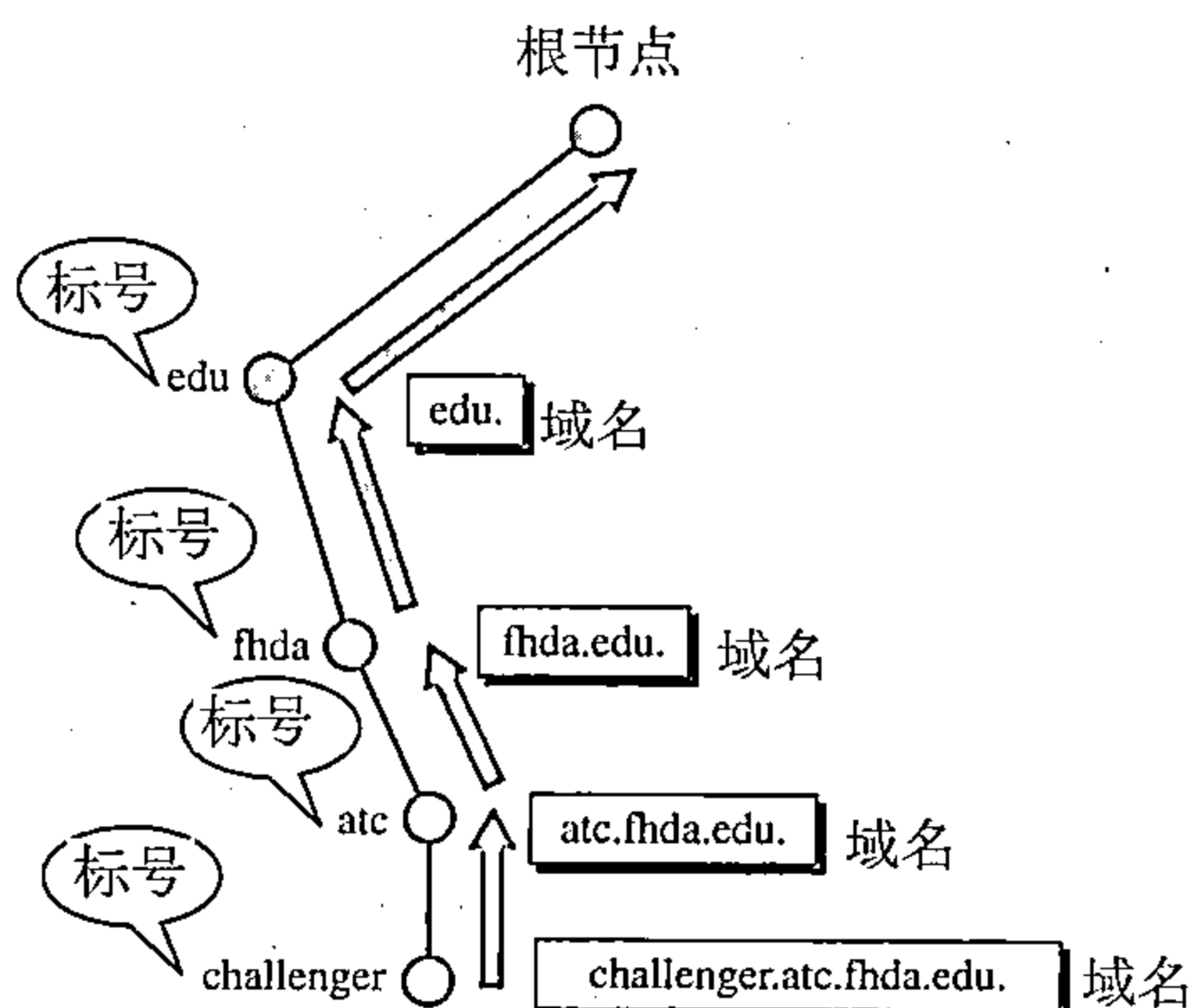


图25.3 域名和标号

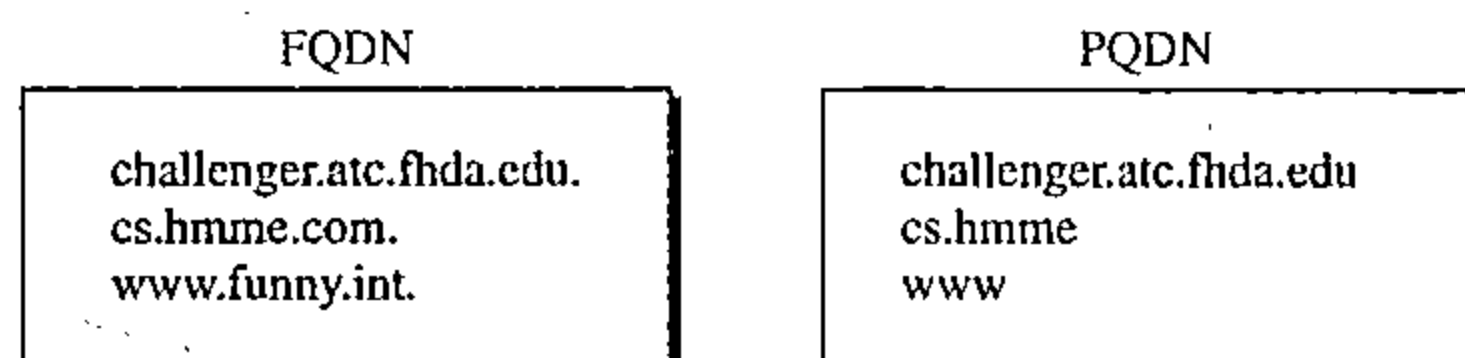


图25.4 FQDN和PQDN

25.3 名字空间的分布

必须将域名空间所包含的信息存储起来。然而，只使用一台计算机存储如此大容量的信息，效率非常低和不安全的。效率低的主要是因为响应来自世界各地的请求会给系统造成非常大的负荷，不安全的原因是因为任何故障将使整个数据库无法使用。

25.3.1 名字服务器的层次结构

解决这些问题的办法是将信息分布在多台称为DNS服务器 (DNS server) 的计算机中。一种方法是将整个空间划分为多个基于第一级的域。换句话说，让根节点保持不动，但创建许多与第一级节点一样多的域 (子树)。这样创建的域会很大，DNS允许将域进一步划分为更小的域 (子域)。每一台服务器对一个大的域或者较小的域是负责的 (授权的)。换句话说，与建立名字的层次结构一样，也建立了服务器的层次结构 (见图25.6)。

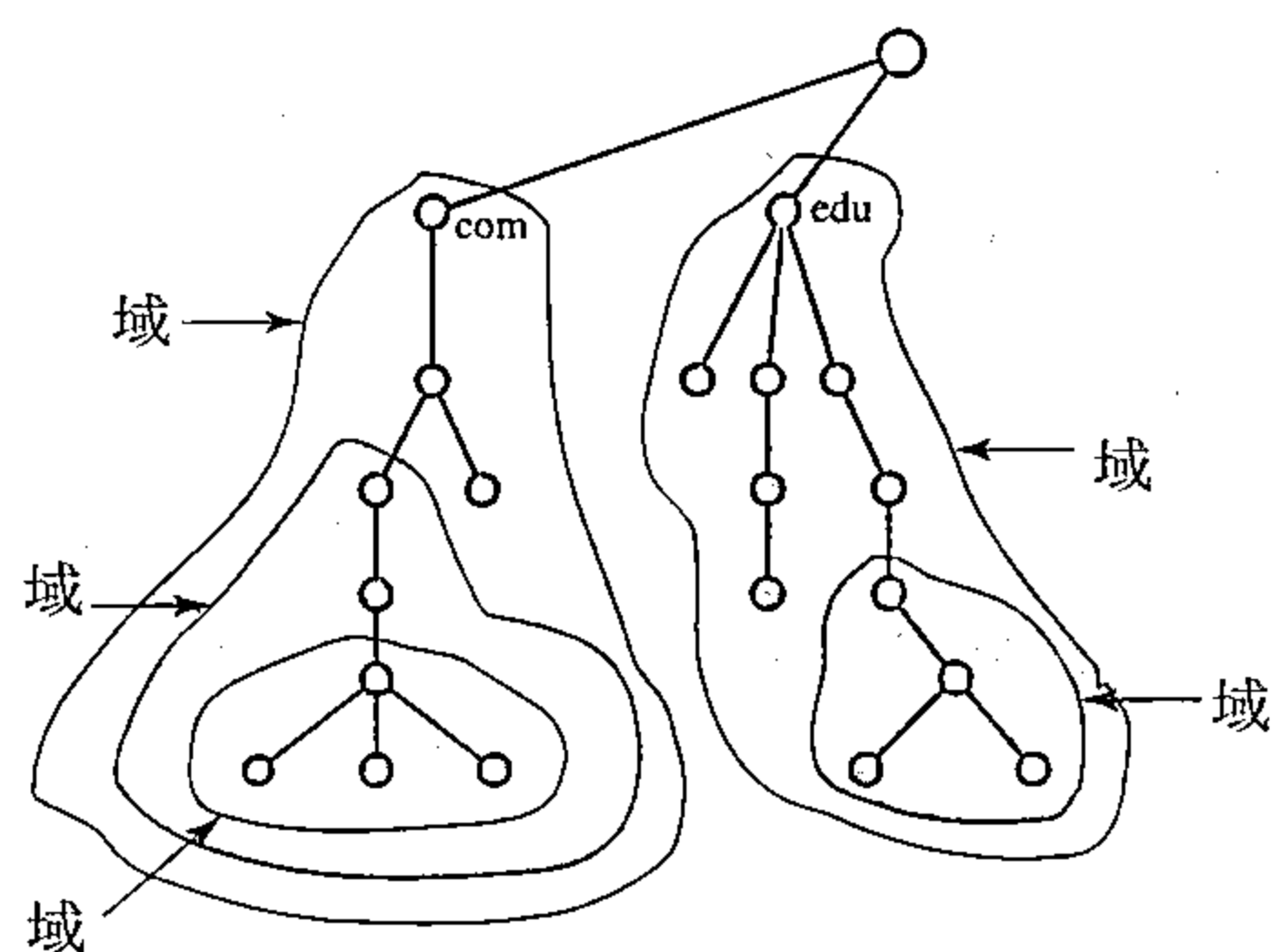


图25.5 域

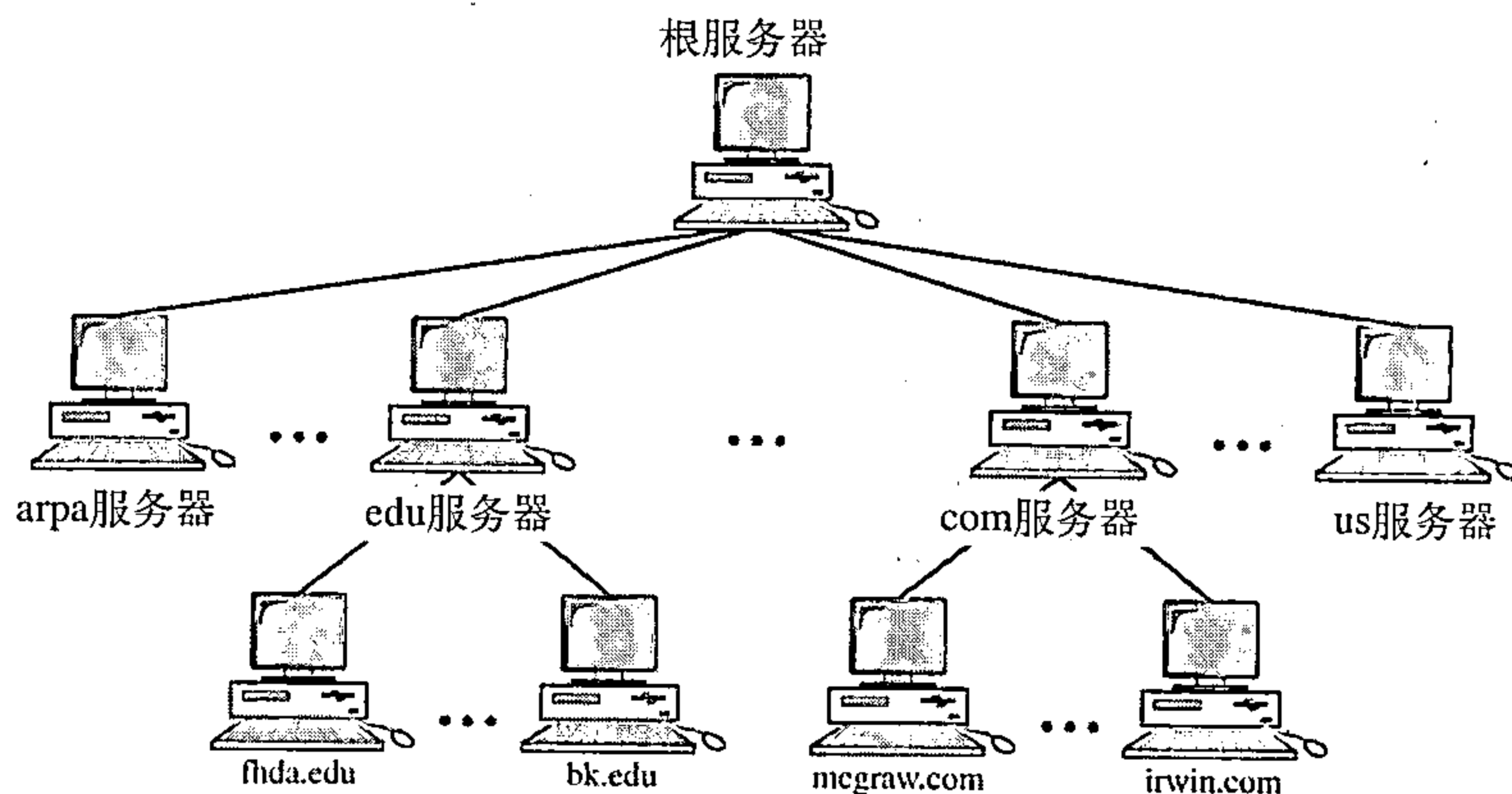


图25.6 名字服务器的层次结构

25.3.2 区域

既然完整的域名层次结构不能保存在单一的服务器上，那么它被分在多个服务器上。一个服务器负责或者授权的范围，称为区域 (zone)。我们可以将一个区域定义为整个树中的一个连续的部分。如果服务器负责一个域，而且这个域并没有进一步被划分为更小的子域，此时域和区域是相同的。服务器有一个数据库，称为区域文件，它保存这个域里所有节点的信息。然而，如果服务器将它的域划分为多个子域，并将其部分授权委托给其他服务器，那么域与区域就不同了。在子域节点的信息会存放在较低层次的服务器中，原来的服务器则保存这些较低层次服务器的某种参照。当然，原来的服务器并不是完全不负责任，它仍然拥有一个区域，只是将详细的信息保存在较低层次的服务器上。(见图25.7所示)。

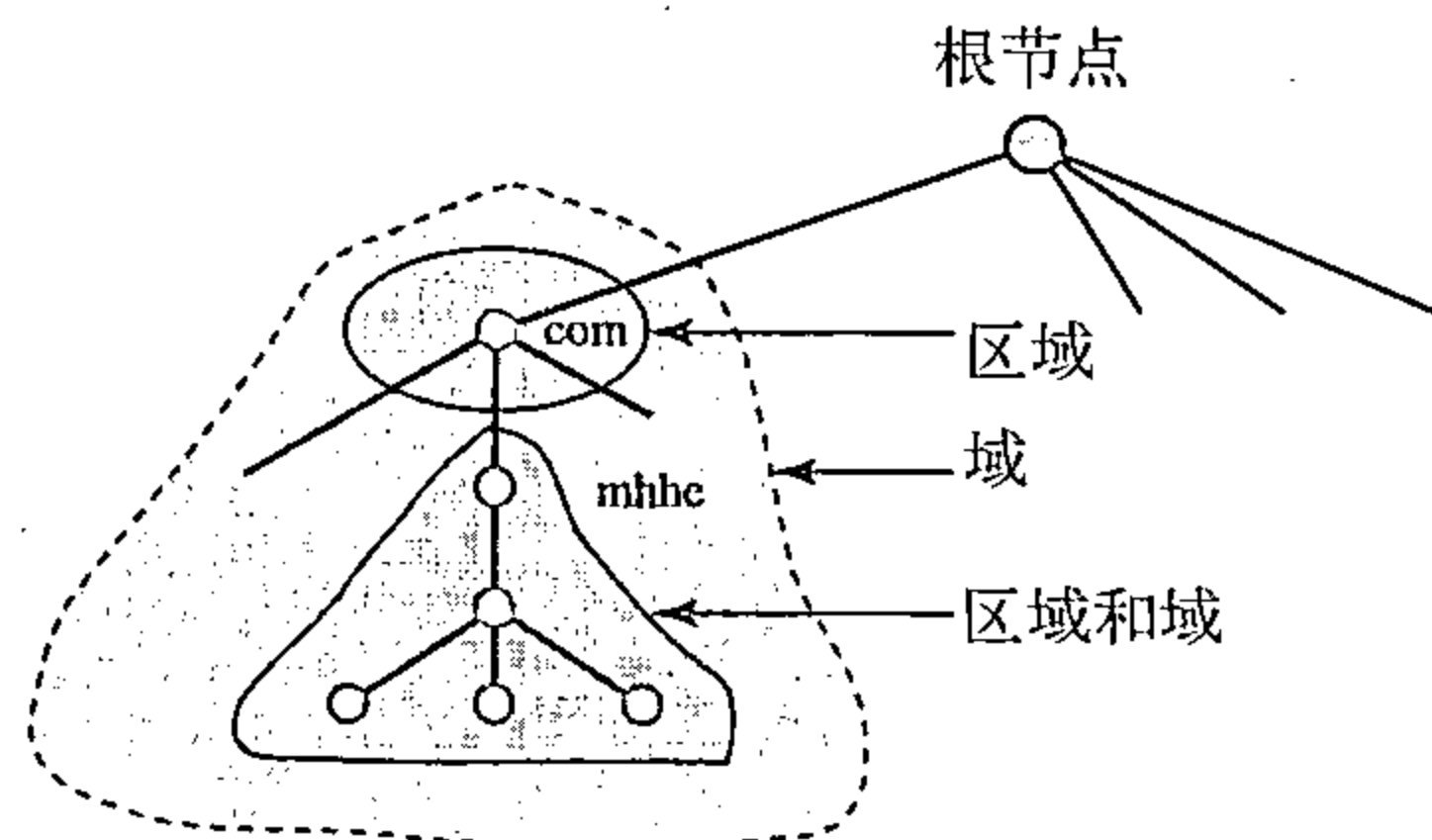


图25.7 区域和域

一台服务器可以将自己域进行划分并委托责任，但是仍然为它自己保存了一部分。在这种情况下，它的区域是由具有详细信息的那部分域所组成，它没有被委托权利，但它有到被委托那些部分的参照。

25.3.3 根服务器

根服务器 (root server) 是指它的区域由整棵树组成的服务器。根服务器通常不保存关于域的任何信息，只是将其授权托给其他服务器，并保存与这些服务器的参照关系。目前有多个根服务器，每一台都覆盖了整个域名空间。这些服务器分布在世界各地。

25.3.4 主服务器和辅助服务器

DNS定义了两类型的服务器：主服务器和辅助服务器。主服务器 (primary server) 是指存储了授权区域有关文件的服务器。它负责创建、维护和更新区域文件，并将区域文件存储在本地磁盘中。

辅助服务器 (secondary server) 负责从另一个服务器 (主服务器或者辅助服务器) 传输一个区域的全部信息，并将文件存储在它的本地磁盘中。辅助服务器既不创建也不更新区域文件。如果需要更新，则必须由主服务器来完成，由主服务器发送更新的版本到辅助服务器中。

主服务器和辅助服务器对它们所服务的区域都有控制权。这种设计思想并不是把辅助服务器置于一个较低的授权层次上，而是为了建立数据的冗余备份，这样当一台服务器出现故障时，另一台服务器可以继续为客户机服务。注意：一台服务器可能是某个特定区域的主服务器，同时也是另一个区域的辅助服务器。所以，当提到一个服务器作为主服务器或者辅助服务器时，需要弄清楚所指的是哪个区域。

主服务器能够从磁盘文件中装载所有信息，辅助服务器从主服务器中装载信息。当辅助服务器从主服务器中下载信息时，这称为区域的传递。

25.4 因特网中的DNS

DNS是一种可以在不同平台上使用的协议。在因特网中，域名空间 (树) 被划分为三个部分：通用域、国家域和反向域 (见图25.8所示)。

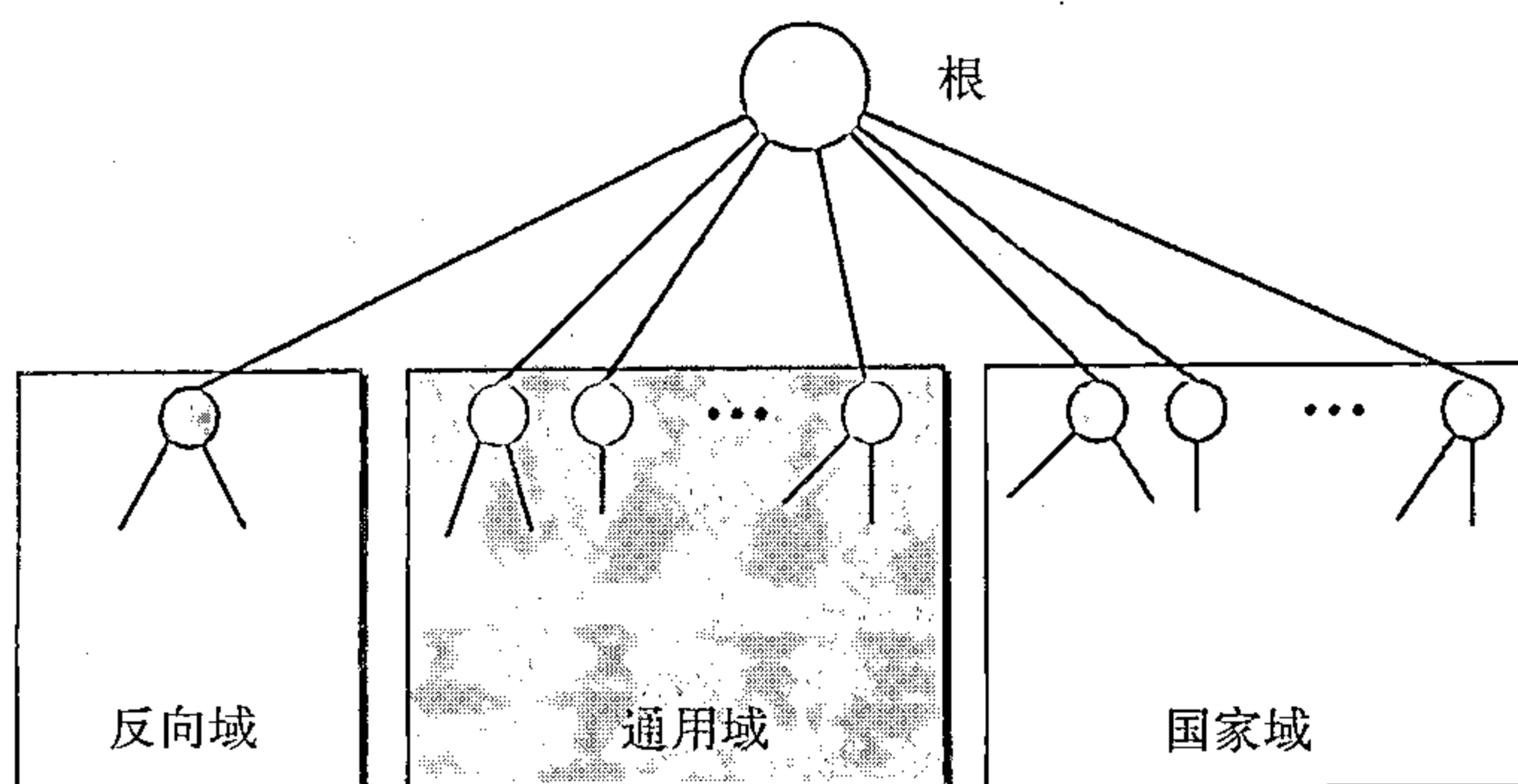


图25.8 因特网中的DNS

25.4.1 通用域

通用域 (generic domain) 按照已经注册主机的一般行为对主机进行定义。树中的每一个

节点定义一个域，它是到域名空间数据库的一个索引（见图25.9所示）。

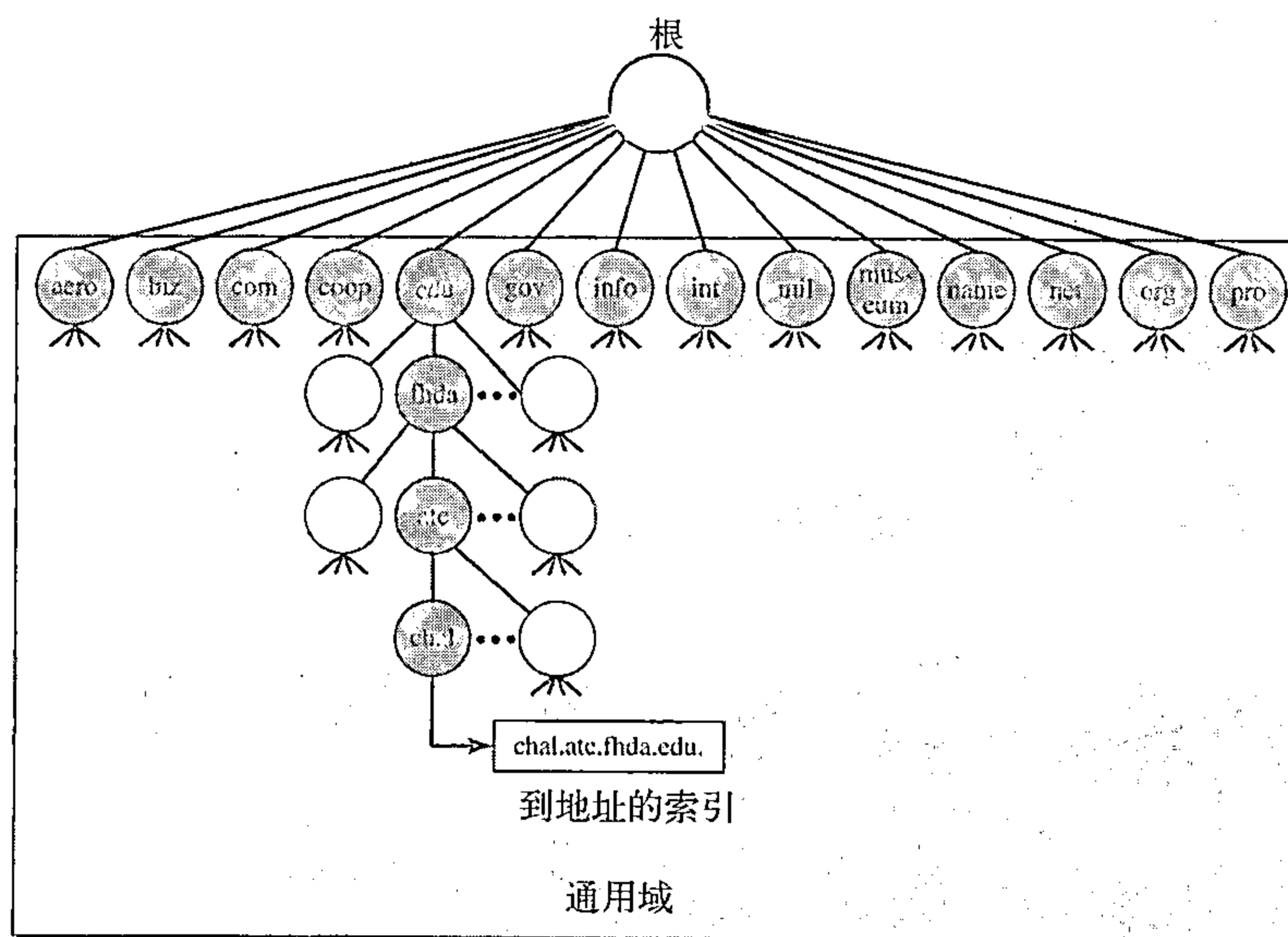


图25.9 通用域

我们从这棵树中可以看出，在通用域的第一层允许有14个可能的标号。这些标号描述了表25.1中列出的组织机构类型。

表25.1 通用域标号

标 号	描 述	标 号	描 述
aero	航空航天公司	int	国际机构
biz	商业或者公司（与com类似）	mil	军事机构
com	商业机构	museum	博物馆和其他非盈利性组织
coop	协作商业组织	name	私人名字（个人名字）
edu	教育机构	net	网络支持中心
gov	政府机构	org	非盈利性组织
info	信息服务提供商	pro	专业个人组织

25.4.2 国家域

国家域（country domain）部分使用两个字母的国家缩写（例如us代表美国），第二级标号可以是组织机构，或者更具体一些，由各个国家自己指定。例如，美国使用州的缩写作为国家域的子域划分（例如ca.us.）。

图25.10列出了国家域部分。地址anza.cup.ca.us可以理解为美国加州的Cuperetino 的De Anza学院。

25.4.3 反向域

反向域（inverse domain）用于将地址映射为名字。例如，当服务器接收到客户机请求完成某项任务的请求时，就会发生这种情况。尽管服务器中有一个包含着授权客户的列表文件，但文件中只列出了客户机的IP地址（从接收到的IP分组中提取出来）。为了确定该客户端是否在授权的列表中，服务器就用它的解析程序向DNS服务器发送一个查询，并请求将地址映射为名字。

这种类型的查询，称为反向或者指针（PTR）查询。为了处理一个指针查询，在域名空间中要增加一个反向域，并且其第一级节点称为*arpa*（由于历史原因）。第二级仍然是单节点，称为*in-addr*（用于反向地址）。域的其他部分定义IP地址。

处理反向域的服务器也是层次结构的。这意味着地址的网络号比子网号的层次更高，而子网号所处的层次要高于主机号所处的层次。同样，为整个网站服务的服务器应该比为子网服务的服务器处于更高层次。与通用域或者国家域相比，这种配置方法使得反向域看起来是倒置的。按照由下向上读取标号的约定，一个IP地址如132.34.45.121（一个B类地址，网络号是132.34）会读取为121.45.34.132.in-addr.arpa。参见图25.11关于反向域配置说明的图例。

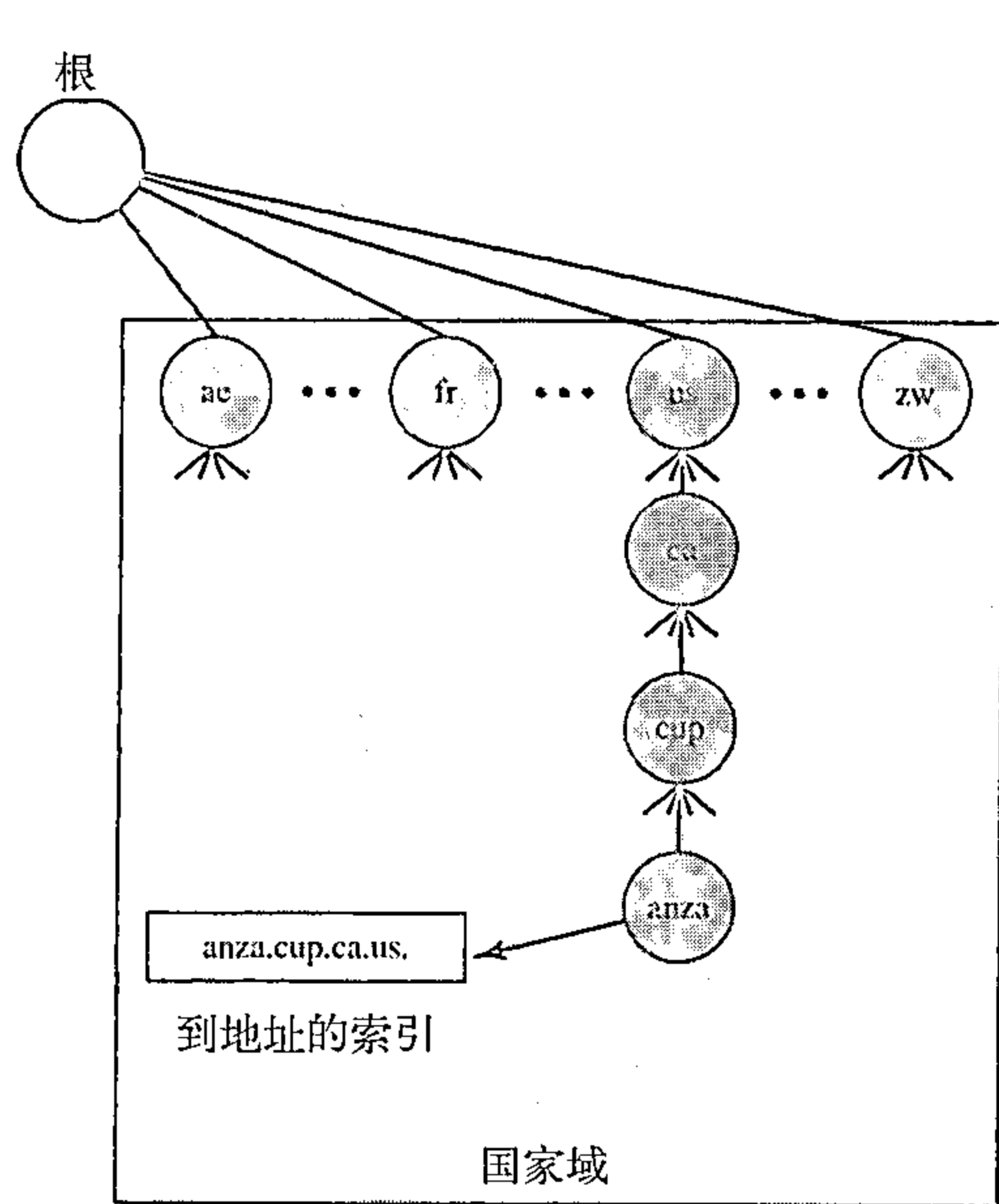


图25.10 国家域

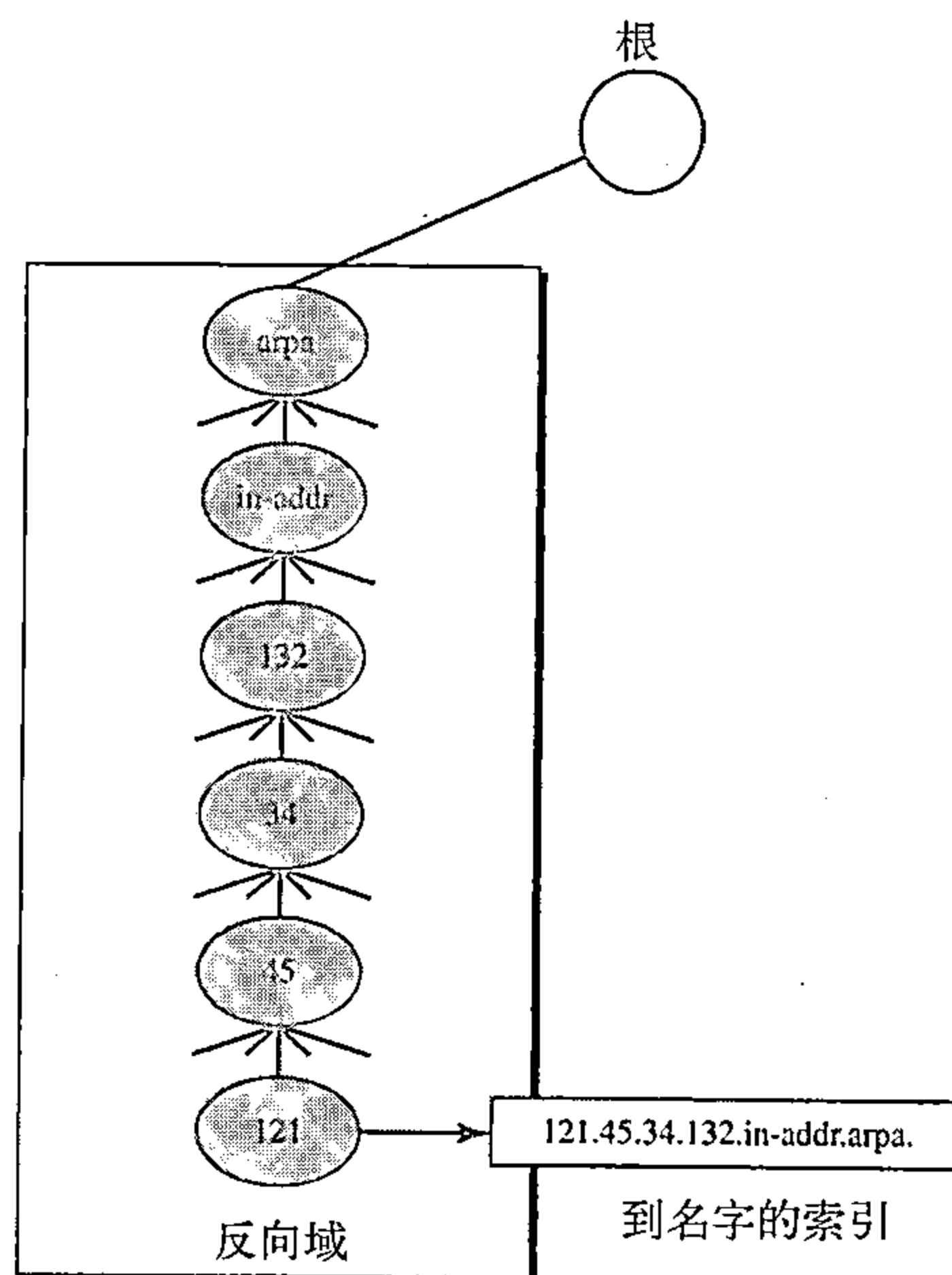


图25.11 反向域

25.5 解析

将名字映射为地址或者将地址映射为名字的过程，称为名字-地址解析。

25.5.1 解析程序

DNS是一个客户机/服务器应用程序。需要将地址映射为名字或者将名字映射为地址时，主机要调用一个称为解析程序（resolver）的DNS客户程序。解析程序用一个映射请求访问最近的一个DNS服务器。如果服务器含有该信息，它就满足解析程序的请求；否则，它将解析程序交付给其他的服务器，或者查询其他的服务器来提供这种信息。

当解析程序接收到映射后，它解释这一响应，以确定它是一个真正的解析还是一个差错，最后将结果传递给发出这一请求的进程。

25.5.2 名字到地址的映射

多数情况下，解析程序将域名提交给服务器，请求给出对应的地址。这种情况下，服务器检查通用域或者国家域以查找相应的映射。

如果域名来自于通用域部分，解析程序就会收到一个域名，如`chal.atc.fhda.edu.`。由解析程序将这个查询发送到本地DNS服务器进行解析。如果本地服务器不能解析这一查询，它或

者把解析程序提交给其他的服务器，或者直接询问其他的服务器。

如果域名来自于国家域部分，那么解析程序会接收到类似于`ch.fhda.cu.ca.us`形式的域名。处理过程与上相同的。

25.5.3 地址到名字的映射

客户机向服务器发送IP地址，请求映射为域名。如前所述，这被称为一个PTR查询。要回复这种查询，DNS使用反向域。然而，在这种请求中，IP地址必须反过来，并且将`in-addr`和`arpa`两个标号附加在最后，以创建能够被反向域部分接受的域。例如，如果解析程序接收到的IP地址为`132.34.45.121`，那么解析程序会首先将地址反过来，然后在发送之前附加两个标号。发送的域名是`121.45.34.132.in-addr.arpa.`，它由本地DNS接收并解析。

25.5.4 递归解析

客户端（解析程序）可以从名字服务器中请求递归应答。这意味着解析程序期望服务器提供最终的答案。如果服务器是这一域名的授权服务器，它会检查它的数据库并做出响应。如果服务器不是授权服务器，它会把请求发送给另一个服务器（通常是父服务器），并等待响应。如果父服务器是授权服务器，它就做出响应。否则，它仍然把这个查询发送给另一个服务器。当这个查询最终得到解析后，响应就后向传送，直到最终到达发出请求的客户机。这就是递归解析（recursive resolution），如图25.12所示。

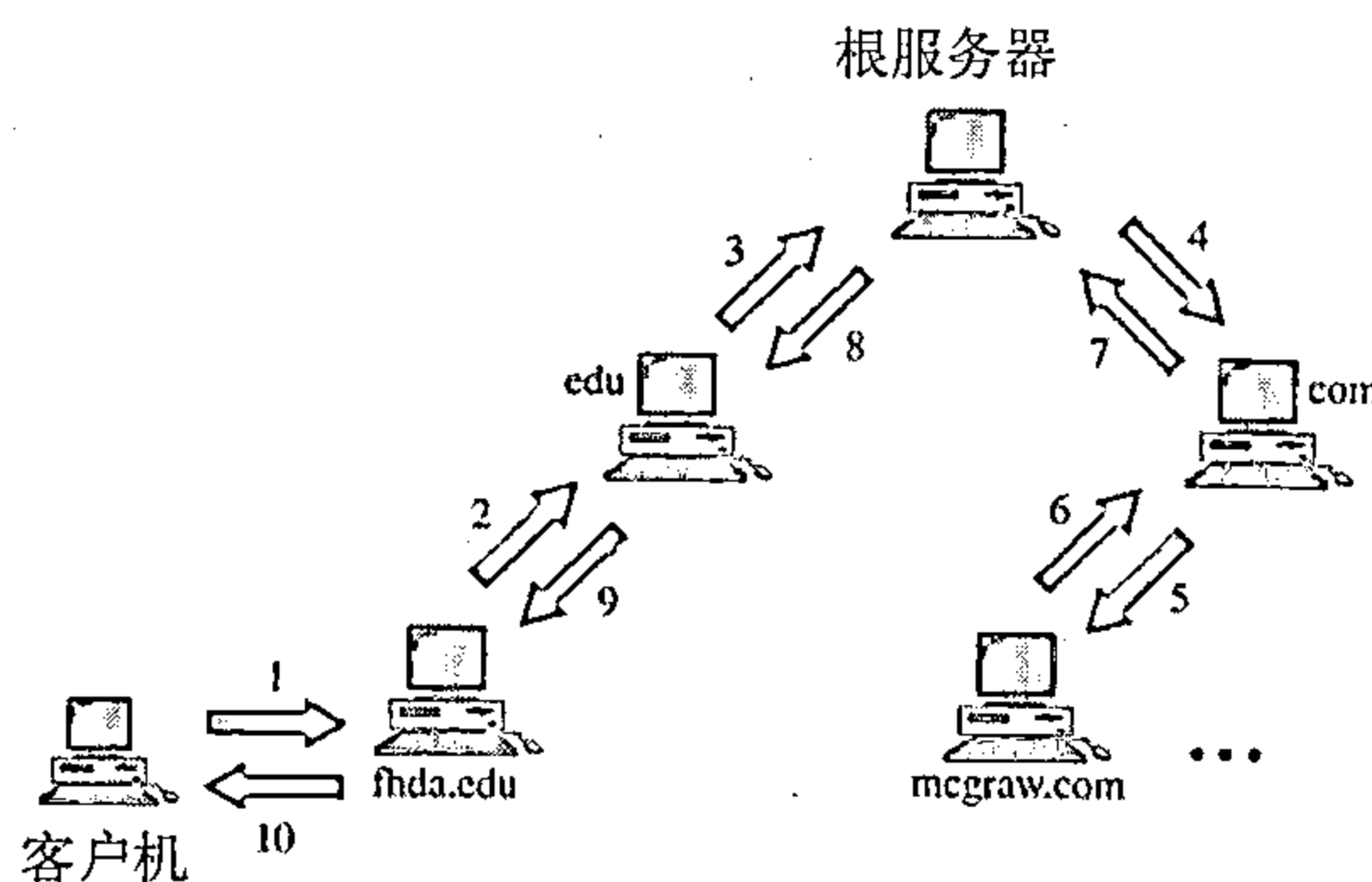


图25.12 递归解析

25.5.5 迭代解析

如果客户端没有请求递归应答，那么映射可以迭代的形式进行。如果服务器是该名字的授权服务器，那么它发送应答；如果它不是，它就返回它认为可以解析该这个查询的服务器的IP地址（给客户端），由客户端负责向第二台服务器重复发送请求。如果新的地址解析服务器能够解析这一问题，那么它就用IP地址响应这一请求；否则，它向客户端返回新服务器的IP地址。这时，客户端必须向第三台服务器重复该请求。这一过程称为迭代解析（iterative resolution），因为客户端向多台服务器重复同样的请求。在图25.13中，客户端在从mcgraw.com服务器得到应答之前，向四台服务器发送了查询请求。

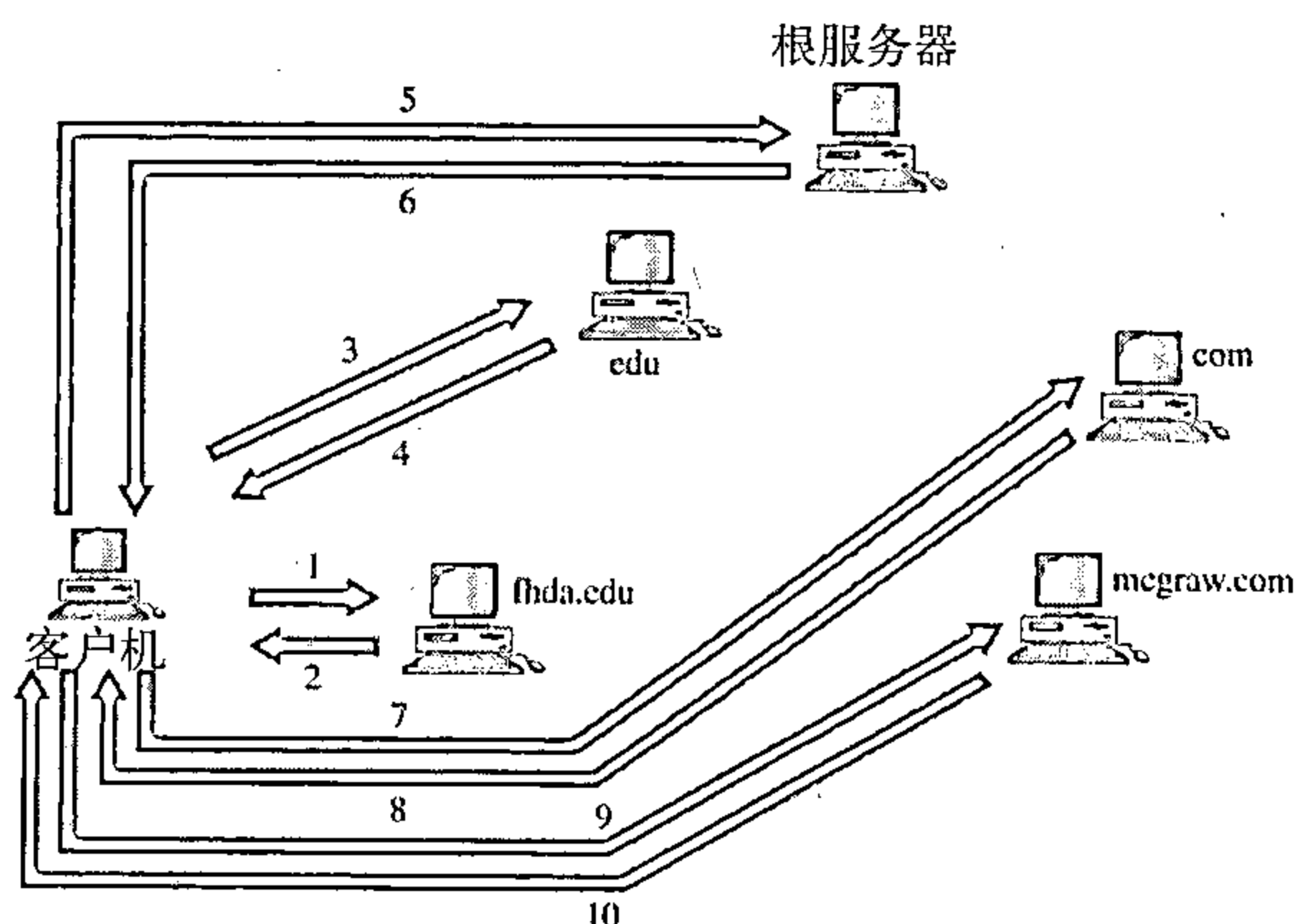


图25.13 迭代解析

25.5.6 高速缓存

每当服务器接收到查询一个不属于自己域的名字时，它需要搜索自己的数据库以查找一

台服务器的IP地址。缩短这一查询时间能提高效率。DNS使用一种称为高速缓存（caching）的机制处理这一问题。当一个服务器向另一个服务器请求映射并得到回应时，它在将该回应发送给客户端之前，先将这一信息存储在高速缓存中。如果同一客户端或者另一个客户端请求同一映射时，它会检查其高速缓存并解决这一问题。然而，要通知客户这一响应来自于高速缓存而不是来自于授权的信息源，该服务器会将这一响应标志为非授权性的。

高速缓存能够加快解析过程，但仍存在问题。如果一台服务器长时间缓存一个映射时，可能会发送给客户端一个过期的映射。为了防止这种情况，使用了两种技术。第一种，授权服务器总是将称为生存时间（TTL）的信息添加在映射上。生存时间定义了接收服务器可以将信息放入高速缓存的时间（以秒计）。超过这一时间，该映射就变为无效，而任何查询必须再次发送到授权服务器。第二种技术，DNS要求每一台服务器对每一个映射保留一个TTL计数器。高速缓存会定期检查，并清除掉TTL已经过期的那些映射。

25.6 DNS报文

DNS有两种类型的报文：查询和响应。这两种类型的报文具有相同的格式。查询报文（query message）由头部和查询记录构成；响应报文（response message）由头部、查询记录、响应记录、授权记录和附加记录组成（见图25.14所示）。

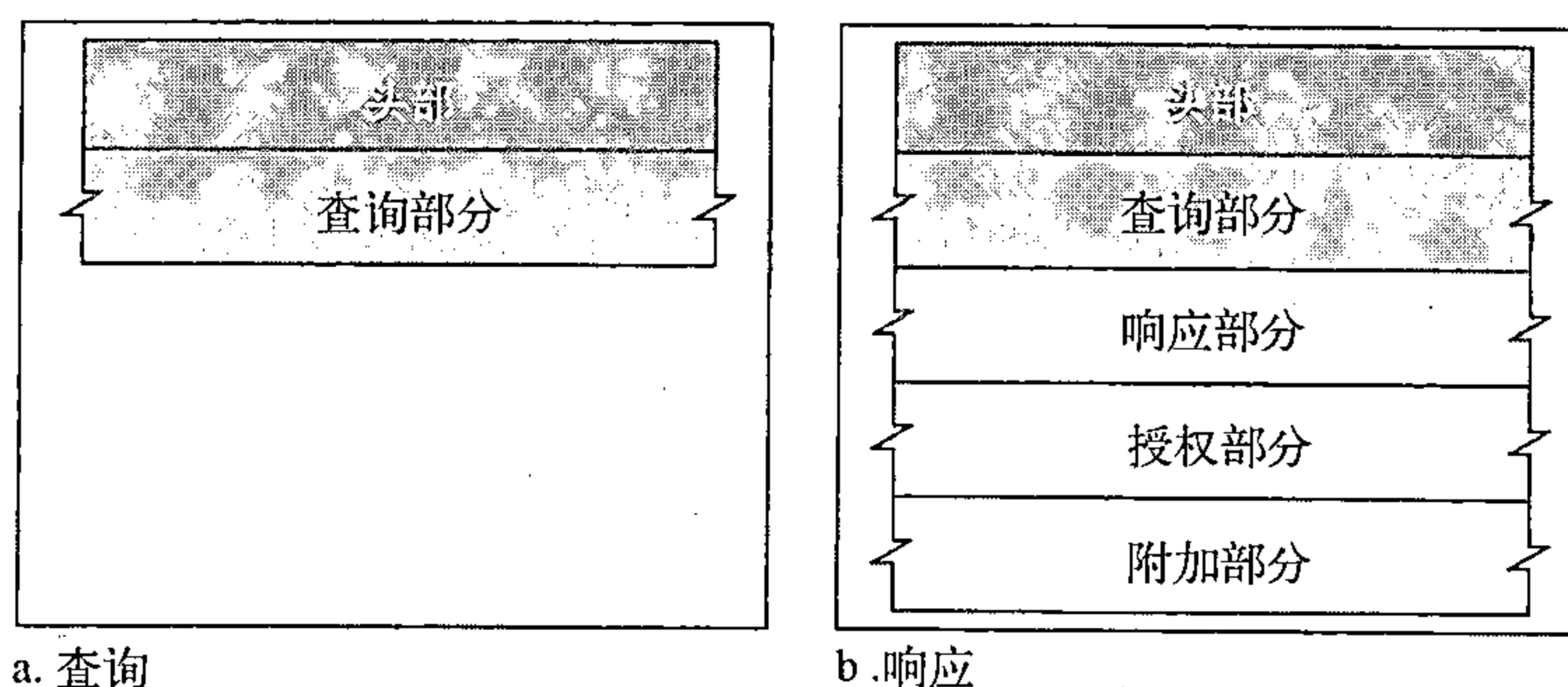


图25.14 查询和响应报文

头部

查询和响应报文的头部格式相同，查询报文头部的某些字段设置为0。头部为12个字节，格式如图25.15所示。

标识	标记
询问记录数	应答记录数
授权记录数 (在查询报文中为全0)	附加记录数 (在查询报文中为全0)

图25.15 头部格式

客户端使用标识子字段来匹配对查询的响应。客户端每次发送查询时会使用不同的标识号。服务器在对应的响应中会重复这一编号。标记是子字段的集合，这些子字段定义了报文的类型、应答的类型、期望的解析类型（递归或迭代）等。询问记录数是指该报文中询问部分所含请求的数量。应答记录数是指响应报文中应答部分所含应答记录的数量。在查询报文中，它的值为0。授权记录数是指响应报文中授权部分所含授权记录的数量。在查询报文中，它的值为0。最后一项，附加记录数是指响应报文中的附加部分所含附加记录的数量。在查询

报文中，它的值为0。

询问部分

它由一条或者多条询问记录构成。查询和响应报文都含有这一部分。在下一节中，我们将讨论询问记录。

应答部分

它由一条或者多条资源记录组成，它仅存在于响应报文中。这一部分包括从服务器到客户端（解析器）的应答。在下一中，我们将讨论资源记录。

授权部分

它由一条或者多条资源记录构成，它仅存在于响应报文中。该部分给出了用于查询的一台或者多台授权服务器的信息（域名）。

附加消息部分

它由一条或者多条资源记录构成，它仅存在于响应报文中。该部分给出了有助于解析程序的附加信息。例如，服务器可以在授权部分为解析程序提供授权服务器的域名，并且把同一授权服务器的IP地址包含在附加信息部分中。

25.7 记录的类型

从25.6节中可以看出，DNS使用了两种类型的记录。在查询和响应报文的询问部分使用了询问记录；在响应报文中的应答、授权、附加信息部分使用了资源记录。

25.7.1 询问记录

客户机使用询问记录（question record）从服务器获取信息，它包含了域名。

25.7.2 资源记录

每一个域名（在树中的每一个节点）都与一个资源记录（resource record）相关联。服务器数据库包含了所有的资源记录，服务器还返回资源记录给客户机。

25.8 注册机构

新的域名是怎样加入到DNS中呢？这是通过注册机构（registrar）来完成的，一个熟知的商业实体是ICANN（因特网名字和编号分配组织）。注册机构首先确认询问的域名是唯一的，然后将它输入到DNS数据库中，这是需要收费的。

现在，有很多的注册机构，他们的名字和地址可以在网站<http://www.intenic.net>中找到。

为了能够注册，组织机构需要给出域名服务器的主机名和IP地址。例如，一个名字为wonderful的商业机构的域名服务器主机名为ws，IP地址为200.200.200.5，就需要给出以下的信息给注册机构：

域名：ws.wonderful.com

IP地址：200.200.200.5

25.9 动态域名系统（DDNS）

在设计DNS时，没有人预料到地址会有如此多的变化。在DNS中，当只有一个变化时，例如增加一台新主机，移除一台主机或改变一个IP地址时，那么这种更改就必然使DNS的主文件发生变化。这些类型的变化涉及许多手工的更新。因特网今天的规模已经不允许使用这

种手工操作。

DNS主文件必须能动态更新。动态域名系统 (Dynamic Domain Name System (DDNS)) 就是为满足这种需求而设计的。在DDNS中, 当名字和地址之间的绑定确定时, 通常是由DHCP (见第21章) 给主DNS服务器发送这种信息。主服务器更新这一区域。通知辅助服务器的方法可以以主动方式或者以被动方式。在主动通知方式中, 主服务器向辅助服务器发送关于区域变化的报文; 而在被动通知方式中, 辅助服务器定期地检查是否有任何变化。无论使用哪一种方式, 当得到变化的通知时, 辅助服务器会请求整个区域的信息 (区域传输)。

为了提供安全性以防止对DNS记录的非授权更改, DDNS可以使用鉴别机制。

25.10 封装

DNS可以使用UDP或者TCP协议。在这两种情况下, 服务器使用的熟知端口是53。当响应报文的长度小于512字节时, 就使用UDP。因为大多数UDP分组有512字节分组大小的限制。如果响应报文的长度大于512字节, 则必须使用TCP连接。在这种情况下, 可能会发生以下两种情况:

- 如果解析程序预先知道响应报文超过512字节, 那么它必须使用TCP连接。例如, 如果辅助名字服务器 (作为客户端) 需要从主服务器进行区域传输, 那么必须使用TCP连接。因为被传输的信息通常是超过512字节的。
- 如果解析程序不知道响应报文的大小, 那么可以使用UDP端口。但是, 如果响应报文超过512字节, 那么服务器会截断这一报文。此时解析程序会开启TCP连接, 并重复该请求, 从服务器中获得完整的响应。

DNS可以在熟知端口53使用UDP或者TCP服务。

推荐读物

为了深入讨论本章有关的主题, 推荐下面的读物和网站。在本章末列出方括号[...]中的参考资料。

读物

DNS分别在[AL98]、[For06]的17章、[PD03]的9.1节、[Tan03]的7.1节中有讨论。

网站

下面的网站与本章中讨论的主题有关:

www.intenic.net/关于注册机构的信息

www.ietf.org/rfc.html关于RFC的信息

请求评论

下面是与DNS相关的RFC:

799, 811, 819, 830, 881, 882, 883, 897, 920, 921, 1034, 1035, 1386, 1480, 1535, 1536, 1537, 1591, 1637, 1664, 1706, 1712, 1713, 1982, 2065, 2137, 2317, 2535, 2671
--

本章小结

- 域名系统 (DNS) 是一个客户端/服务器应用程序, 它用一个唯一的用户友好的名字标

识因特网上的每一台主机。

- DNS以层次结构来组织名字空间，以使涉及名字的各种任务分散化。
- DNS可以描述为一棵倒过来的层次结构的树，根节点位于最顶层，最多有128层。
- 树上的每一个节点都有一个域名。
- 一个域定义为域名空间的任何一棵子树。
- 名字空间信息分布在DNS的各个服务器中，每一台服务器对它的区域都具有管辖权。
- 根服务器的区域是整个DNS树。
- 主服务器创建、维护和更新它的区域信息。
- 辅助服务器从主服务器中获取信息。
- 因特网中的域名空间划分为三个部分：通用域、国家域和反向域。
- 共有14种通用域，每个域指明了一个组织类型。
- 每一个国家域指明了一个国家。
- 反向域根据一个给定的IP地址查找一个域名，这称为由地址到名字的解析。
- 名字服务器，即运行DNS服务器程序的计算机，是按层次结构组织的。
- DNS客户端称为解析程序，它将名字映射为地址或者将地址映射为名字。
- 在递归解析中，客户端将它的请求发送到服务器，服务器最终返回一个应答。
- 在迭代解析中，客户端在获得一个应答之前可能会把请求发送给多台服务器。
- 高速缓存是一种将查询的应答存放在存储器中（存放有限的时间）以便以后的请求方便使用的方法。
- 全称域名（FQDN）是一个域名，该域名由主机标号以及向上到达根节点所经过的各级节点的标号组成。
- 部分域名（PQDN）是一种域名，它不全部包括主机到根节点之间所有层次的域名。
- DNS报文有两种类型：查询和响应。
- DNS记录有两种类型：询问记录和资源记录。
- 动态DNS（DDNS）自动更新DNS主文件。
- DNS在报文长度小于512字节时，使用UDP服务；否则，使用TCP。

习题

复习题

1. 对于因特网这样规模的系统，层次结构的名称空间比平面结构的名称空间有哪些优势？
2. 主服务器与辅助服务器之间有什么区别？
3. 域名空间有哪三个域？
4. 反向域的用途是什么？
5. 递归解析与迭代解析有什么不同？
6. 什么是全称域名（FQDN）？
7. 什么是部分域名（PQDN）？
8. 什么是区域？
9. 高速缓存是如何提高名称解析的效率？
10. DNS报文主要分为哪两种类型？
11. 为什么需要DNS？

练习题

12. 下列哪些是全称域名 (FQDN)，哪些是部分域名 (PQDN)？
 - a. xxx
 - b. xxx.yyy.
 - c. xxx.yyy.net
 - d. zzz.yyy.xxx.edu
13. 下列哪一个是全称域名 (FQDN)，哪一个是部分域名 (PQDN)？
 - a. mil.
 - b. edu.
 - c. xxx.yyy.net
 - d. zzz.yyy.xxx.edu
14. 你的系统使用的是哪一个域，通用域还是国家域？
15. 为什么需要DNS系统？什么情况下我们可以直接使用IP地址？
16. 为了查找到目的IP地址，需要DNS服务。DNS需要UDP或者TCP服务，UDP或者TCP需要IP服务，IP需要目的地址，这是一个恶性循环吗？
17. 如果一个DNS域名是 *voyager.fhda.edu*，那么它使用了多少个标号？有多少个层次？
18. 一个部分域名 (PQDN) 必须比对应的全称域名 (FQDN) 短吗？
19. 一个域名是 *hello.customer.info*。这是一个通用域还是一个国家域？
20. 你认为一个递归解析通常会比迭代解析快吗？试解释之。
21. 一条查询报文只有一个询问部分，而对应的响应报文却有几个应答部分，可能出现这种情况吗？

第26章 远程登录、电子邮件与文件传输

因特网的主要任务是向用户提供服务。最流行的应用是远程登录、电子邮件和文件传输。本章讨论这三种应用。我们将在第27章讨论因特网的另一个普遍的应用，访问万维网。

26.1 远程登录

在因特网中，用户希望能够在—个远程网站上运行多个应用程序，而产生的结果能够传送到本地网站。例如，学生们希望能够从他们的家里连接到校园计算机实验室，以便访问应用程序来做家庭作业或完成项目。满足这种需求的一种方法是对每一种需要的服务创建不同的客户/服务器应用程序。现在可用的应用程序有文件传送程序（FTP）和电子邮件等。但是，要对每一个需求编写一个特定客户/服务器程序是不可能的。

更好的方法是使用通用的客户/服务器程序，它让用户能够访问远程计算机上的任何应用程序。换言之，允许用户在远程计算机上登录。在登录后，用户可使用远程计算机提供的服务，并将结果返回到本地计算机上。

TELNET

在这一节中，我们讨论这样一个客户/服务器应用程序TELNET，TELNET是终端网络（Terminal NETwork）的缩写。它是国际标准化组织（ISO）建议的虚终端服务的标准TCP/IP协议。TELNET使用户能够建立一个到远程系统的连接，就好像本地终端连接到远程系统上—样。

TELNET是一个通用的客户/服务器应用程序。

分时环境

在TELNET设计出来的时候，多数操作系统（如UNIX）工作在分时（timesharing）的环境下。在这样的环境中，—个大型计算机支持许多个用户。用户计算机通过—个终端产生交互，这种终端通常是由键盘、监视器和鼠标组成，甚至—个微机也能够用—个终端仿真程序来模拟—个终端。

登录

在分时环境下，用户是系统的一部分，并具有使用资源的某些权利。每—被授权的用户都有—个标识符，也可能还有—个口令。用户标识符定义用户是系统—部分。要访问系统，用户要使用用户标识符或用户登录名登录到系统。系统还要进行口令检查以防止非授权用户使用资源。图26.1表示了登录过程。

当用户登录到本地的分时系统时，它称为本地登录（local log-in）。用户在终端上键入时，或在工作站上运行终端仿真程序时，他的击键就被终端驱动程序接受。终端驱动程序将字符传递给操作系统。操作系统解释字符的组合，并调用所需的应用程序或实用程序。

当用户想访问远程机器上—个应用程序或实用程序时，他就需要进行远程登录（remote log-in），此时就需要使用TELNET客户程序和服务器程序。用户将其击键发送给终端驱动程序，同时本地操作系统接受这些字符，但并不解释它们。这些字符被送到TELNET客户机，

它将这些字符转换成称为网络虚拟终端（NVT）字符的通用字符集，然后将其传送给本地TCP/IP协议堆栈。

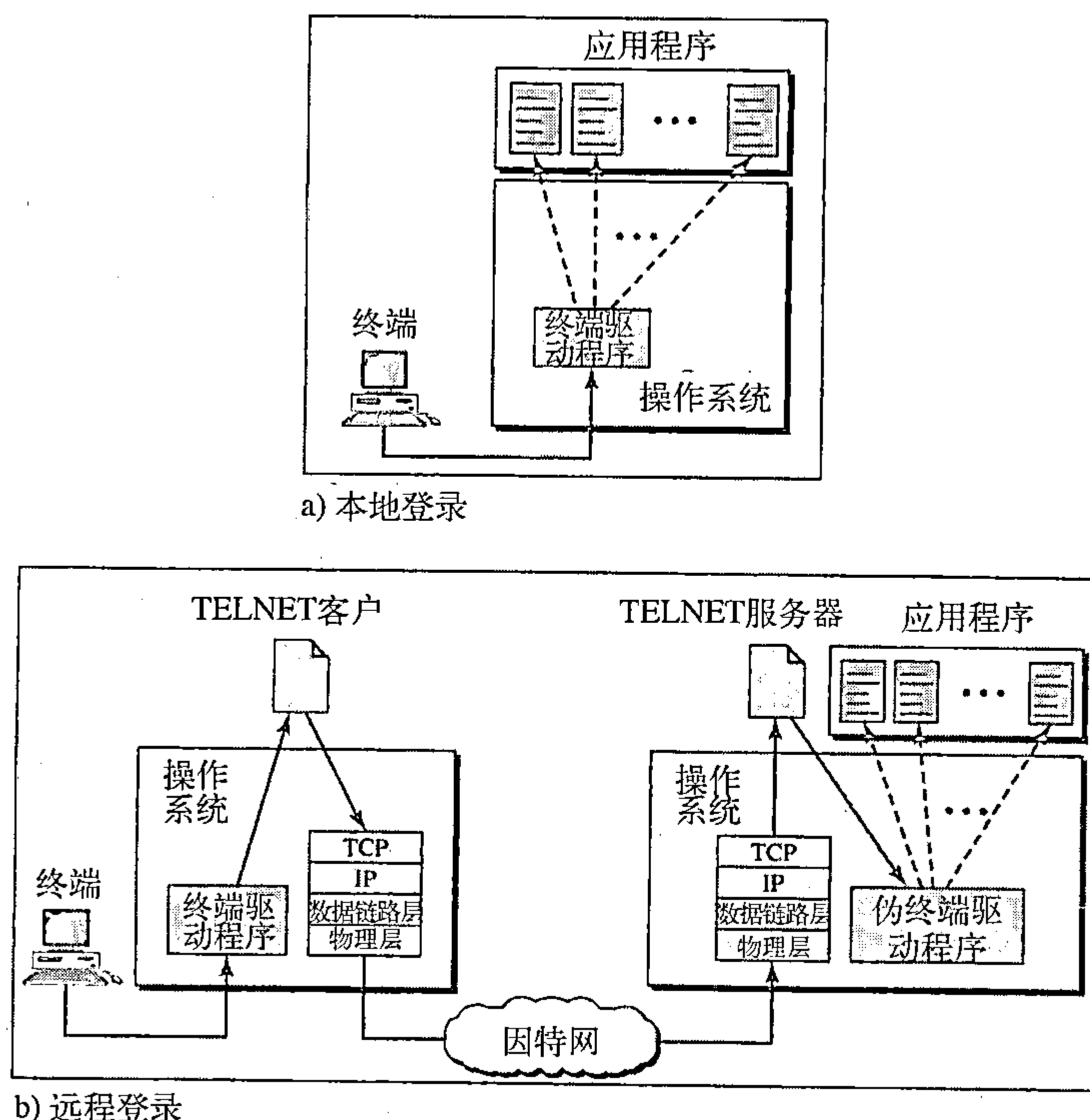


图26.1 本地和远程的登录过程

采用网络虚拟终端（NVT）形式的命令或文本通过因特网传送到远程机器的TCP/IP堆栈，在那里字符传递给操作系统，然后传送给TELNET服务器，TELNET服务器将这些字符转换成远程计算机可以理解的字符。但是，这些字符不能直接传送给操作系统，因为远程的操作系统不能接收来自TELNET服务器的字符，它只能接收来自终端驱动程序的字符。解决方法是增加一个称为伪终端驱动程序的软件块，它将这些字符伪装成好像是从一个终端发来的，然后操作系统将这些字符传送给适当的应用程序。

网络虚拟终端

访问一个远程计算机的过程是很复杂的。这是因为每一个计算机以及其操作系统都接受一个特殊的字符组合作为一个记号。例如，在运行DOS操作系统的计算机中，文件结束标记是Ctrl+z，但在UNIX操作系统中，则是Ctrl+d。

我们现在是和异构系统打交道。如果我们想要访问世界上的任何远程计算机，那么我们必须先知道将要连接的计算机是什么类型，我们还必须安装那个计算机所用的终端仿真程序。TELNET解决这个问题的方法是定义一个通用的接口，称为网络虚拟终端（network virtual terminal, NVT）字符集。通过这个接口，客户TELNET将来自本地终端的字符（数据或命令）转换成NVT形式，然后传递给网络。而服务器TELNET将来自NVT形式的数据或命令转换成远程计算机可接受的形式。图26.2表示了这一概念。

NVT字符集 NVT使用二个字符集：一个是数据字符，另一个是控制字符。两者都是8位的字符集。对于数据，它的7个最低位与ASCII是一样的，而最高位是0。在计算机之间发送（从服务器到客户机或从客户到服务器）控制字符（control character），NVT使用一个8位的字